Diffusion-based Generative Models. Lectures.

Denis Rakitin
HSE University [1]

Lecture 1. Discrete-time Diffusion Models

We start by constructing diffusion models in discrete time. The main ideology behind diffusion models consists in defining the *forward noising process* that takes an object $\mathbf{X}_0$ from the data distribution p^{data} and gradually transforms it into pure noise that has a simple tractable distribution (e.g. standard normal $\mathcal{N}(0, I)$) and does not contain almost any information about the initial data. If we manage to construct a tractable *backward* process that is (in some sense) the time-reversal of the defined forward process, we will obtain a sampling scheme of gradually transforming noise into data. Throughout the course we will mainly work with the two standard options of the forward processes.

Forward noising processes

Definition. The process $(\mathbf{X}_t, t \in \{0, \dots, T\})$ that starts from the data distribution $\mathbf{X}_0 \sim p^{\text{data}}$ and performs its transitions as

$$\mathbf{X}_t = \mathbf{X}_{t-1} + g(t)\boldsymbol{\varepsilon}_t, \tag{1}$$

where $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(0, I)$ is independent from $\mathbf{X}_0, \dots, \mathbf{X}_{t-1}$ is called the **Variance Exploding (VE)** process.

Definition. The process $(\mathbf{X}_t, t \in \{0, \dots, T\})$ that starts from the data distribution $\mathbf{X}_0 \sim p^{\text{data}}$ and performs its transitions as

$$\mathbf{X}_t = \sqrt{1 - \beta_t}\mathbf{X}_{t-1} + \sqrt{\beta_t}\boldsymbol{\varepsilon}_t, \tag{2}$$

where $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(0, I)$ is independent from $\mathbf{X}_0, \dots, \mathbf{X}_{t-1}$ is called the **Variance Preserving (VP)** process.

Names of the processes speak for themselves: each step of the **VE** process consists in simply adding an independent Gaussian noise with some pre-defined variance schedule

[1] Higher School of Economics, Moscow, Russia.

$g^2(t)$. The variance schedule is defined in such a way that the magnitude of the signal $\mathbf{X}_0$ is negligible compared to the variance accumulated during the process. The **VP** process, on the other hand, performs progressive mixing of the object with Gaussian distribution by deleting part of the object (performing multiplication by a value less than 1) and replacing it with the noise. The name **Variance Preserving** stands for the fact that if $\operatorname{Var} \mathbf{X}_0 = I$ then $\operatorname{Var} \mathbf{X}_t = I$ for all $t = 0, \ldots, T$.

We continue with several important observations about the defined noising processes. One of them is the *Markov property*:

Definition. A process $(\mathbf{X}_t, t \in \{0, \ldots, T\})$ is called Markov/a Markov chain (the joint distribution $p_{\mathbf{X}_0,\ldots,\mathbf{X}_T}$ has the Markov property) if for all t holds

$$p_{\mathbf{X}_t|\mathbf{X}_{t-1},\ldots\mathbf{X}_0}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}, \ldots, \boldsymbol{x}_0) = p_{\mathbf{X}_t|\mathbf{X}_{t-1}}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}), \tag{3}$$

which informally means that the «future» $\mathbf{X}_t$ is independent with the «past» $\mathbf{X}_{t-2}, \ldots, \mathbf{X}_0$ given the value of the «present» $\mathbf{X}_{t-1}$.

Claim. *The Markov property holds for all processes of the form* $\mathbf{X}_t = a_t\mathbf{X}_{t-1} + b_t\boldsymbol{\varepsilon}_t$, *where* $\boldsymbol{\varepsilon}_t$ *is independent from all the previous history* $\mathbf{X}_0, \ldots, \mathbf{X}_{t-1}$.

Proof. Let us rewrite $\mathbf{X}_t$ in the joint density using the definition:

$$p_{\mathbf{X}_t|\mathbf{X}_{t-1},\ldots,\mathbf{X}_0}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}, \ldots, \boldsymbol{x}_0) = p_{a_t\mathbf{X}_{t-1}+b_t\boldsymbol{\varepsilon}_t|\mathbf{X}_{t-1},\ldots,\mathbf{X}_0}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}, \ldots, \boldsymbol{x}_0). \tag{4}$$

The condition gives us a possibility to replace $\mathbf{X}_{t-1}$ with $\boldsymbol{x}_{t-1}$ and obtain

$$p_{a_t\boldsymbol{x}_{t-1}+b_t\boldsymbol{\varepsilon}_t|\mathbf{X}_{t-1},\ldots,\mathbf{X}_0}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}, \ldots, \boldsymbol{x}_0). \tag{5}$$

Here, the value $a_t\boldsymbol{x}_{t-1} + b_t\boldsymbol{\varepsilon}_t$ is independent with the previous history since $\boldsymbol{x}_t$ is constant and $\boldsymbol{\varepsilon}_t$ is independent with it. Hense, the conditional density is equal to the unconditional $p_{a_t\boldsymbol{x}_{t-1}+b_t\boldsymbol{\varepsilon}_t}(\boldsymbol{x}_t)$ and we obtain

$$p_{\mathbf{X}_t|\mathbf{X}_{t-1},\ldots,\mathbf{X}_0}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}, \ldots, \boldsymbol{x}_0) = p_{a_t\boldsymbol{x}_{t-1}+b_t\boldsymbol{\varepsilon}_t}(\boldsymbol{x}_t). \tag{6}$$

At the same time, all the transitions hold if we replace the condition $\mathbf{X}_{t-1}, \ldots, \mathbf{X}_0$ with only the current state $\mathbf{X}_{t-1}$:

$$p_{\mathbf{X}_t|\mathbf{X}_{t-1}}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = p_{a_t\mathbf{X}_{t-1}+b_t\boldsymbol{\varepsilon}_t}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = p_{a_t\boldsymbol{x}_{t-1}+b_t\boldsymbol{\varepsilon}_t}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}) = p_{a_t\boldsymbol{x}_{t-1}+b_t\boldsymbol{\varepsilon}_t}(\boldsymbol{x}_t), \tag{7}$$

which implies the desired $p_{\mathbf{X}_t|\mathbf{X}_{t-1},\ldots,\mathbf{X}_0}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1}, \ldots, \boldsymbol{x}_0) = p_{\mathbf{X}_t|\mathbf{X}_{t-1}}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})$. □

We then deduce that both VE and VP processes are Markov chains, the property we will heavily use in the derivations of diffusion models. Next, it is important to find how the

statistics of the process evolve through time. For this purpose we propose to calculate the transitional distributions $p_{\mathbf{X}_t|\mathbf{X}_0}$ directly from $\mathbf{X}_0$ to $\mathbf{X}_t$ instead of the pre-defined sequential transitions $p_{\mathbf{X}_t|\mathbf{X}_{t-1}}$.

Calculating transitional density $p_{\mathbf{X}_t|\mathbf{X}_0}$ given the initial point $\mathbf{X}_0 = \boldsymbol{x}_0$ is equivalent to calculating the unconditional density $p_{\mathbf{Y}_t}$ of the Markov chain $\mathbf{Y}_t$ that starts at $\boldsymbol{x}_0$ and has the same transitions $p_{\mathbf{Y}_t|\mathbf{Y}_{t-1}} = p_{\mathbf{X}_t|\mathbf{X}_{t-1}}$. In both VE and VP cases the process $\mathbf{Y}_t$ has the form

$$\mathbf{Y}_t = \boldsymbol{x}_0 + \sum_{s=1}^{t} a_s \boldsymbol{\varepsilon}_s, \tag{8}$$

where $\boldsymbol{\varepsilon}_s$ are independent $\mathcal{N}(0, I)$. The sum of independent Gaussian variables is also Gaussian (see Section A.3), which means one only has to calculate their mean and variance. For the VE process we have:

$$\mathbb{E}\mathbf{Y}_t = \mathbb{E}\left[\mathbf{Y}_{t-1} + g(t)\boldsymbol{\varepsilon}_t\right] = \mathbb{E}\mathbf{Y}_{t-1}; \tag{9}$$

$$\mathrm{Var}\mathbf{Y}_t = \mathrm{Var}\left(\mathbf{Y}_{t-1} + g(t)\boldsymbol{\varepsilon}_t\right) = \mathrm{Var}\mathbf{Y}_{t-1} + g^2(t)\mathrm{Var}\,\boldsymbol{\varepsilon}_t = \mathrm{Var}\mathbf{Y}_{t-1} + g^2(t)I. \tag{10}$$

Solving this simple recursion with the initial condition $\mathbb{E}\mathbf{Y}_0 = \boldsymbol{x}_0$ and $\mathrm{Var}\mathbf{Y}_0 = 0$

$$p^{\mathrm{VE}}_{\mathbf{X}_t|\mathbf{X}_0}(\boldsymbol{x}_t|\boldsymbol{x}_0) = p^{\mathrm{VE}}_{\mathbf{Y}_t}(\boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{x}_t|\boldsymbol{x}_0, \sigma_t^2 I), \tag{11}$$

where $\sigma_t^2 = \sum_{s=1}^{t} g^2(s)$ is the accumulated variance.

VP process is a little more complicated:

$$\mathbb{E}\mathbf{Y}_t = \mathbb{E}\left[\sqrt{1-\beta_t}\mathbf{Y}_{t-1} + \sqrt{\beta_t}\boldsymbol{\varepsilon}_t\right] = \mathbb{E}\sqrt{1-\beta_t}\mathbf{Y}_{t-1} = \sqrt{1-\beta_t}\,\mathbb{E}\mathbf{Y}_{t-1}; \tag{12}$$

$$\mathrm{Var}\mathbf{Y}_t = \mathrm{Var}\left(\sqrt{1-\beta_t}\mathbf{Y}_{t-1} + \sqrt{\beta_t}\boldsymbol{\varepsilon}_t\right) = (1-\beta_t)\mathrm{Var}\mathbf{Y}_{t-1} + \beta_t I \iff \tag{13}$$

$$\iff I - \mathrm{Var}\mathbf{Y}_t = (1-\beta_t)I - (1-\beta_t)\mathrm{Var}\mathbf{Y}_{t-1} = (1-\beta_t)(I - \mathrm{Var}\mathbf{Y}_t); \tag{14}$$

Solving the recursion with the initial condition $\mathbb{E}\mathbf{Y}_0 = \boldsymbol{x}_0$ and $I - \mathrm{Var}\mathbf{Y}_0 = I$, we obtain

$$p^{\mathrm{VP}}_{\mathbf{X}_t|\mathbf{X}_0}(\boldsymbol{x}_t|\boldsymbol{x}_0) = \mathcal{N}\left(\boldsymbol{x}_t|\sqrt{\alpha_t}\boldsymbol{x}_0, (1-\alpha_t)I\right), \tag{15}$$

where $\alpha_t = \prod_{s=1}^{t}(1-\beta_s)$.

Since sampling a value of the process at time t amounts to taking the data point $\mathbf{X}_0$ and then sampling from the conditional distribution $p_{\mathbf{X}_t|\mathbf{X}_0}$, one can obtain $\mathbf{X}_t$ by calculating the corresponding weighted sum with one independent Gaussian variable. This is a desirable property that allows one to sample any point of the process without generating the whole sequence, which will further be very useful for training.

Finally, our observations allow us to verify convergence of both processes to a simple tractable distribution. In the VE case, for large enough σ_T (e.g. 80.0 used in [13]) one could ignore the data point in the formula $\mathbf{X}_0 + \sigma_T \boldsymbol{\varepsilon}$ due to its negligible magnitude. In the VP case one can see that for an appropriate choice of β_t α_t is a product of variables less than 1, which converges to 0. We illustrate the main properties of the processes in Table 1.

	VE process	VP process
Transition $p_{t\|t-1}$	$\mathbf{X}_t = \mathbf{X}_{t-1} + g(t)\boldsymbol{\varepsilon}_t$	$\mathbf{X}_t = \sqrt{1-\beta_t}\mathbf{X}_{t-1} + \sqrt{\beta_t}\boldsymbol{\varepsilon}_t$
Marginal p_t	$\mathbf{X}_t = \mathbf{X}_0 + \sigma_t \boldsymbol{\varepsilon}$ $\sigma_t^2 = \sum_{s=1}^{t} g^2(s)$	$\mathbf{X}_t = \sqrt{\alpha_t}\,\mathbf{X}_0 + \sqrt{1-\alpha_t}\,\boldsymbol{\varepsilon}$ $\alpha_t = \prod_{s=1}^{t}(1-\beta_s)$
Endpoint distribution p_T	$\approx \mathcal{N}(0, \sigma_T^2)$	$\approx \mathcal{N}(0, I)$

Table 1: Basic properties of the noising processes: endpoint distribution and sampling from marginal/transitional densities.

Despite a different approach to replacing data with noise, the processes are closely connected:

Exercise. Prove that if $(\mathbf{X}_t)$ is a VE process then there exists such a sequence a_t that $(\mathbf{Y}_t)$ defined by $\mathbf{Y}_t = a_t \mathbf{X}_t$ is a VP process. Derive the connection between α_t and σ_t.

This observation means that almost in all of the cases both processes are interchangeable. And this is indeed an expected property: scaling the process at each noising step is equivalent to successively adding noise and then properly scaling the process.

Model and training

At this point, we have constructed two versions of the forward noising process. Given the data set of samples, it is easy to generate the forward process $\mathbf{X}_0, \dots, \mathbf{X}_T$ by performing consecutive sampling from the forward transition distributions $p_{\mathbf{X}_t|\mathbf{X}_{t-1}}$. If we manage to find the backward transition distributions $p_{\mathbf{X}_{t-1}|\mathbf{X}_t,\dots,\mathbf{X}_T}$, we will have the possibility to generate data samples by successively performing backward steps since the endpoint distribution $p_{\mathbf{X}_T}$ is tractable. Here, the Markov property of the forward process makes this task a little bit easier by ensuring the Markov dependency between samples in the backward process as well:

Exercise. Let $\mathbf{X}_0, \dots, \mathbf{X}_T$ be a Markov chain. Then, $\mathbf{X}_T, \dots, \mathbf{X}_0$ is also a Markov chain:

$$p_{\mathbf{X}_{t-1}|\mathbf{X}_t,\dots,\mathbf{X}_T}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \dots, \boldsymbol{x}_T) = p_{\mathbf{X}_{t-1}|\mathbf{X}_t}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t). \tag{16}$$

This means it is enough for us to be able to sample from the corresponding transitions $p_{\mathbf{X}_{t-1}|\mathbf{X}_t}$. However, this set of distributions is still intractable. Due to the Bayes' theorem we can represent them as

$$p_{\mathbf{X}_{t-1}|\mathbf{X}_t}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \frac{p_{\mathbf{X}_t|\mathbf{X}_{t-1}}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})p_{\mathbf{X}_{t-1}}(\boldsymbol{x}_{t-1})}{p_{\mathbf{X}_t}(\boldsymbol{x}_t)}, \tag{17}$$

where $p_{\mathbf{X}_t|\mathbf{X}_{t-1}}$ is a simple Gaussian transition, but e.g.

$$p_{\mathbf{X}_t}(\boldsymbol{x}_t) = \int_{\mathbb{R}^d} p_{\mathbf{X}_t|\mathbf{X}_0}(\boldsymbol{x}_t|\boldsymbol{x}_0)p^{\text{data}}(\boldsymbol{x}_0)\mathrm{d}\boldsymbol{x}_0 \tag{18}$$

is the distribution of the noisy data, which is intractable (except for very high noise levels) since the data distribution itself is intractable.

At this moment, as almost always in Deep Learning, if something is necessary for the model but intractable, one should learn it! For this purpose, authors of the discrete-time diffusion models [38, 11] define the «learnable» backward process $\mathbf{X}_t^\theta$ to have the following form:

1. Sample the endpoint from the noise distribution: $\mathbf{X}_T^\theta \sim p_{\mathbf{X}_T}$;
2. Perform transitions of the form $\mathbf{X}_{t-1}^\theta \sim p_{t-1|t}^\theta(\cdot|\mathbf{X}_t^\theta)$, where the learnable backward transition probability is a Gaussian distribution with learnable mean and scalar variance:
$$p_{t-1|t}^\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = \mathcal{N}(\boldsymbol{x}_{t-1}|\mu_t^\theta(\boldsymbol{x}_t), \gamma_t^2 I), \tag{19}$$
or equivalently
$$\mathbf{X}_{t-1}^\theta = \mu_t^\theta(\mathbf{X}_t^\theta) + \gamma_t\boldsymbol{\varepsilon}_t, \tag{20}$$
where $\boldsymbol{\varepsilon}_t$ is independent from the history $\mathbf{X}_t^\theta, \ldots, \mathbf{X}_T^\theta$. In practice, we assume that $\mu_t^\theta(\boldsymbol{x}_t)$ is a neural network with weights θ that takes both t and $\boldsymbol{x}_t$ as its inputs.

Ideally, we want our learnable backward process to be as close as it can be to the true backward process $\mathbf{X}_T, \ldots, \mathbf{X}_0$. It is then a natural solution to minimize some measure of discrepancy between the two distributions, the most popular of which is the KL divergence.

Definition. Given the two random variables $\mathbf{Y}, \mathbf{Z} \in \mathbb{R}^d$ the KL divergence between their probability distributions is defined as

$$\mathrm{KL}(p_{\mathbf{Y}}\|p_{\mathbf{Z}}) = \int_{\mathbb{R}^d} p_{\mathbf{Y}}(\boldsymbol{y}) \log \frac{p_{\mathbf{Y}}(\boldsymbol{y})}{p_{\mathbf{Z}}(\boldsymbol{y})}\mathrm{d}\boldsymbol{y} = \mathbb{E} \log \frac{p_{\mathbf{Y}}(\mathbf{Y})}{p_{\mathbf{Z}}(\mathbf{Y})}. \tag{21}$$

KL divergence is not a metric between distributions in the proper sense (e.g. it is not symmetric and does not satisfy the triangle inequality). At the same time, it has two very important properties of a *divergence* that make it a reasonable candidate for the measure of distance:

Exercise. Prove that $\mathrm{KL}(p_{\mathbf{Y}}\|p_{\mathbf{Z}}) \geq 0$ and $\mathrm{KL}(p_{\mathbf{Y}}\|p_{\mathbf{Z}}) = 0$ if and only if $p_{\mathbf{Y}} = p_{\mathbf{Z}}$.

Our main goal is then to derive a meaningful and tractable optimization objective that will allow to learn the true backward transition probabilities and ensure $p^{\theta}_{t-1|t} = p_{\mathbf{X}_{t-1}|\mathbf{X}_t}$ if the objective is perfectly optimized. From this point on, we will denote the distribution of the «true» forward noising process as $p_{0,\dots,T}$ instead of $p_{\mathbf{X}_0,\dots,\mathbf{X}_T}$. The same goes for the trainable backward process, which distribution we will denote as $p^{\theta}_{0,\dots,T}$ instead of $p_{\mathbf{X}^{\theta}_0,\dots,\mathbf{X}^{\theta}_T}$. We will analogously denote all the possible conditional distributions by e.g. $p_{t|t-1}$ and $p^{\theta}_{t|t-1}$, respectively.

The basic functional that we will simplify is the KL divergence between distributions of the ground truth and the trainable processes:

$$\mathrm{KL}(p_{0,\dots,T}\|p^{\theta}_{0,\dots,T}) = \mathbb{E}\log\frac{p_{0,\dots,T}(\mathbf{X}_0,\dots,\mathbf{X}_T)}{p^{\theta}_{0,\dots,T}(\mathbf{X}_0,\dots,\mathbf{X}_T)} \to \min_{\theta}. \tag{22}$$

The reasonable approach to simplify this equation would be to decompose the joint densities into the chain of backward transitions (since this is essentially what we want to optimize) in the following way:

$$\mathbb{E}\log\frac{p_{0,\dots,T}(\mathbf{X}_0,\dots,\mathbf{X}_T)}{p^{\theta}_{0,\dots,T}(\mathbf{X}_0,\dots,\mathbf{X}_T)} = \mathbb{E}\log\frac{p_T(\mathbf{X}_T)\prod\limits_{t=1}^{T}p_{t-1|t}(\mathbf{X}_{t-1}|\mathbf{X}_t)}{p^{\theta}_T(\mathbf{X}_T)\prod\limits_{t=1}^{T}p^{\theta}_{t-1|t}(\mathbf{X}_{t-1}|\mathbf{X}_t)} = \tag{23}$$

$$= \mathrm{KL}(p_T\|p^{\theta}_T) + \sum_{t=1}^{T}\mathbb{E}\log\frac{p_{t-1|t}(\mathbf{X}_{t-1}|\mathbf{X}_t)}{p^{\theta}_{t-1|t}(\mathbf{X}_{t-1}|\mathbf{X}_t)} = \tag{24}$$

$$= \mathrm{KL}(p_T\|p^{\theta}_T) + \int_{\mathbb{R}^d}\left(\int_{\mathbb{R}^d}p_{t-1|t}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)\log\frac{p_{t-1|t}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)}{p^{\theta}_{t-1|t}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)}\mathrm{d}\boldsymbol{x}_{t-1}\right)p_t(\boldsymbol{x}_t)\mathrm{d}\boldsymbol{x}_t = \tag{25}$$

$$= \mathrm{KL}(p_T\|p^{\theta}_T) + \int_{\mathbb{R}^d}\mathrm{KL}\left(p_{t-1|t}(\cdot|\boldsymbol{x}_t)\|p^{\theta}_{t-1|t}(\cdot|\boldsymbol{x}_t)\right)p_t(\boldsymbol{x}_t)\mathrm{d}\boldsymbol{x}_t. \tag{26}$$

The ideological result of this derivation consists in observing that minimizing KL divergence between two Markov chains is equivalent to simultaneously minimizing the

KL divergence between their starting points and forward transitions (or, equivalently, their endpoints and backward transitions). However, it is still impossible to evaluate $\log p_{t-1|t}(\mathbf{X}_{t-1}|\mathbf{X}_t)$, since it implicitly contains information about the noisy data distribution p_t, so we need to find a tractable modification.

At this point, we will perform the trick that will follow us throughout the course: if some distribution or its characteristic is intractable, try to find the condition such that the conditional distribution is tractable! In our case, the only intractable part of the transitions $p_{t-1|t}$ is the data distribution. However, if we additionally condition it on the data point $\mathbf{X}_0$, we will obtain

$$p_{t-1|t,0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) = \frac{p_{t|t-1}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1})p_{t-1|0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_0)}{p_{t|0}(\boldsymbol{x}_t|\boldsymbol{x}_0)}, \tag{27}$$

where all three multipliers are tractable (and were previously derived). And this totally makes sense: performing a «denoising» step from t to $t-1$ given the original image from the data set amounts to adjust the noise level, while keeping the signal. Essentially, finding $p_{t-1|t,0}$ is equivalent to finding the conditional distribution of one component of the Gaussian vector given the other component (see Appendix A.3), which produces the Gaussian distribution of the form

$$p_{t-1|t,0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_{t-1}|A_t\boldsymbol{x}_t + B_t\boldsymbol{x}_0, C_t^2 I). \tag{28}$$

Exercise. Prove that in case of the VE/VP processes the conditional backward transitional distribution has the aforementioned form with

$$A_t = \frac{\sqrt{1-\beta_t}(1-\alpha_{t-1})}{1-\alpha_t}; \quad B_t = \frac{\sqrt{\alpha_{t-1}}\beta_t}{1-\alpha_t}; \quad C_t^2 = \frac{1-\alpha_{t-1}}{1-\alpha_t}\beta_t \tag{29}$$

for the VP process and

$$A_t = \frac{\sigma_{t-1}^2}{\sigma_t^2}; \quad B_t = 1 - \frac{\sigma_{t-1}^2}{\sigma_t^2}; \quad C_t^2 = \frac{\sigma_{t-1}^2}{\sigma_t^2}g^2(t) \tag{30}$$

for the VE process.

We then modify our decomposition of the forward process from $p_T \prod_{t=1}^T p_{t-1|t}$ to the form that will include $p_{t-1|t,0}$ instead. To this end, we will use the following fact:

Exercise. Let the joint distribution $p_{\mathbf{X}_0,\ldots,\mathbf{X}_T}$ have the Markov property. Then distribution of the same process conditioned on the end points is also Markov. In particular,

$$p_{\mathbf{X}_0,\ldots,\mathbf{X}_T}(\boldsymbol{x}_0,\ldots,\boldsymbol{x}_T) = p_{\mathbf{X}_0,\mathbf{X}_T}(\boldsymbol{x}_0,\boldsymbol{x}_T)\prod_{t=1}^{T-1} p_{\mathbf{X}_t|\mathbf{X}_{t-1},\mathbf{X}_T}(\boldsymbol{x}_t|\boldsymbol{x}_{t-1},\boldsymbol{x}_T) = \tag{31}$$

$$= p_{\mathbf{X}_0,\mathbf{X}_T}(\boldsymbol{x}_0,\boldsymbol{x}_T)\prod_{t=2}^{T} p_{\mathbf{X}_{t-1}|\mathbf{X}_t,\mathbf{X}_0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0), \tag{32}$$

which means that if we decompose the conditioned process into the forward transitions, we only need to add the condition on $\mathbf{X}_T$. If we decompose it into the backward transitions, we only need to add the condition on $\mathbf{X}_0$.

Denoising Diffusion Probabilistic Models

Using this conditional decomposition, we perform the analogous manipulations with the KL divergence:

$$\mathbb{E}\log\frac{p_{0,\dots,T}(\mathbf{X}_0,\dots,\mathbf{X}_T)}{p^\theta_{0,\dots,T}(\mathbf{X}_0,\dots,\mathbf{X}_T)} = \mathbb{E}\log\frac{p_{0,T}(\mathbf{X}_0,\mathbf{X}_T)\prod\limits_{t=2}^{T}p_{t-1|t,0}(\mathbf{X}_{t-1}|\mathbf{X}_t,\mathbf{X}_0)}{p^\theta_T(\mathbf{X}_T)\prod\limits_{t=2}^{T}p^\theta_{t-1|t}(\mathbf{X}_{t-1}|\mathbf{X}_t)\cdot p^\theta_{0|1}(\mathbf{X}_0|\mathbf{X}_1)}. \tag{33}$$

Here we note that $p_{0,T}$ does not depend on θ. The starting distribution of the backward process p^θ_T also does not depend on θ since this is the distribution of the pure noise. Finally, $p^\theta_{0|1}$ actually depends on θ, but in the case of hundreds or even thousands of transitions one could say that the last transition does not almost change the object, so $p^\theta_{0|1}(\boldsymbol{x}_0|\boldsymbol{x}_1)\approx I\{\boldsymbol{x}_0=\boldsymbol{x}_1\}$ and its dependence on θ is negligible. We then approximate the KL divergence (without the aforementioned constants) as

$$\mathbb{E}\log\frac{\prod\limits_{t=2}^{T}p_{t-1|t,0}(\mathbf{X}_{t-1}|\mathbf{X}_t,\mathbf{X}_0)}{\prod\limits_{t=2}^{T}p^\theta_{t-1|t}(\mathbf{X}_{t-1}|\mathbf{X}_t)} = \tag{34}$$

$$=\sum_{t=2}^{T}\int_{(\mathbb{R}^d)^3}p_{0,t-1,t}(\boldsymbol{x}_0,\boldsymbol{x}_{t-1},\boldsymbol{x}_t)\log\frac{p_{t-1|t,0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)}{p^\theta_{t-1|t}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)}\mathrm{d}\boldsymbol{x}_0\mathrm{d}\boldsymbol{x}_{t-1}\mathrm{d}\boldsymbol{x}_t = \tag{35}$$

$$=\sum_{t=2}^{T}\int_{(\mathbb{R}^d)^2}p_{0,t}(\boldsymbol{x}_0,\boldsymbol{x}_t)\left(\int_{\mathbb{R}^d}p_{t-1|t,0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)\log\frac{p_{t-1|t,0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0)}{p^\theta_{t-1|t}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t)}\mathrm{d}\boldsymbol{x}_{t-1}\right)\mathrm{d}\boldsymbol{x}_0\mathrm{d}\boldsymbol{x}_t = \tag{36}$$

$$=\sum_{t=2}^{T}\int_{(\mathbb{R}^d)^2}p_{0,t}(\boldsymbol{x}_0,\boldsymbol{x}_t)\,\mathrm{KL}\left(p_{t-1|t,0}(\cdot|\boldsymbol{x}_t,\boldsymbol{x}_0)\,\|\,p^\theta_{t-1|t}(\cdot|\boldsymbol{x}_t)\right)\mathrm{d}\boldsymbol{x}_0\mathrm{d}\boldsymbol{x}_t. \tag{37}$$

We have thus obtained an equivalent optimization functional

$$\sum_{t=2}^{T}\mathbb{E}\,\mathrm{KL}\left(p_{t-1|t,0}(\cdot|\mathbf{X}_t,\mathbf{X}_0)\,\|\,p^\theta_{t-1|t}(\cdot|\mathbf{X}_t)\right)\to\min_\theta. \tag{38}$$

At this point, all the parts of the optimized objective are either tractable or depend on the trained neural network. In particular, both $p_{t-1|t,0}$ and $p^\theta_{t-1|t}$ are multivariate Gaussian distributions. The KL divergence in this case has a closed form:

Exercise. Let $\mathbf{Y} \sim \mathcal{N}(\mu_\mathbf{Y}, \Sigma_\mathbf{Y})$ and $\mathbf{Z} \sim \mathcal{N}(\mu_\mathbf{Z}, \Sigma_\mathbf{Z})$ be d-dimensional Gaussian vectors. Then,

$$\mathrm{KL}(p_\mathbf{Y} \,\|\, p_\mathbf{Z}) = \frac{1}{2}\left(\mathrm{Tr}\left(\Sigma_\mathbf{Z}^{-1}\Sigma_\mathbf{Y}\right) + \log\frac{\det\Sigma_\mathbf{Z}}{\det\Sigma_\mathbf{Y}} + (\mu_\mathbf{Z}-\mu_\mathbf{Y})^\top\Sigma_\mathbf{Z}^{-1}(\mu_\mathbf{Z}-\mu_\mathbf{Y}) - d\right). \quad (39)$$

In particular, if $\Sigma_\mathbf{Y} = \sigma^2_\mathbf{Y} I$ and $\Sigma_\mathbf{Z} = \sigma^2_\mathbf{Z} I$, then

$$\mathrm{KL}(p_\mathbf{Y} \,\|\, p_\mathbf{Z}) = \frac{1}{2}\left(d\cdot\frac{\sigma^2_\mathbf{Y}}{\sigma^2_\mathbf{Z}} + d\cdot\log\frac{\sigma^2_\mathbf{Z}}{\sigma^2_\mathbf{Y}} + \frac{1}{\sigma^2_\mathbf{Z}}\|\mu_\mathbf{Z}-\mu_\mathbf{Y}\|^2 - d\right). \quad (40)$$

Remember that in our case

$$p_{t-1|t,0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t,\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_{t-1}|A_t\boldsymbol{x}_t + B_t\boldsymbol{x}_0, C_t^2 I); \quad p^\theta_{t-1|t}(\boldsymbol{x}_{t-1}|\mu^\theta_t(\boldsymbol{x}_t), \gamma_t^2 I). \quad (41)$$

We then obtain

$$\mathrm{KL}(p_{t-1|t,0}(\cdot|\boldsymbol{x}_t,\boldsymbol{x}_0) \,\|\, p^\theta_{t-1|t}(\cdot|\boldsymbol{x}_t)) = \mathrm{const} + \frac{1}{2\gamma_t^2}\left\|\mu^\theta_t(\boldsymbol{x}_t) - (A_t\boldsymbol{x}_t + B_t\boldsymbol{x}_0)\right\|^2, \quad (42)$$

which means that the optimized functional essentially pushes the neural network $\mu^\theta_t(\boldsymbol{x}_t)$ that represents the mean of the backward transition $p^\theta_{t-1|t}$ to be as close as possible to the mean $A_t\boldsymbol{x}_t + B_t\boldsymbol{x}_0$ of the conditional backward transition $p_{t-1|t,0}$. It is then common to perform a little different parameterization of the neural network and set $\mu^\theta_t(\boldsymbol{x}_t) = A_t\boldsymbol{x}_t + B_t\, D^\theta_t(\boldsymbol{x}_t)$ along with $\gamma_t^2 = C_t^2$, which mimics the conditional backward transition, but replaces the unknown $\boldsymbol{x}_0$ with its prediction by the neural network $D^\theta_t(\boldsymbol{x}_t)$. In short, we set

$$p^\theta_{t-1|t}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = p_{t-1|t,0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, D^\theta_t(\boldsymbol{x}_t)). \quad (43)$$

Then, the KL divergence is equal to

$$\mathrm{KL}(p_{t-1|t,0}(\cdot|\boldsymbol{x}_t,\boldsymbol{x}_0) \,\|\, p^\theta_{t-1|t}(\cdot|\boldsymbol{x}_t)) = \mathrm{const} + \frac{B_t^2}{2C_t^2}\|D^\theta_t(\boldsymbol{x}_t) - \boldsymbol{x}_0\|^2, \quad (44)$$

which means that the original objective is equal to (up to the additive constants)

$$\sum_{t=2}^{T}\omega_t\,\mathbb{E}\|D^\theta_t(\mathbf{X}_t) - \mathbf{X}_0\|^2 \to \min_\theta \quad (45)$$

	Parameters
General	Forward process $p_{t\|t-1}$, parameterization $p^\theta_{t-1\|t}$
VE	Variance schedule defined by $g(t)$ or σ_t^2
VP	Noising schedule defined by β_t or α_t
	Training
General	$\sum_{t=2}^T \mathbb{E}\,\mathrm{KL}\left(p_{t-1\|t,0}(\cdot\|\mathbf{X}_t,\mathbf{X}_0)\,\|\,p^\theta_{t-1\|t}(\cdot\|\mathbf{X}_t)\right)\to\min_\theta$
VP / VE	$\sum_{t=2}^T \omega_t\,\mathbb{E}\|D_t^\theta(\mathbf{X}_t)-\mathbf{X}_0\|^2\to\min_\theta$
	Sampling transition
General	$\mathbf{X}^\theta_{t-1}\sim p^\theta_{t-1\|t}(\cdot\|\mathbf{X}^\theta_t)$;
VE	$\mathbf{X}^\theta_{t-1}=\frac{\sigma^2_{t-1}}{\sigma_t^2}\mathbf{X}^\theta_t+\left(1-\frac{\sigma^2_{t-1}}{\sigma_t^2}\right)D_t^\theta(\mathbf{X}^\theta_t)+\frac{\sigma_{t-1}}{\sigma_t}g(t)\boldsymbol{\varepsilon}_t$;
VP	$\mathbf{X}^\theta_{t-1}=\frac{\sqrt{1-\beta_t}(1-\alpha_{t-1})}{1-\alpha_t}\mathbf{X}^\theta_t+\frac{\sqrt{\alpha_{t-1}}\beta_t}{1-\alpha_t}D_t^\theta(\mathbf{X}^\theta_t)+\sqrt{\frac{1-\alpha_{t-1}}{1-\alpha_t}\beta_t}\,\boldsymbol{\varepsilon}_t$

Table 2: Summarization of discrete-time diffusion models.

with some weights $\omega_t = B_t^2/(2C_t^2)$, determined by the forward process. Essentially, optimizing the KL divergence between the true and the learned processes amounts to training a *denoiser* network $D_t^\theta(\boldsymbol{x}_t)$ that tries to predict the clean image $\mathbf{X}_0$ given its noisy versions $\mathbf{X}_t$ from different noise levels $t = 2,\dots,T$.

In practice, one cannot directly calculate the expected value, so each summand is approximated with its Monte Carlo estimate using the samples from the dataset. Analogously, calculating the L_2 denoising loss for all (hundreds/thousands) of timesteps is too expensive for performing just one gradient update. Instead, one takes a batch of clean images and samples one time step per object. Thus, performing one gradient step amounts to

1. Sample a batch $\mathbf{X}_0^{(1)},\dots,\mathbf{X}_0^{(b)}$ of clean objects from the data set;
2. Sample a batch of time steps $\tau^{(1)},\dots,\tau^{(b)}$ from a distribution proportional to the weights ω_t; [2]
3. Sample a batch of standard normal variables $\boldsymbol{\varepsilon}^{(1)},\dots,\boldsymbol{\varepsilon}^{(b)}$ and perform the one-step

[2]In fact, weights do not affect the theoretical optimum of the functional. Thus, in practice one usually defines the sampling schedule not necessarily using the theoretical KL weights but according to the empirical research on the matter.

noising of images:

$$\mathbf{X}_{\text{noisy}}^{(j)} = \begin{cases} \mathbf{X}_0^{(j)} + \sigma_{\tau^{(j)}} \boldsymbol{\varepsilon}^{(j)} & \text{in VE case;} \\ \sqrt{\alpha_{\tau^{(j)}}}\, \mathbf{X}_0^{(j)} + \sqrt{1 - \alpha_{\tau^{(j)}}}\, \boldsymbol{\varepsilon}^{(j)} & \text{in VP case;} \end{cases} \tag{46}$$

4. Perform the gradient update with respect to the L_2 between the clean images and the denoiser predictions:

$$\nabla_\theta \frac{1}{b} \sum_{j=1}^{b} \| D_{\tau^{(j)}}^\theta(\mathbf{X}_{\text{noisy}}^{(j)}) - \mathbf{X}_0^{(j)} \|^2. \tag{47}$$

Sampling from the trained backward process amounts to generating pure noise $\mathbf{X}_T^\theta \sim p_T$ and performing sequential updates using $p_{t-1|t}^\theta$, specified as the Gaussian updates that mimic the conditional backward transitions $p_{t-1|t}^\theta(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t) = p_{t-1|t,0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, D_t^\theta(\boldsymbol{x}_t))$. We summarize the derived training and sampling schemes in Table 2.

We call the presented family of models **Diffusion models in discrete time**. In particular, the derived model corresponding to the VP process with a little different parameterization of the neural network is presented in the seminal paper **Denoising Diffusion Probabilistic Models (DDPM)** [11].

Lecture 6. Distillation of Diffusion Models.

In the previous lectures, we have constructed the diffusion models and have derived the most of methodology needed for working with them. However, there is still one topic which is very relevant from the practical standpoint: **acceleration** of diffusion models. Remember the Generative Trilemma: the three desirable properties of a generative model are sample quality, mode coverage and fast generation. While fulfilling the first two properties, diffusion models struggle with the last one: accurate approximation of the backward ODE/SDE requires dozens of neural network evaluations. There are numerous ways of reducing the amount of computations without the additional training: from quantization [6, 7, 9] and caching of computations [25, 47] to careful design of solvers [23, 52, 51], timestep selection [34, 48] and straightening of the trajectories [21, 22]. In the lecture, we fill focus on training-based **diffusion distillation** procedures that aim at compressing the multi-step diffusion models into one-step or few-step generators.

First, let us revise the concept of knowledge distillation [10]. It is most illustrative in case of simple regression/classification tasks, when we are given with a data set of pairs $(\mathbf{X}_1, \mathbf{Y}_1), \dots, (\mathbf{X}_n, \mathbf{Y}_n)$ and aim to learn their functional dependence, i.e. find such

function F that $F(\mathbf{X}_i) = \mathbf{Y}_i$. One can train a neural network on this task from scratch by minimizing expected distance $\mathbb{E}\, d(F_\theta(\mathbf{X}),\, \mathbf{Y})$ between the prediction and the ground truth. However, given an already trained **teacher** network, one can use its knowledge about the problem to **distill** it into a smaller **student** network by encouraging it to mimic the teacher. It can be achieved, for example, by minimizing the expected distance $\mathbb{E}\, d(F_\theta(\mathbf{X}),\, G(\mathbf{X}))$ between the student's prediction $F_\theta(\mathbf{X})$ and the teacher's prediction $G(\mathbf{X})$. This can be beneficial, for example, in classification problems, where the teacher network produces soft labels instead of the hard labels from the data set, which can result in accelerated convergence and overall better training of the student.

In the context of diffusion models, goal of the distillation is to construct such a generator $G^\theta(\mathbf{Z})$ that transforms a simple (e.g. $\mathcal{N}(0, I)$) input into a sample from the data distribution p^{data}. Here, the data distribution p^{data} will be most of the time identified with the distribution generated by the pre-trained diffusion model. We will denote it as $\boldsymbol{s}_t(\boldsymbol{x})$, $\mathrm{D}_t(\boldsymbol{x})$ or $\boldsymbol{\varepsilon}_t(\boldsymbol{x})$, depending on the parameterization.

Progressive Distillation

Diffusion distillation techniques are mainly divided in two families. Methods from the first family consider a pre-trained diffusion model as the noise→data mapping and try to learn the corresponding functional dependency. Let us note, however, that the neural network itself does not represent such a mapping. If it is, for example, parameterized as the denoiser $\mathrm{D}_t(\boldsymbol{x})$, it defines the map $\mathbb{E}[\mathbf{X}_0|\mathbf{X}_t = \boldsymbol{x}]$, which transforms a noisy sample into the best L_2 prediction of the clear image. However, for large noise level T it predicts an almost constant function $\mathbb{E}[\mathbf{X}_0|\mathbf{X}_T = \boldsymbol{x}] \approx \mathbb{E}\, \mathbf{X}_0$ and cannot be regarded as an actual noise→data mapping.

One of the first methods of diffusion distillation, named **Progressive Distillation (PD)** [35], defines the corresponding noise→data mapping as the (backward-time) solution $\mathbf{Y}_0$ of the induced PF-ODE [3]

$$\begin{cases} \mathrm{d}\mathbf{Y}_t = f_t(\mathbf{Y}_t)\mathrm{d}t; \\ \mathbf{Y}_T \sim \mathcal{N}(0, \sigma_T^2 I), \end{cases} \tag{48}$$

where

$$f_t(\boldsymbol{x}) = (1/2) \cdot g(t)\boldsymbol{s}_t(\boldsymbol{x}). \tag{49}$$

Indeed, the PF-ODE defines a deterministic mapping that transforms a fully noisy input $\mathbf{Y}_T$ into samples by integrating the ODE, defined by the corresponding diffusion model

[3] PF is the abbreviation for «Probability Flow», which is the usual naming for the ODE equivalent to the forward noising process.

$\boldsymbol{s}_t(\boldsymbol{x})$. Thus, one can learn this noise→data transformation by simply training the student on the functional

$$\mathbb{E}\, d\Big(G^\theta(\mathbf{Y}_T), \text{ODESolve}\left(\mathbf{Y}_T, T \to 0, f\right)\Big) \to \min_\theta, \tag{50}$$

where d defines some distance in the image space and $\text{ODESolve}\left(\mathbf{Y}_T, T \to 0, f\right)$ is approximated by some solver that we choose at inference time. In the original paper the distance $d(\boldsymbol{x}, \boldsymbol{y})$ is chosen to be the simple L_2, however, one can use other metrics as LPIPS, which has been proven more semantically reach and overall more suitable for distillation [14, 40].

The method is simple, but ineffective. First, it is empirically shown to have worse performance than many baselines [50, 14, 40]. Second, it requires simulating full trajectory of the corresponding ODE. Denoting the corresponding number of discretization steps by N, we obtain the method that requires $N+1$ forward step (diffsuion + generator) and 1 backward step, which we will denote as $(N+1, 1)$. The main advantage of the method is that it is **data-free**: it does not require ground truth samples and only uses samples from the pre-trained model for training.

Consistency Models

Consistency Distillation [40] is the natural development of the idea of learning the ODE-induced mapping. Progressive Distillation tries to learn such mapping by only using its input-output examples. However, since diffusion models use multiple neural network evaluations and large architecture, the corresponding mapping is very complicated. The trained generator may need additional information about the problem and the structure of the mapping to facilitate training. To this end, the authors parameterize it as $G_t^\theta(\boldsymbol{x})$ and aim at transforming noisy samples into clear samples at multiple noise levels t instead of the simple noise→ data parameterization from PD.

Specifically, authors propose to train $G_t^\theta(\boldsymbol{x})$ as an **integrator** of the PF-ODE, induced by the pre-trained diffusion model. How can we describe this requirement mathematically? For all time steps t the generator should map noisy sample from the ODE trajectory to the clear sample from the same trajectory: $G_t^\theta(\mathbf{Y}_t) = \mathbf{Y}_0$ should hold for all t. This requirement can also be rewritten as the combination of two:

1. Generator should induce the identity mapping at the time step 0: $G_0^\theta(\mathbf{Y}_0) = \mathbf{Y}_0$;
2. Generator should be **consistent** and produce equal outputs for inputs from the same trajectory: $G_t^\theta(\mathbf{Y}_t) = G_s^\theta(\mathbf{Y}_s)$ for all s, t.

The boundary condition can be easily satisfied architecturally: if one parameterizes generator as

$$G_t^\theta(\boldsymbol{x}) = c_{\text{skip}}(t)\boldsymbol{x} + c_{\text{out}}(t)F_t^\theta(\boldsymbol{x}), \tag{51}$$

it sufficies to guarantee $c_{\text{skip}}(0) = 1, c_{\text{out}}(0) = 0$. A natural choice of the scaling functions is extensively described in [13] for the diffusion denoiser parameterization. However, due to the similar nature of the generator $G_t^\theta(\boldsymbol{x})$, authors of the Consistency Distillation [40] use the same parameterization. The second property is called **consistency** and is the central property that should be pushed during training.

The whole procedure of the Consistency Distillation (see Figure 1) is very natural: we need to obtain two points from the ODE trajectory, use generator to predict the clear image from both points, measure the difference between them and use it as the loss function. In contrast to Progressive Distillation, Consistency Distillation makes use of the data set and chooses the first point on the trajectory by randomly choosing a time step t, sampling $\mathbf{X} \sim p^{\text{data}}$ and setting $\mathbf{Y}_t = \mathbf{X} + \sigma_t\boldsymbol{\varepsilon}$. The second point on the trajectory is obtained by solving the PF-ODE induced by the pre-trained diffusion model: $\mathbf{Y}_s = \text{ODESolve}(\mathbf{Y}_t, t \to s, f)$. Finally, we make two generator predictions $G_t^\theta(\mathbf{Y}_t)$ and $G_s^\theta(\mathbf{Y}_s)$, and define the loss function as

$$\mathcal{L}(\theta) = \mathbb{E}\, d\left(G_t^\theta(\mathbf{Y}_t), G_s^\theta(\mathbf{Y}_s)\right). \tag{52}$$

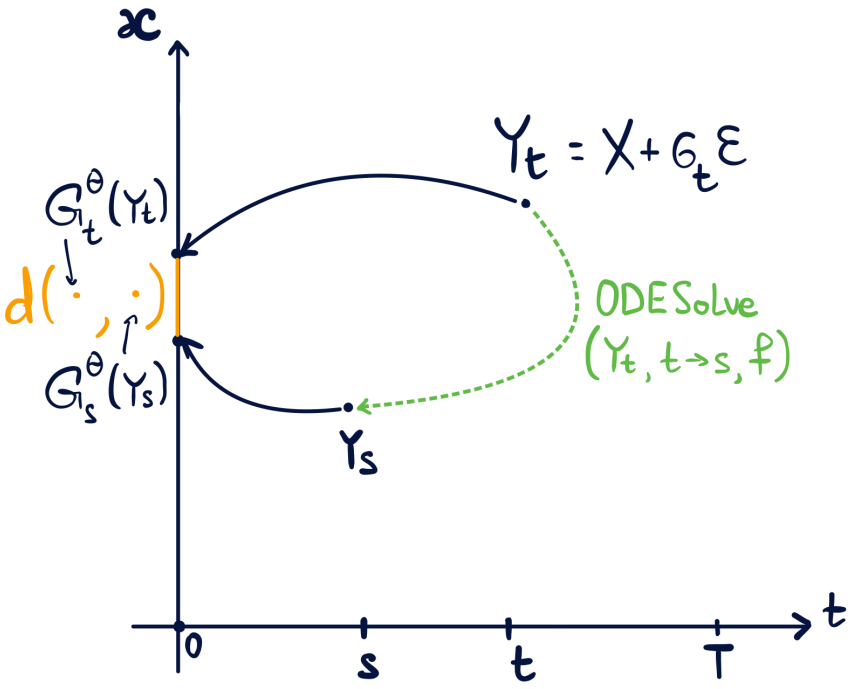

Figure 1: Illustration of the Consistency Distillation.

There is, however, one problem with the proposed functional: one of them, $G_t^\theta(\mathbf{Y}_t)$, is typically worse than the other, $G_s^\theta(\mathbf{Y}_s)$. When the generator is undertrained, having less noise at the input is crucial to perform good prediction. This leads to instabilities

of training, likely caused by the ability of the network to push the «better prediction» towards the «worse prediction». To deal with such problem, the authors modify the objective by stopping the gradient of the «better prediction»:

$$\mathcal{L}(\theta) = \mathbb{E}\, d\Big(G_t^\theta(\mathbf{Y}_t), \mathrm{SG}\left[G_s^\theta(\mathbf{Y}_s)\right]\Big). \tag{53}$$

This essentially makes the «better prediction» $G_s^\theta(\mathbf{Y}_s)$ the target value for regression with the sole difference that this «target» changes during training. The modification acts as an inductive bias to push the model make its worse prediction closer to its better prediction.

Sampling from the model is very simple: generate $\mathbf{Y}_T \sim \mathcal{N}(0, \sigma_T I)$ and return $G_T^\theta(\mathbf{Y}_T)$. However, one can increase model quality by performing iterative refinement of the corresponding predictions. Formally, we choose a few number of time steps $t_1, \ldots, t_M$ and perform the following iterations: $\widehat{\mathbf{X}}_0 = G_T^\theta(\mathbf{Y}_T)$ and $\widehat{\mathbf{X}}_i = G_{t_i}^\theta(\widehat{\mathbf{X}}_{i-1} + \sigma_{t_i}\boldsymbol{\varepsilon})$. This is very simple to the way the diffusion models work, but with the actual one-step generator instead of the fundamentally multi-step denoiser.

The model is much more computationally efficient than the Progressive Distillation due to the possibility of choosing the interval on which we solve the corresponding ODE, and to choose the number of discretization steps K accordingly. Complexity of the method is $(K+2, 1)$: 2 forward passes for the generator and K for solving the PF-ODE. In the case where we choose $s \approx t - h$, one can use one discretization step and obtain 3 forward steps and 1 backward. The only drawback is the need to have access to the data set.

Consistency Models dive deep into the structure of the noise→data mapping, which allows them to induce the corresponding knowledge into generator's architecture and loss and obtain high quality of one-sample generation. A natural development of the idea, called Consistency Trajectory Models [14] is parameterizing the generator with two time steps s, t, corresponding to the initial and final time steps of the PF-ODE. With this parameterization the generator should learn integrating the PF-ODE between any time steps. This introduces even more structure into the model, but makes the overall procedure more complicated. On top of the consistency-based loss, the authors add the GAN loss to push the generator's realsim even further. The combination of all these ideas results in one of the SOTA distillation method at the moment.

Distribution Matching Distillation [4]

The second family of methods do not use the underlying structure of the ODE mapping, induced by a pre-trained diffusion model, but train the generator to match its distribution

[4] This section is originally written in the paper [33].

at output. One of the SOTA methods in this family is Distribution Matching Distillation [50] (with its predecessor Diff-Instruct [24], concurrent work SwiftBrush [29] and the modification [49]).

Essentially, **Distribution Matching Distillation (DMD)** aims to train a generator $G^\theta(\mathbf{Z})$ to match the data distribution p^{data}. Its input $\mathbf{Z}$ is assumed to come from a tractable input distribution p^{noise}. Formally, matching two distributions can be achieved by optimizing the KL divergence between the distribution p^{G^θ} of $G^\theta(\mathbf{Z})$ and the data distribution p^{data}:

$$\text{KL}(p^{G^\theta} \,\|\, p^{\text{data}}) = \mathbb{E} \log \frac{p^{G^\theta}(G^\theta(\mathbf{Z}))}{p^{\text{data}}(G^\theta(\mathbf{Z}))} \to \min_\theta . \tag{54}$$

To perform gradient descent with respect to generator's parameters, one should be able to calculate the corresponding gradient:

$$\nabla_\theta \text{KL}(p^{G^\theta} \,\|\, p^{\text{data}}) = \nabla_\theta \, \mathbb{E} \log \frac{p^{G^\theta}(G^\theta(\mathbf{Z}))}{p^{\text{data}}(G^\theta(\mathbf{Z}))} = \mathbb{E} \, \nabla_\theta \log \frac{p^{G^\theta}(G^\theta(\mathbf{Z}))}{p^{\text{data}}(G^\theta(\mathbf{Z}))}. \tag{55}$$

The log density ratio depends on θ in two places: in the input $G^\theta(\mathbf{Z})$ and in the definition of the underlying density p^{G^θ}. Gradient with respect to the input can be easily calculated by the chain rule:

$$\mathbb{E} \left(s^{G^\theta}(G^\theta(\mathbf{Z})) - \boldsymbol{s}^{\text{data}}(G^\theta(\mathbf{Z})) \right) \nabla_\theta G^\theta(\mathbf{Z}). \tag{56}$$

At the same time, gradient with respect to the density parameter is equal to

$$\mathbb{E} \left(\nabla_\theta \log \frac{p^{G^\theta}(\boldsymbol{x})}{p^{\text{data}}(\boldsymbol{x})} \right) \Bigg|_{\boldsymbol{x}=G^\theta(\mathbf{Z})} = \int_{\mathbb{R}^d} \left(\nabla_\theta \log \frac{p^{G^\theta}(\boldsymbol{x})}{p^{\text{data}}(\boldsymbol{x})} \right) p^{G^\theta}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = \tag{57}$$

$$= \int_{\mathbb{R}^d} p^{G^\theta}(\boldsymbol{x}) \nabla_\theta \log p^{G^\theta}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = \int_{\mathbb{R}^d} \nabla_\theta p^{G^\theta}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = \nabla_\theta \int_{\mathbb{R}^d} p^{G^\theta}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = \nabla_\theta 1 = 0, \tag{58}$$

which is a famous result, widely used in the so-called REINFORCE [46] algorithm and policy-gradient methods in Reinforcement Learning. Thus, gradient of the KL divergence is equal to the value in Equation 56, which main part is the difference between the score functions of the corresponding distributuions.

The pure data score function can be very non-smooth due to the Manifold Hypothesis [42] and is generally difficult to train [41]; therefore, the authors address this challenge using the diffusion framework. To this end, they relax the original loss by using an ensemble

of KL divergences between distributions, which are perturbed by the forward diffusion process:

$$\int_0^T \omega_t \, \mathrm{KL}\Big(p_t^{G^\theta} \,\|\, p_t^{\text{data}}\Big)\mathrm{d}t = \int_0^T \omega_t \, \mathbb{E} \, \log \frac{p_t^{G^\theta}(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon})}{p_t^{\text{data}}(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon})} \, \mathrm{d}t. \tag{59}$$

Here, ω_t is a weighting function, $p_t^{G^\theta}$ and p_t^{data} are the perturbed versions of the generator distribution and p^{data} up to the time step t. In theory, the minima of Eq. 59 objective coincides[45, Thm. 1] with the original minima from Eq. 54. Meanwhile, in practice, taking the gradient of the new loss results in the difference $\boldsymbol{s}_t^{G^\theta}(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon}) - \boldsymbol{s}_t^{\text{data}}(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon})$, which can be approximated using diffusion models.

Given this, authors approximate $\boldsymbol{s}_t^{\text{data}}$ with the pre-trained diffusion model, which we will denote $\boldsymbol{s}_t^{\text{data}}$ as well with a slight abuse of notation. The whole procedure now can be considered as distillation of $\boldsymbol{s}_t^{\text{data}}$ into G^θ. At the same time, $\boldsymbol{s}_t^{G^\theta}$ represents the score of the noised distribution of the generator, which is intractable and is therefore approximated by an additional «fake» diffusion model $\boldsymbol{s}_t^\phi$ and the corresponding denoiser D_t^ϕ. It is trained on the standard denoising score matching objective with the generator's samples at the input. The joint training procedure is essentially the coordinate descent

$$\begin{cases} \int\limits_0^T \omega_t \, \mathbb{E} \, \log \dfrac{p_t^\phi(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon})}{p_t^{\text{data}}(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon})} \mathrm{d}t \to \min\limits_\theta; \\ \int\limits_0^T \beta_t \, \mathbb{E} \, \|\mathrm{D}_t^\phi(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon}) - G^\theta(\mathbf{Z})\|^2 \, \mathrm{d}t \to \min\limits_\phi, \end{cases} \tag{60}$$

where the stochastic gradient with respect to the fake network parameters ϕ is calculated by backpropagation and the generator's stochastic gradient is calculated directly as (analogous to the original KL)

$$\omega_t \, \Big(\boldsymbol{s}_t^\phi \left(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon}\right) - \boldsymbol{s}_t^{\text{data}} \left(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon}\right)\Big) \, \nabla_\theta G^\theta(\mathbf{Z}). \tag{61}$$

Minimization of the fake network's objective ensures $\boldsymbol{s}_t^\phi = \boldsymbol{s}_t^{G^\theta} \Leftrightarrow p_t^\phi = p_t^{G^\theta}$. Under this condition, the generator's objective is equal to the original ensemble of KL divergences from Eq. 59, minimizing which solves the initial problem and implies $p^{G^\theta} = p^{\text{data}}$.

In practice, one could calculate the score difference directly and pass it as an argument to the *backward* function. However, one could calculate it in a more interpretable way by using the stop-grad operation. Let us define the «adjusted» output of the generator as

$$\widehat{G} = \mathrm{SG}\left[G^\theta(\mathbf{Z}) + \omega_t \, \boldsymbol{s}_t^{\text{data}}(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon}) - \omega_t \, \boldsymbol{s}_t^\phi(G^\theta(\mathbf{Z}) + \sigma_t \boldsymbol{\varepsilon})\right] \tag{62}$$

and calculate the gradient of the L_2 difference between the original output and the adjusted. It is equal to (remember that $\widehat{G}$ does not propagate gradients!)

$$\nabla_\theta \|G^\theta(\mathbf{Z}) - \widehat{G}\|^2 = 2 \left(G^\theta(\mathbf{Z}) - \widehat{G}\right) \nabla_\theta G^\theta(\mathbf{Z}) = \tag{63}$$

$$= 2\left(G^\theta(\mathbf{Z}) - G^\theta(\mathbf{Z}) - \omega_t\, \boldsymbol{s}_t^{\text{data}}(G^\theta(\mathbf{Z}) + \sigma_t\boldsymbol{\varepsilon}) + \omega_t\, \boldsymbol{s}_t^{\phi}(G^\theta(\mathbf{Z}) + \sigma_t\boldsymbol{\varepsilon})\right)\nabla_\theta G^\theta(\mathbf{Z}) = \quad (64)$$

$$= 2\,\omega_t\,\left(\boldsymbol{s}_t^{\phi}\left(G^\theta(\mathbf{Z}) + \sigma_t\boldsymbol{\varepsilon}\right) - \boldsymbol{s}_t^{\text{data}}\left(G^\theta(\mathbf{Z}) + \sigma_t\boldsymbol{\varepsilon}\right)\right)\nabla_\theta G^\theta(\mathbf{Z}), \quad (65)$$

which equals the actual gradient from Equation 61 up to the multiplication by 2.

Representing score functions by the corresponding denoisers, one can represent the «adjusted» output $\widehat{G}$ as (we omit SG for convenience):

$$\widehat{G} = G^\theta(\mathbf{Z}) + \omega_t\, \boldsymbol{s}_t^{\text{data}}(G^\theta(\mathbf{Z}) + \sigma_t\boldsymbol{\varepsilon}) - \omega_t\, \boldsymbol{s}_t^{\phi}(G^\theta(\mathbf{Z}) + \sigma_t\boldsymbol{\varepsilon}) = \quad (66)$$

$$= G^\theta(\mathbf{Z}) + \omega_t\,\left(\frac{\mathrm{D}_t^{\text{data}}(\cdot) - (\cdot)}{\sigma_t^2}\right) - \omega_t\,\left(\frac{\mathrm{D}_t^{\phi}(\cdot) - (\cdot)}{\sigma_t^2}\right) = \quad (67)$$

$$= G^\theta(\mathbf{Z}) + \frac{\omega_t}{\sigma_t^2}\left(\mathrm{D}_t^{\text{data}}(G^\theta(\mathbf{Z}) + \sigma_t\boldsymbol{\varepsilon}) - \mathrm{D}_t^{\phi}(G^\theta(\mathbf{Z}) + \sigma_t\boldsymbol{\varepsilon})\right). \quad (68)$$

This has a very natural interpretation: we want to improve quality of our generator's output and make it more realistic. To this end, we add noise to its outputs and denoise with the pretrained denoiser and the generator's denoiser. When the generator is undertrained, the pre-trained denoiser outputs more realistic samples than the generator's denoiser. DMD then calculates the difference between the «good» and «bad» denoised outputs and adds in to the original output with some weight. Thus, similarly to Consistency Models, DMD follows the philosophy of pushing the «worse» prediction towards the «better» prediction by using the stop-grad operation accordingly.

With careful implementation, DMD achieves SOTA results in 1-step generation [49]. At the same time, DMD is a data-free procedure, which makes it applicable in a wider set of scenarios then the Consistency Distillation. As for the method's complexity, it requires 3 forward passes (two for the diffusion models, one for the generator) and 1 backward pass per generator update, which is comparable with consistency models. On top of that, it requires training an additional fake network, which significantly increases memory consumption and moderately increases training time (2 forward and 1 backward pass per diffusion step).

The described version of DMD, however, has mode collapse and needs to be modified. In the original DMD paper [50] the authors combine DMD loss with Progressive Distillation, which introduces the additional computational overhead, but significantly stabilizes training. In the Improved DMD paper [49], however, the authors remove the PD loss and perform several fake diffusion updates per generator update, which makes the actual loss closer to the KL ensemble and also stabilizes training. Additionally endowing procedure with the GAN loss improves quality even further and allows to achieve SOTA quality.

Distribution Matching is a very natural and effective approach to diffusion distillation. This and relative methods have spawned a whole family of procedures that match the

generator's distribution with the pre-trained diffusion, including matching their scores [53] and moments [36]. The paradigm can also be applied in a wide set of problems beyond unconditional generation. Famous examples include using the pre-trained model to guide generation of text-to-3D models [32, 45]. On top of that, DMD can be used in image-to-image translation [33], where it is regularized by pushing the generator's input and output being close to each other.

Lecture 7. Flow Matching.

The first part of the course was devoted to constructing the framework of diffusion models. During this construction, we have performed the following steps:

1. Defined the forward noising process via VE-SDE $\mathrm{d}\mathbf{X}_t = g(t)\mathrm{d}\mathbf{W}_t$ or VP-SDE $\mathrm{d}\mathbf{X}_t = -\frac{\beta_t}{2}\mathbf{X}_t\mathrm{d}t + \sqrt{\beta_t}\mathrm{d}\mathbf{W}_t$;
2. Constructed the equivalent PF-ODE $\mathrm{d}\mathbf{Y}_t = -(1/2)g^2(t)\nabla \log p_t(\mathbf{Y}_t)\mathrm{d}t$ or $\mathrm{d}\mathbf{Y}_t = -(\beta_t/2)(\mathbf{X}_t + \nabla \log p_t(\mathbf{Y}_t))\mathrm{d}t$;
3. Derived the one-step generation procedures $\mathbf{X}_t = \mathbf{X}_0 + \sigma_t\boldsymbol{\varepsilon}$ or $\mathbf{X}_t = \sqrt{\alpha_t}\mathbf{X}_0 + \sqrt{1-\alpha_t}\boldsymbol{\varepsilon}$.

The relation between the first two points was crucial for constructing the sampling algorithm for diffusion models: since $p_{\mathbf{X}_t} = p_{\mathbf{Y}_t}$, one can generate from the data distribution $p_{\mathbf{X}_0}$ by sampling from the noise distribution and solving the corresponding ODE backwards in time. The one-step sampling procedure from the forward process was mainly needed to develop the efficient score training and did not play a major role in theoretical derivations. However, looking at this interpretation from another point of view will allow us to construct a generalization of diffusion models, called the Flow Matching [18, 21, 2, 43] method.

Let us, with some abuse of notation, define the interpolation process for both VE- and VP-SDE by sampling an element $\mathbf{X}_0$ from the data distribution, noise $\boldsymbol{\varepsilon}$ from $\mathcal{N}(0, I)$ and setting

$$\begin{cases} \mathbf{X}_t = \mathbf{X}_0 + \sigma_t\boldsymbol{\varepsilon} & \text{for VE-SDE;} \\ \mathbf{X}_t = \sqrt{\alpha_t}\mathbf{X}_0 + \sqrt{1-\alpha_t}\boldsymbol{\varepsilon} & \text{for VP-SDE.} \end{cases} \tag{69}$$

Taking a closer look at this dynamics, one can see that, unlike the original forward diffusion or the equivalent complicated PF-ODE, this random process represents a simple time-dependent linear combination of the noise $\boldsymbol{\varepsilon}$ and the data sample $\mathbf{X}_0$. If σ_t or α_t

are smooth functions (which we always assume in practice), both random processes can be differentiated:

$$\begin{cases} \frac{\mathrm{d}}{\mathrm{d}t}\mathbf{X}_t = \sigma_t'\boldsymbol{\varepsilon} & \text{for VE-SDE;} \\ \frac{\mathrm{d}}{\mathrm{d}t}\mathbf{X}_t = \frac{\alpha_t'}{2\sqrt{\alpha_t}}\mathbf{X}_0 - \frac{\alpha_t'}{2\sqrt{1-\alpha_t}}\boldsymbol{\varepsilon} & \text{for VP-SDE.} \end{cases} \tag{70}$$

While this seems like an ODE, which can be used for generation, it requires knowing $\mathbf{X}_0$. It is evident from the form in the VP-SDE, but is also true for VE-SDE: the corresponding ODE should start from the sample $\mathbf{X}_0 + \sigma_T\boldsymbol{\varepsilon}$, then solving it backwards in time will generate $\mathbf{X}_0$. If we try to solve it from $\sigma_T\boldsymbol{\varepsilon}$, we will obtain zero. However, this can be considered as a **conditional ODE**, which defines a noise-data interpolation given fixed $\mathbf{X}_0$ and $\boldsymbol{\varepsilon}$ and is actually tractable (compare it with making the score function tractable when replacting the distribution p_t with the conditional distribution $p_{t|0}$).

Flow Matching: from conditional dynamics to unconditional

Given the above observation, let us reformulate the task of learning an ODE that translates data to noise (and vice versa) in the following way:

Problem. Given a process $\mathbf{X}_t$ and some latent variable $\mathbf{Z}$, assume the **conditional dynamics** $p_{\mathbf{X}_t|\mathbf{Z}}(\boldsymbol{x}|\boldsymbol{z})$ is generated by an ODE with the **conditional velocity field** $f_t(\boldsymbol{x}|\boldsymbol{z})$. Find such a velocity field $f_t^*(\boldsymbol{x})$ that the corresponding ODE

$$\mathrm{d}\mathbf{X}_t = f_t^*(\mathbf{X}_t)\mathrm{d}t \tag{71}$$

generates the unconditional dynamics $p_{\mathbf{X}_t}(\boldsymbol{x})$.

It is useful to keep in mind that the latent variable will always be something interpretable (and something intractable): e.g. data sample $\mathbf{X}_0$ or the data-noise pair $(\mathbf{X}_0, \boldsymbol{\varepsilon})$ from the previous example. We already know the answer for the mentioned cases: if $\mathbf{X}_t$ is the interpolation process from VE/VP-SDE, its dynamics $p_{\mathbf{X}_t}$ can be generated by the PF-ODE. However, solving this problem in the general setting will allow for using diffusion-like models in applications beyond unconditional generation. Now, we will prove the following

Claim. *If the conditional dynamics $p_{\mathbf{X}_t|\mathbf{Z}}(\boldsymbol{x}|\boldsymbol{z})$ is generated by the ODE with the conditional velocity field $f_t(\boldsymbol{x}|\boldsymbol{z})$, then the unconditional dynamics $p_{\mathbf{X}_t}(\boldsymbol{x})$ is also generated by the ODE with the velocity field*

$$f_t^*(\boldsymbol{x}) = \mathbb{E}[f_t(\mathbf{X}_t|\mathbf{Z})|\mathbf{X}_t = \boldsymbol{x}]. \tag{72}$$

Proof. The most convenient technique for working with distribution dynamics, when they are generated by ODE, is the continuity equation. Recall that saying the dynamics $p_{\mathbf{X}_t}$ is generated by the ODE with the velocity field $f_t(\boldsymbol{x})$ is equivalent to saying $p_{\mathbf{X}_t}$ and $f_t(\boldsymbol{x})$ are connected by the equation

$$\frac{\partial}{\partial t} p_{\mathbf{X}_t}(\boldsymbol{x}) = -\mathrm{div}\big(p_{\mathbf{X}_t}(\boldsymbol{x}) f_t(\boldsymbol{x})\big), \tag{73}$$

so our goal is to find $f_t(\boldsymbol{x})$, satisfying this equation with $p_{\mathbf{X}_t}(\boldsymbol{x})$. At the same time, we know that $p_{\mathbf{X}_t|\mathbf{Z}}(\boldsymbol{x}|\boldsymbol{z})$ is generated by the ODE with $f_t(\boldsymbol{x}|\boldsymbol{z})$, thus satisfies the continuity equation (CE)

$$\frac{\partial}{\partial t} p_{\mathbf{X}_t|\mathbf{Z}}(\boldsymbol{x}|\boldsymbol{z}) = -\mathrm{div}\big(p_{\mathbf{X}_t|\mathbf{Z}}(\boldsymbol{x}|\boldsymbol{z}) f_t(\boldsymbol{x}|\boldsymbol{z})\big). \tag{74}$$

There is more or less one direction that we could go in: expressing the unconditional density in terms of conditional, use the conditional CE and express the result in terms of the unconditional density again. We start with rewriting the marginal density by writing it as the integral and replacing the integral with the time-derivative:

$$\frac{\partial}{\partial t} p_{\mathbf{X}_t}(\boldsymbol{x}) = \frac{\partial}{\partial t} \int p_{\mathbf{X}_t|\mathbf{Z}}(\boldsymbol{x}|\boldsymbol{z}) p_{\mathbf{Z}}(\boldsymbol{z}) \mathrm{d}\boldsymbol{z} = \int \left(\frac{\partial}{\partial t} p_{\mathbf{X}_t|\mathbf{Z}}(\boldsymbol{x}|\boldsymbol{z}) \right) p_{\mathbf{Z}}(\boldsymbol{z}) \mathrm{d}\boldsymbol{z}. \tag{75}$$

The time-derivative of the conditional dynamics can be expressed by the right-hand side of the corresponding CE:

$$-\int \mathrm{div}\big(p_{\mathbf{X}_t|\mathbf{Z}}(\boldsymbol{x}|\boldsymbol{z}) f_t(\boldsymbol{x}|\boldsymbol{z})\big) p_{\mathbf{Z}}(\boldsymbol{z}) \mathrm{d}\boldsymbol{z} = -\mathrm{div} \int \big(p_{\mathbf{X}_t|\mathbf{Z}}(\boldsymbol{x}|\boldsymbol{z}) f_t(\boldsymbol{x}|\boldsymbol{z})\big) p_{\mathbf{Z}}(\boldsymbol{z}) \mathrm{d}\boldsymbol{z}, \tag{76}$$

where we move the divergence outside of the integral, since it contains only $\boldsymbol{x}$-derivatives. The only detail that differs the equation from the right-hand side of the desired CE is the multiplication on the density $p_{\mathbf{X}_t}(\boldsymbol{x})$, so we perform the classical multiplication-division and obtain

$$\frac{\partial}{\partial t} p_{\mathbf{X}_t}(\boldsymbol{x}) = -\mathrm{div} \left(p_{\mathbf{X}_t}(\boldsymbol{x}) \int f_t(\boldsymbol{x}|\boldsymbol{z}) \frac{p_{\mathbf{X}_t|\mathbf{Z}}(\boldsymbol{x}|\boldsymbol{z}) p_{\mathbf{Z}}(\boldsymbol{z})}{p_{\mathbf{X}_t}(\boldsymbol{x})} \mathrm{d}\boldsymbol{z} \right). \tag{77}$$

Under the integral we encounter the Bayes' theorem for $p_{\mathbf{X}_t|\mathbf{Z}}$ and $p_{\mathbf{Z}}$. The resulting expression corresponds to the expectation over the conditional density $p_{\mathbf{Z}|\mathbf{X}_t}$:

$$\frac{\partial}{\partial t} p_{\mathbf{X}_t}(\boldsymbol{x}) = -\mathrm{div} \left(p_{\mathbf{X}_t}(\boldsymbol{x}) \int f_t(\boldsymbol{x}|\boldsymbol{z}) p_{\mathbf{Z}|\mathbf{X}_t}(\boldsymbol{z}|\boldsymbol{x}) \mathrm{d}\boldsymbol{z} \right). \tag{78}$$

Given this, the unconditional dynamics and the integral satisfy the continuity equation. Thus, the unconditional dynamics is generated by the ODE with the velocity field

$$f_t^*(\boldsymbol{x}) = \int f_t(\boldsymbol{x}|\boldsymbol{z}) p_{\mathbf{Z}|\mathbf{X}_t}(\boldsymbol{z}|\boldsymbol{x}) \mathrm{d}\boldsymbol{z} = \mathbb{E}[f_t(\boldsymbol{x}|\mathbf{Z})|\mathbf{X}_t = \boldsymbol{x}] = \mathbb{E}[f_t(\mathbf{X}_t|\mathbf{Z})|\mathbf{X}_t = \boldsymbol{x}], \tag{79}$$

which completes the proof. □

The result is very similar to the representation of the score through the conditional expectation of the conditional score. While providing the description for the unconditional dynamics, it also gives a way to learn the unconditional vector field in practice! Since it is represented as the conditional expectation and its left-hand side is assumed to be tractable, one can learn $f_t^*(\boldsymbol{x})$ by minimizing the objective

$$\int_0^1 \mathbb{E}\left\|f_t^\theta(\mathbf{X}_t) - f_t(\mathbf{X}_t|\mathbf{Z})\right\|^2 \mathrm{d}t \to \min_\theta. \tag{80}$$

The obtained training procedure, coupled with sampling by solving the corresponding ODE

$$\mathrm{d}\mathbf{Y}_t = f_t^\theta(\mathbf{Y}_t)\mathrm{d}t, \tag{81}$$

is called **Flow Matching (FM)**.

Flow Matching for interpolation dynamics

While the derived result holds true for any latent variable $\mathbf{Z}$, let us establish the two most important use cases.

The first is the diffusion setup: $\mathbf{X}_t$ is the data-noise interpolation process (VE/VP), $\mathbf{Z}$ is the data-noise pair $(\mathbf{X}_0, \boldsymbol{\varepsilon})$. A good *exercise* consists in verifying that the optimal velocity field $f_t^*(\boldsymbol{x})$, learned by the Flow Matching procedure, coincides with the velocity field from the PF-ODE, thus generalizing the ODE formulation of diffusion models.

Here comes the point, where Flow Matching becomes more capable than the diffusion models. Let $(\mathbf{X}_0, \mathbf{X}_1) \sim p_{0,1}$ be a pair of samples from an **arbitrary** joint distribution, where $\mathbf{X}_0 \sim p_0$ and $\mathbf{X}_1 \sim p_1$. Later, we will define $p_{0,1}$ for particular applications. For now, one could think that $p_{0,1}(\boldsymbol{x}_0, \boldsymbol{x}_1) = \mathcal{N}(\boldsymbol{x}_0|0, I)p^{\text{data}}(\boldsymbol{x}_1)$ is the distribution of independent noise and data samples.

We define $\mathbf{Z}$ as the pair of endpoints $\mathbf{Z} = (\mathbf{X}_0, \mathbf{X}_1)$ and define the interpolation process as

$$\mathbf{X}_t = I_t(\mathbf{X}_0, \mathbf{X}_1), \tag{82}$$

where $I_t(\boldsymbol{x}_0, \boldsymbol{x}_1)$ is an **interpolant** — smooth function, connecting the endpoints $\boldsymbol{x}_0 = I_0(\boldsymbol{x}_0, \boldsymbol{x}_1)$ and $\boldsymbol{x}_1 = I_1(\boldsymbol{x}_0, \boldsymbol{x}_1)$. In this setting, the conditional dynamics is concentrated in one point:

$$p_{\mathbf{X}_t|\mathbf{X}_0,\mathbf{X}_1}(\boldsymbol{x}|\boldsymbol{x}_0, \boldsymbol{x}_1) = I\{\boldsymbol{x} = I_t(\boldsymbol{x}_0, \boldsymbol{x}_1)\} \tag{83}$$

and is generated by the conditional ODE

$$\mathrm{d}\mathbf{X}_t^{0,1} = \frac{\partial}{\partial t} I_t(\boldsymbol{x}_0, \boldsymbol{x}_1)\mathrm{d}t, \tag{84}$$

which can be easily verified from the fact that solving it from the initial condition $\mathbf{X}_0^{0,1} = \boldsymbol{x}_0$ results in obtaining

$$\mathbf{X}_t^{0,1} = \boldsymbol{x}_0 + \int_0^t \partial_s I_s(\boldsymbol{x}_0, \boldsymbol{x}_1)\mathrm{d}s = \boldsymbol{x}_0 + I_t(\boldsymbol{x}_0, \boldsymbol{x}_1) - I_0(\boldsymbol{x}_0, \boldsymbol{x}_1) = I_t(\boldsymbol{x}_0, \boldsymbol{x}_1), \tag{85}$$

which at each timestep t has distribution, concentrated in $I_t(\boldsymbol{x}_0, \boldsymbol{x}_1)$. Thus, the dynamics $p_{\mathbf{X}_t^{0,1}}(\boldsymbol{x})$ is concentrated in the point $I_t(\boldsymbol{x}_0, \boldsymbol{x}_1)$ and coincides with the conditional dynamics $= p_{\mathbf{X}_t|\mathbf{X}_0,\mathbf{X}_1}(\boldsymbol{x}|\boldsymbol{x}_0, \boldsymbol{x}_1)$. In this scenario, the conditional velocity field is defined by the time-derivative of the interpolant $\partial_t I_t(\boldsymbol{x}_0, \boldsymbol{x}_1)$, and the unconditional vector field is equal to

$$f_t^*(\boldsymbol{x}) = \mathbb{E}\left[\frac{\partial}{\partial t} I_t(\mathbf{X}_0, \mathbf{X}_1) \mid \mathbf{X}_t = \boldsymbol{x}\right] \tag{86}$$

and can be optimized by solving

$$\int_0^1 \mathbb{E}\left\| f_t^\theta(\mathbf{X}_t) - \frac{\partial}{\partial t} I_t(\mathbf{X}_0, \mathbf{X}_1)\right\| \mathrm{d}t \to \min_\theta . \tag{87}$$

How should we define the interpolant $I_t(\boldsymbol{x}_0, \boldsymbol{x}_1)$? The most straightforward way is to consider the constant-speed linear interpolant $I_t(\boldsymbol{x}_0, \boldsymbol{x}_1) = t\boldsymbol{x}_1 + (1-t)\boldsymbol{x}_0$. But does it possess any useful properties? First, recent studies [8] show that this type of interpolation is the most effective by running tons of experiments and validating it empirically. Second, we will establish that choosing this interpolant results in a positive impact on the curvature of the generated trajectory. The more straight it is, the less number of steps we need for accurate sampling. Thus, we will stick to this **linear interpolant** most of the time. The optimization procedure results in a simple regression on the difference between the endpoints:

$$\int_0^1 \mathbb{E}\| f_t^\theta(\mathbf{X}_t) - (\mathbf{X}_1 - \mathbf{X}_0)\|^2 \mathrm{d}t \to \min_\theta, \tag{88}$$

and the optimal velocity field is equal to

$$f_t^*(\boldsymbol{x}) = \mathbb{E}[\mathbf{X}_1 - \mathbf{X}_0 | \mathbf{X}_t = \boldsymbol{x}]. \tag{89}$$

The result is illustrated visually in Figure 2. The figure depicts a very natural idea that stands behind representing unconditional velocity through conditional. Transporting between two distributions is hard. At the same time, transporting between two of the corresponding distributions is easy: one can move along the line that connects them with constant speed. Now, given the position $\boldsymbol{x}$ at time t, where should we go? If we knew the endpoints, we would go in the corresponding conditional direction. Since we do not know

the endpoints, we go through all pairs that intersect at the point $\boldsymbol{x}$ at the time t and ensemble the corresponding directions with weights according to the posterior probability of the pair.

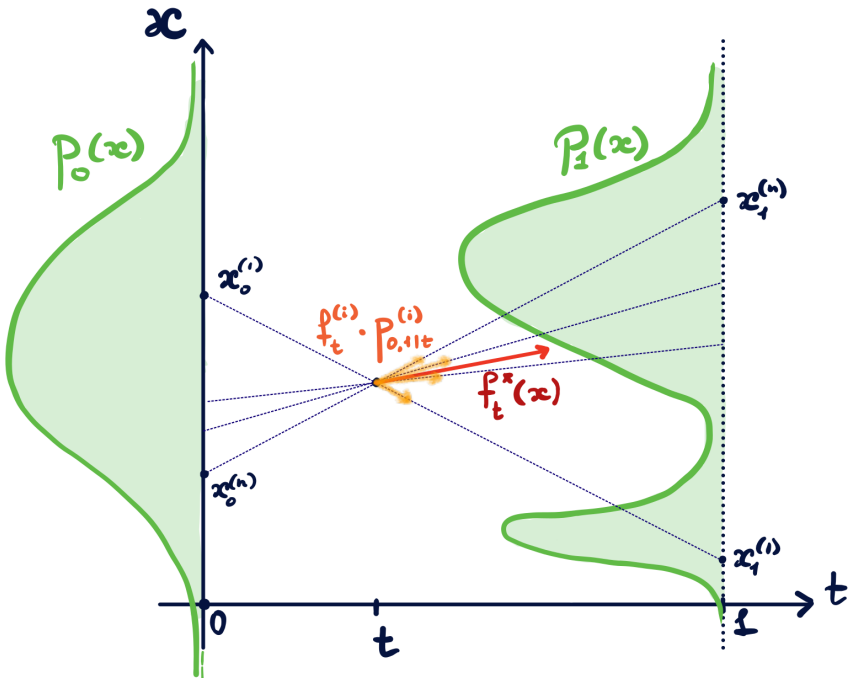

Figure 2: Representation of the unconditional velocity $f_t^*(\boldsymbol{x})$ through the weighted ensemble of the conditional. Here, $f_t^{(i)}$ and $p_{0,1|t}^{(i)}$ denote $f_t(\boldsymbol{x}|\boldsymbol{x}_0^{(i)}, \boldsymbol{x}_1^{(i)})$ and $p_{0,1|t}(\boldsymbol{x}_0^{(i)}, \boldsymbol{x}_1^{(i)}/\boldsymbol{x})$, respectively.

Another consequence of finding the ODE that generates the unconditional dynamics is that we have obtained two processes with completely different properties, but the same marginal distributions $p_{\mathbf{X}_t} = p_{\mathbf{Y}_t}$.

	Interpolation process	Trained ODE
Definition	$(\mathbf{X}_0, \mathbf{X}_1) \sim p_{0,1}$ $\mathbf{X}_t = t\mathbf{X}_t + (1-t)\mathbf{X}_0$	$\mathbf{Y}_0 \sim p_0$ $\mathrm{d}\mathbf{Y}_t = f_t^*(\mathbf{Y}_t)\mathrm{d}t$
Dependencies	Non-Markov	Markov
Trajectories	Intersect	Do not intersect
Known part	Interpolation	Starting point
Intractable part	Endpoints	Transitions

Table 3: Properties of the interpolation process and the ODE obtained by training the Flow Matching method.

Differences between the two processes are summarized in Table 3. First of all, the ODE

generated by the Flow Matching procedure is a Markov process: the transition from $\mathbf{Y}_t$ to $\mathbf{Y}_{t+h}$ depends only on the current position $\mathbf{Y}_t$. The original interpolation process is not Markov: its middle values inherently depend on the future value $\mathbf{X}_1$. Since $\mathbf{Y}_t$ solves an ODE, it has non-intersecting trajectories, while trajectories of the interpolation process clearly can intersect. Finally, similarly to the diffusion models, the intractable part (that is related to the problem we are solving) of the interpolation process is concentrated on the endpoints. The learned ODE shifts this intractable data-dependent part in the velocity field $f_t^*(\boldsymbol{x})$.

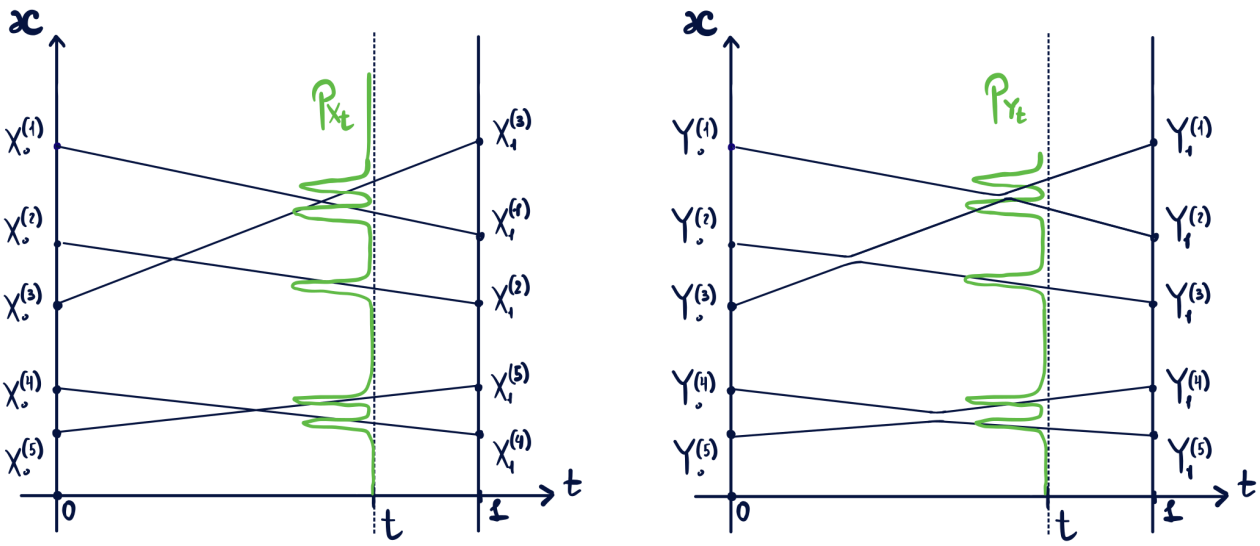

Figure 3: Illustration of the interpolation process and the ODE generated by the Flow Matching procedure. Both processes share the same marginal distributions $p_{\mathbf{X}_t} = p_{\mathbf{Y}_t}$, but the second does not have intersections, which (in 1d) leads to the monotone map.

The most interesting difference between the processes consists in the fact that the interpolation process $\mathbf{X}_t$ has intersections, while the learned ODE $\mathbf{Y}_t$ does not. Figure 3 visualizes the possible placement of the ODE trajectories that fulfill the marginal constraint $p_{\mathbf{X}_t} = p_{\mathbf{Y}_t}$. Let p_0 be a distribution (almost) concentrated in a finite number of points, paired with another distribution p_1, (almost) concentrated in the same number of points, and let $p_{1|0}$ be a deterministic mapping between them, visualized in Figure 3 at the left. Let us then try to visualize trajectories of the learned ODE. They should start in the same set of points at the time $t = 0$ and produce the same set of points in any time t, which means that the drawings should coincide with a sole difference that at the right they should not intersect. This means that the trajectories should be rearranged in the monotonic way: if we encounter an intersection, we should go to the left, if there was no trajectories in the point, and to the right, if there was one. Interestingly, in a depicted 1-dimensional example it leads to a monotonic mapping between p_0 and p_1, which will be a crucial observation for domain translation applications.

Flow Matching for domain translation

Previously, we announced that Flow Matching is a more capable method than the original diffusion models, since it can learn the ODE that corresponds to any dynamics that interpolates between any pair $(\mathbf{X}_0, \mathbf{X}_1)$. In particular, Flow Matching can learn a mapping between arbitrary distributions p_0 and p_1, which, in the general setting, are called **source** and **target**, respectively. But what types of problems can be described by the necessity of transporting between a pair of distributions? They are generally called **domain translation** problems and mainly consist of the classes presented in the Table 4.

	Joint distribution $p_{0,1}$	Examples
Unconditional Generation	$(\mathbf{X}_0, \mathbf{X}_1) \sim p^{\text{data}}(\boldsymbol{x}_1)\mathcal{N}(\boldsymbol{x}_0 \vert 0, I)$	**Learning any data set**
Inverse Problems	$\mathbf{X}_1 \sim p^{\text{data}}$, $\mathbf{X}_0 = \text{Corrupt}(\mathbf{X}_1)$	**Inversing degradations** Inpainting Deblurring Super-resolution
Paired translation	$\mathbf{X}_1 = f(\mathbf{X}_0)$ or $\mathbf{X}_0 = f(\mathbf{X}_1)$	**Ground-truth mapping** Day→Night Sketch→Photo Aerial→Map
Unpaired Translation	$(\mathbf{X}_0, \mathbf{X}_1) \sim p_0(\boldsymbol{x}_0)p_1(\boldsymbol{x}_1)$	**No GT Mapping** Cat→Dog Male→Female Face→Anime

Table 4: Different domain translation problems and the corresponding distribution on the endpoints.

Mainly, domain translation problems can be divided into 3 categories: unconditional generation, paired and unpaired translation. We are mainly interested in paired and unpaired translation, since we already know how to perform Noise→Data generation.

An important class of the paired problems consists of the so-called **inverse problems**, in which the original data is corrupted, and the goal is to perform the corresponding restoration. The examples here are inpainting, deblurring, super-resolution, JPEG restoration etc. The joint endpoint distribution of the interpolation process is defined by the pair $(\mathbf{X}_0, \mathbf{X}_1)$, corresponding to the original data sample $\mathbf{X}_1$ and its degraded version $\mathbf{X}_0$. In this scenario the Flow Matching algorithm will learn to translate between the degraded

and non-degraded images.

Generally, **paired domain translation** problems consist of all tasks in which there is a ground truth mapping between the two domains that we aim to learn. Apart from the inverse problems, it can be any task for which there is a paired data set $\{(\mathbf{X}_0^{(i)}, \mathbf{X}_1^{(i)})\}_{i=1}^n$ that defines this ground-truth dependence. The examples include (but are not limited to) translation between day and night photos of the same place, transforming sketch of the picture into the original picture etc. More examples can be found in the seminal paper [12] that introduces the Pix2Pix model for tackling paired problems with adversarial training.

Finally, **unpaired domain translation** problems differ from the previous examples by inability (or high complexity) of collecting the paired data set and of defining the ground-truth dependence between the samples. The examples are translating between cats and dogs, male and female faces, human and anime faces. Unpaired translation also includes many more specific tasks that arise in image editing, such as generating masks in mobile apps. Since we are not able to collect a paired data set, we define $(\mathbf{X}_0, \mathbf{X}_1)$ to be the independent samples from the corresponding domains p_0 and p_1.

In all the mentioned instances of the domain translation problems Flow Matching guarantees that if the learned ODE starts from the source distribution p_0, it will come to the target distribution p_1. Can we conclude that training Flow Matching will solve both paired and unpaired translation problems? Well, knowing that a sample from p_0 transforms into a sample from p_1 does not provide any guarantees that these samples will be connected! As an example, a cat can be transformed into a dog that has no common characterisitcs with the original cat. We need to somehow guarantee that some of the original properties of the picture (e.g. color palette) will be preserved. To this end, it is common to introduce the so-called transport cost between two samples:

Definition. Let $(\mathbf{X}, \mathbf{Y})$ be a pair of random variables and $c(\boldsymbol{x}, \boldsymbol{y})$ be a *cost function*, that acts as a measure of distance. We define **the transport cost** between $\mathbf{X}$ and $\mathbf{Y}$, or the transport cost of the joint distribution $p_{\mathbf{X},\mathbf{Y}}$ as the expected value

$$\mathbb{E}\, c(\mathbf{X}, \mathbf{Y}) \tag{90}$$

of the cost function.

Definition of the cost function depends on the properties of the objects that we aim to preserve. It can vary from the standard per-pixel L_2 cost $\|\boldsymbol{x} - \boldsymbol{y}\|^2$ to the semantic distance between the embeddings of the corresponding images.

Going back to the Flow Matching, what could one say about the transport cost $\mathbb{E}\, c(\mathbf{Y}_0, \mathbf{Y}_1)$ of the learned ODE

$$\begin{cases} \mathrm{d}\mathbf{Y}_t = f_t^*(\mathbf{Y}_t)\mathrm{d}t; \\ \mathbf{Y}_0 \sim p_0? \end{cases} \tag{91}$$

It turns out that one could guarantee that the transport cost between $\mathbf{Y}_0$ and $\mathbf{Y}_1$ does not exceed the transport cost between $\mathbf{X}_0$ and $\mathbf{X}_1$ for all (relatively simple) convex costs:

Theorem 1. *Let $(\mathbf{X}_0, \mathbf{X}_1)$ be a pair of samples from some joint distribution $p_{0,1}$ and $\mathbf{X}_t = t\mathbf{X}_1 + (1-t)\mathbf{X}_0$ be the corresponding linear constant-speed interpolation process. Let the transport cost $c(\boldsymbol{x}, \boldsymbol{y})$ be a convex function $\hat{c}$ of the difference: $\hat{c}(\boldsymbol{y} - \boldsymbol{x})$. If $\mathbf{Y}_t$ is the solution to the ODE, learned by the Flow Matching method, then*

$$\mathbb{E}\, c(\mathbf{Y}_0, \mathbf{Y}_1) \leq \mathbb{E}\, c(\mathbf{X}_0, \mathbf{X}_1). \tag{92}$$

Before starting the proof, we first note that it is crucial to use the **linear** interpolant, mentioned earlier, and that the following reasoning will not generally work for other interpolants. Second, this result is fundamentally connected with the illustration in the Figure 3. Learning ODE from the interpolation process means avoiding intersections, which leads to defining a map that is more «monotone» then the original pairing. Monotonocity (and the so-called cycle monotonicity in $\mathbb{R}^d$) is in fact a crucial property of a mapping that makes it «optimal» in the sence of minimizing the transport cost. The reason behind this is pretty simple: removing intersections reduces the average distance since the sum of the parallelogram diagonals is greater than the sum of its parallel sides. Now, let us move on to the proof.

Proof. Since $\mathbf{Y}_t$ satisfies the ODE $\mathrm{d}\mathbf{Y}_t = f_t^*(\mathbf{Y}_t)\mathrm{d}t$, one could rewrite the difference $\mathbf{Y}_1 - \mathbf{Y}_0$ using the Newton-Leibniz rule:

$$\mathbb{E}\, c(\mathbf{Y}_0, \mathbf{Y}_1) = \mathbb{E}\, \hat{c}(\mathbf{Y}_1 - \mathbf{Y}_0) = \mathbb{E}\, \hat{c}\left(\int_0^1 f_t^*(\mathbf{Y}_t)\mathrm{d}t\right). \tag{93}$$

What type of bounds can we apply if we see integrals, expectations and a convex function? Obviously, Jensen's inequality. Since we need an upper bound, we will not obtain anything by interchanging with the expectation. However, we can obtain an upper bound by swapping $\hat{c}$ and the integral (which defines a uniform distribution on $t \in [0, 1]$):

$$\mathbb{E}\, \hat{c}\left(\int_0^1 f_t^*(\mathbf{Y}_t)\mathrm{d}t\right) \leq \mathbb{E} \int_0^1 \hat{c}\left(f_t^*(\mathbf{Y}_t)\right)\mathrm{d}t. \tag{94}$$

Next, we need to somehow use the exact representation of the optimal velocity field $f_t^*(\boldsymbol{x}) = \mathbb{E}[\mathbf{X}_1 - \mathbf{X}_0 | \mathbf{X}_t = \boldsymbol{x}]$, or $f_t^*(\mathbf{X}_t) = \mathbb{E}[\mathbf{X}_1 - \mathbf{X}_0 | \mathbf{X}_t]$. Note, that we know how to properly express f_t^* if we evaluate it on the corresponding sample $\mathbf{X}_t$ from the interpolation process. Can we somehow replace $\mathbf{Y}_t$ with $\mathbf{X}_t$? Actually, the upper bound depends only on the marginal distributions $p_{\mathbf{Y}_t}$ of the process: by Fubini's theorem one can interchange integral and expectation and obtain

$$\int_0^1 \mathbb{E}\, \hat{c}(f_t^*(\mathbf{Y}_t))\mathrm{d}t = \int_0^1 \int_{\mathbb{R}^d} \hat{c}(f_t^*(\boldsymbol{y}))p_{\mathbf{Y}_t}(\boldsymbol{y})\mathrm{d}y\mathrm{d}t = \int_0^1 \int_{\mathbb{R}^d} \hat{c}(f_t^*(\boldsymbol{x}))p_{\mathbf{X}_t}(\boldsymbol{x})\mathrm{d}x\mathrm{d}t, \tag{95}$$

since by construction of the Flow Matching method $p_{\mathbf{X}_t} = p_{\mathbf{Y}_t}$. Thus, the upper bound is equal to

$$\int_0^1 \mathbb{E}\,\hat{c}(f_t^*(\mathbf{X}_t))\mathrm{d}t = \int_0^1 \mathbb{E}\,\hat{c}\left(\mathbb{E}[\mathbf{X}_1 - \mathbf{X}_0|\mathbf{X}_t]\right)\mathrm{d}t. \tag{96}$$

Here, we obtain another (conditional) expectation inside the convex function and by the Jensen's inequality we can interchange it to further upper bound the initial value:

$$\int_0^1 \mathbb{E}\,\hat{c}\left(\mathbb{E}[\mathbf{X}_1 - \mathbf{X}_0|\mathbf{X}_t]\right)\mathrm{d}t \leq \int_0^1 \mathbb{E}\,\mathbb{E}[\hat{c}(\mathbf{X}_1 - \mathbf{X}_0) \mid \mathbf{X}_t]\mathrm{d}t. \tag{97}$$

Due to the total expectation formula $\mathbb{E}\,\mathbb{E}[\xi|\eta] = \mathbb{E}\xi$, we obtain

$$\int_0^1 \mathbb{E}\,\mathbb{E}[\hat{c}(\mathbf{X}_1 - \mathbf{X}_0) \mid \mathbf{X}_t]\mathrm{d}t = \int_0^1 \mathbb{E}\,\hat{c}(\mathbf{X}_1 - \mathbf{X}_0)\mathrm{d}t = \mathbb{E}\,\hat{c}(\mathbf{X}_1 - \mathbf{X}_0), \tag{98}$$

which finishes the proof. □

What does the obtained result tells us? The transport cost between the input and the output of the model becomes not greater than the transport cost between the samples from the training data set. Generally, these are good news, given that the transport cost formalizes the input-output connection, essential for the domain translation problems. Unfortunately, this proof does not explicitly tell how much does the transport cost reduce after training. However, this is not a problem in case of the **paired** domain translation problems.

Here, we know by construction that the source sample $\mathbf{X}_0$ and the target sample $\mathbf{X}_1$ satisfy some functional dependencies and are close to each other. This is best illustrated by the inverse problems: if we take a picture from the data set, set a fixed subset of pixels to zero, then the corrupted version of the original picture is the closest to the original among all of the corrupted pictures (since for non-corrupted pixels the difference is zero). This means that the transport cost $\mathbb{E}\,\|\mathbf{X}_0 - \mathbf{X}_1\|^2$ is already the lowest possible among the samples from the corresponding distributions. As a consequence, $\mathbf{Y}_0$ and $\mathbf{Y}_1$ will also be connected.

This reasoning, however, only applies for such paired translation problems, for which L_2 or any other simple convex per-pixel distance adequately captures the desired connection between the samples. This is true for the mentioned inverse problems, but may fail for general paired translation problems such as Sketch→Photo, for which the per-pixel transport cost is not meaningful. We thus have proved the following informal statement:

Claim. *Flow Matching is applicable for such paired translation problems, for which convex transport costs properly capture the desired input-output connection [1].*

However, what can one say about the unpaired translation problems? Here, the transport cost $\mathbb{E}\, c(\mathbf{Y}_0, \mathbf{Y}_1)$ is not greater than the transport cost $\mathbb{E}\, c(\mathbf{X}_0, \mathbf{X}_1)$ between the independent samples $\mathbf{X}_0$ and $\mathbf{X}_1$, which are not connected at all. So, the transport cost $\mathbb{E}\, c(\mathbf{Y}_0, \mathbf{Y}_1)$ is upper-bounded with a large value, which does not imply anything. Therefore, training Flow Matching on independent samples from two domains generally does not lead to anything meaningful. In the next lecture, we will present the modification of the Flow Matching procedure, that will (theoretically) allow to apply it for the unpaired domain translation problems.

Lecture 8. Optimal Transport and Rectified Flow.

We finished the previous lecture by introducing the **domain translation** problems, in which we aim to translate samples from the source distribution $p^{\mathcal{S}}$ to the target distribution $p^{\mathcal{T}}$, while preserving the desired properties of the original sample. We discussed that the natural way to define this «preservation of properties» is by encoding them in the corresponding cost function $c(\boldsymbol{x}, \boldsymbol{y})$ and maintaining low transport cost $\mathbb{E}\, c(\mathbf{Y}_0, \mathbf{Y}_1)$, where $\mathbf{Y}_0$ and $\mathbf{Y}_1$ are input and output of the model, respectively. Thus, constructing a map G that solves the domain translation problem amounts in satisfying two properties:

1. If $\mathbf{X} \sim p^{\mathcal{S}}$, then $G(\mathbf{X}) \sim p^{\mathcal{T}}$;
2. Input $\mathbf{X}$ and output $G(\mathbf{X})$ are related with respect to the cost function $c(\boldsymbol{x}, \boldsymbol{y})$, i.e. the transport cost $\mathbb{E}\, c(\mathbf{X}, G(\mathbf{X})))$ is low.

Given this, one could formalize the domain translation problem mathematically through solving the following constrained optimization problem:

$$\begin{cases} \mathbb{E}\, c(\mathbf{X}, G(\mathbf{X})) \to \min\limits_{G}; \\ \mathbf{X} \sim p^{\mathcal{S}}; \quad G(\mathbf{X}) \sim p^{\mathcal{T}}. \end{cases} \tag{99}$$

The problem amounts in finding such a generator G that has the least transport cost among all generators that map $p^{\mathcal{S}}$ to $p^{\mathcal{T}}$. It is called the **Monge Problem**. Generally, problems that aim at minimizing the transport cost, while satisfying the distributional constraints, are called **optimal transport (OT)** problems. The generator that solves the Monge problem is called the **optimal transport map**. The Monge problem arises as one of the most natural ways to formulate a problem of this type. However, in general, Monge problem is not always convenient. For example, if both $p^{\mathcal{S}}$ and $p^{\mathcal{T}}$ are concentrated in a finite (but different) number of points, there is no generator that can map $p^{\mathcal{S}}$ to $p^{\mathcal{T}}$.

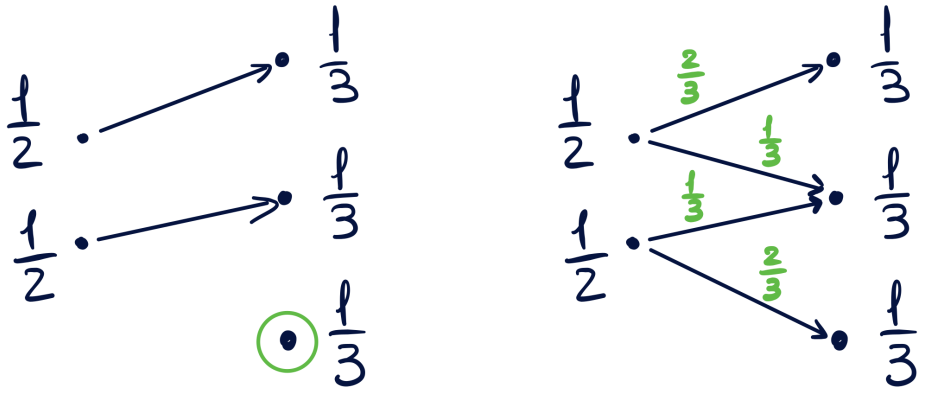

Figure 4: Difference between the Monge and the Kantorovich OT problems.

How could one modify the problem to overcome such type of issues? The answer is to allow «splitting» the mass, or defining the stochastic mapping instead of the deterministic one. How can we do it mathematically? Replace the generator G with the conditional distribution. The corresponding OT problem can be defined as the following:

$$\begin{cases} \int_{\mathbb{R}^d \times \mathbb{R}^d} c(\boldsymbol{x}, \boldsymbol{y}) p^{\mathcal{S}}(\boldsymbol{x}) \pi(\boldsymbol{y}|\boldsymbol{x}) \mathrm{d}\boldsymbol{x} \mathrm{d}\boldsymbol{y} \to \min_{\pi}; \\ \int_{\mathbb{R}^d} p^{\mathcal{S}}(\boldsymbol{x}) \pi(\boldsymbol{y}|\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = p^{\mathcal{T}}(\boldsymbol{y}), \end{cases} \tag{100}$$

which is called the **Kantorovich problem**. Here, the conditional distribution $\pi(\cdot|\boldsymbol{x})$ acts as a generator $G(\boldsymbol{x})$ replacement. The condition below requires this conditional distribution producing the target distribution at the output. The difference between the problems is illustrated in Figure 4: a deterministic generator cannot perfrom transportation between the uniform distributions at two and three points, while the «stochastic generator» can.

The Kantorovich problem is often defined in a little bit different way. Optimizing over conditional distributions is equivalent to optimizing over joint distribution, while satisfying the marginal constraint at the input. As previously, this joint distribution should also satisfy the marginal constraint at the output. Thus, we arrive at the following reformulation of the Kantorovich problem:

$$\begin{cases} \int_{\mathbb{R}^d \times \mathbb{R}^d} c(\boldsymbol{x}, \boldsymbol{y}) \pi(\boldsymbol{x}, \boldsymbol{y}) \mathrm{d}\boldsymbol{x} \mathrm{d}\boldsymbol{y} \to \min_{\pi}; \\ \int_{\mathbb{R}^d} \pi(\boldsymbol{x}, \boldsymbol{y}) \mathrm{d}\boldsymbol{y} = p^{\mathcal{S}}(\boldsymbol{x}); \\ \int_{\mathbb{R}^d} \pi(\boldsymbol{x}, \boldsymbol{y}) \mathrm{d}\boldsymbol{x} = p^{\mathcal{T}}(\boldsymbol{y}). \end{cases} \tag{101}$$

Any joint distribution that satisfies the aforementioned marginal constraints is called a **plan** between $p^{\mathcal{S}}$ and $p^{\mathcal{T}}$. The solution to the Kantorovich problem is called the **optimal**

transport plan, or just optimal plan between $p^{\mathcal{S}}$ and $p^{\mathcal{T}}$. This name comes from the most common interpretation of the Kantorovich problem. Imagine that $p^{\mathcal{S}}$ defines a distribution of buns produced in all bakeries in the cities, and $p^{\mathcal{T}}$ defines a distribution on the buns sold in the coffee shops. Then, $\pi(\boldsymbol{x}, \boldsymbol{y})$ defines a portion of buns (relative to the total number of buns) that is transported from the bakery $\boldsymbol{x}$ to the coffee shop $\boldsymbol{y}$. At the same time, the corresponding conditional distribution $\pi(\boldsymbol{y}|\boldsymbol{x}) := \pi(\boldsymbol{x}, \boldsymbol{y})/p^{\mathcal{S}}(\boldsymbol{x})$ defines the portion of buns transported from the bakery $\boldsymbol{x}$ to the coffee shop $\boldsymbol{y}$, relative to the total number of buns, produced in the bakery $\boldsymbol{x}$. In this setting, $c(\boldsymbol{x}, \boldsymbol{y})$ is interpreted as a cost of transporting a unit of buns from the bakery $\boldsymbol{x}$ to the coffee shop $\boldsymbol{y}$. Here, the Kantorovich OT problem corresponds to minimizing the total cost of transportation.

It is common to denote by $\Pi(p^{\mathcal{S}}, p^{\mathcal{T}})$ the set of all plans between $p^{\mathcal{S}}$ and $p^{\mathcal{T}}$, i.e. all the joint distributions that satisfy the aforementioned marginal constraints. Then, the Kantorovich problem can be shortly written as

$$\int_{\mathbb{R}^d \times \mathbb{R}^d} c(\boldsymbol{x}, \boldsymbol{y}) \pi(\boldsymbol{x}, \boldsymbol{y}) \mathrm{d}\boldsymbol{x} \mathrm{d}\boldsymbol{y} \to \min_{\pi \in \Pi(p^{\mathcal{S}}, p^{\mathcal{T}})}. \tag{102}$$

At the same time, it can be written in terms of random variables akin to the Monge problem:

$$\begin{cases} \mathbb{E}\, c(\mathbf{X}, \mathbf{Y}) \to \min\limits_{\pi}; \\ (\mathbf{X}, \mathbf{Y}) \sim \pi; \\ \mathbf{X} \sim p^{\mathcal{S}}; \quad \mathbf{Y} \sim p^{\mathcal{T}}. \end{cases} \tag{103}$$

While the Kantorovich problem is more convenient from the mathematical standpoint (i.e. the problem is convex on π and has is feasible since $\pi(\boldsymbol{x}, \boldsymbol{y}) = p^{\mathcal{S}}(\boldsymbol{x}) p^{\mathcal{T}}(\boldsymbol{y})$ satisfies the marginal constraints), it turns out that there is no difference between the two problems in the relevant scenarios:

Theorem 2. *If $p^{\mathcal{S}}$ and $p^{\mathcal{T}}$ have densities, then the Kantorovich problem is equivalent to the Monge problem. Specifically, if π^* is the optimal transport plan and G^* is the optimal transport map, then they are related as*

$$\pi^*(\boldsymbol{y}|\boldsymbol{x}) := \pi^*(\boldsymbol{x}, \boldsymbol{y})/p^{\mathcal{S}}(\boldsymbol{x}) = I\{\boldsymbol{y} = G^*(\boldsymbol{x})\}, \tag{104}$$

which means that the conditional distribution, induced by the optimal transport plan, is concentrated in one point, corresponding to the output of the optimal transport map.

This theorem will allow us to treat the optimal transport map and the optimal transport plan as synonims.

Rectified Flow

Now we return to investigating the Flow Matching method and its applications for the domain translation problems. Since we have mainly proved its applicability for the paired problems, now we are interested at solving the unpaired. But first let us reformulate what we achieved in the previous lecture. Let us define **Rectify**$(\cdot)$ to be the mapping that takes the joint distribution $p_{\mathbf{X}_0,\mathbf{X}_1}$, on which the Flow Matching is trained, and returns the joint distribution $p_{\mathbf{Y}_0,\mathbf{Y}_1}$ of the learned ODE

$$\begin{cases} \mathrm{d}\mathbf{Y}_t = f_t^*(\mathbf{Y}_t)\mathrm{d}t; \\ \mathbf{Y}_0 \sim p_{\mathbf{X}_0}. \end{cases} \tag{105}$$

Schematically, **Rectify** is represented in Algorithm 1.

Algorithm 1 Rectify.

Input: joint distribution (plan) $p_{\mathbf{X}_0,\mathbf{X}_1}$.
Output: joint distribution $p_{\mathbf{Y}_0,\mathbf{Y}_1}$, induced by the FM ODE.

$(\mathbf{X}_0, \mathbf{X}_1) \sim p_{\mathbf{X}_0,\mathbf{X}_1}$; ▷ sample the endpoints
$\mathbf{X}_t = t\mathbf{X}_1 + (1-t)\mathbf{X}_0$; ▷ define the linear interpolation process
$f_t^*(\boldsymbol{x}) = \mathbb{E}[\mathbf{X}_1 - \mathbf{X}_0 | \mathbf{X}_t = \boldsymbol{x}]$; ▷ learn the optimal velocity field
$\mathbf{Y}_0 \sim p_{\mathbf{X}_0}$; $\mathbf{Y}_1 = \mathrm{ODESolve}(\mathbf{Y}_0, 0 \to 1, f_t^*(\cdot))$. ▷ solve the learned ODE

Note that our previous observations about the Flow Matching method allow us to establish the two crucial properties of the **Rectify** procedure:

1. If $\mathbf{X}_0 \sim p^{\mathcal{S}}$ and $\mathbf{X}_1 \sim p^{\mathcal{T}}$, both its input $p_{\mathbf{X}_0,\mathbf{X}_1}$ and output $p_{\mathbf{Y}_0,\mathbf{Y}_1}$ are **plans** between the distributions $p^{\mathcal{S}}$ and $p^{\mathcal{T}}$ (since FM training preserves all marginal distributions);
2. The transport cost $\mathbb{E}\left\|\mathbf{Y}_1 - \mathbf{Y}_0\right\|^2$ of the plan $p_{\mathbf{Y}_0,\mathbf{Y}_1}$ does not exceed the transport cost $\mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2$ of the plan $p_{\mathbf{X}_0,\mathbf{X}_1}$.

Combination of these two observations may lead to a very natural idea. Since in the unpaired setting we are only able to generate independent samples from the corresponding domains, training Flow Matching does not guarantee the desired relation between the input and the output of the model. However, the transport cost is decreased and the marginal distributions are preserved. Then, one can repeat the procedure multiple times to decrease the transport cost, while still trasporting $p^{\mathcal{S}}$ to $p^{\mathcal{T}}$! This algorithm is called **Rectified Flow** [21, 20], and is summarized in Algorithm 2.

First, let us discuss how the algorithm will work in practice. Assume that we first have two independent data sets, denoted by the corresponding variables $\mathbf{X}_0 \sim p^{\mathcal{S}}$ and $\mathbf{X}_1 \sim p^{\mathcal{T}}$.

Algorithm 2 Rectified Flow.

Input: joint distribution (plan) $p_{\mathbf{X}_0,\mathbf{X}_1}$; number of iterations N.
Output: joint distribution $p_{\mathbf{Y}_0^{(N)},\mathbf{Y}_1^{(N)}}$, induced by the N-th FM ODE.

$p_{\mathbf{Y}_0^{(0)},\mathbf{Y}_1^{(0)}} := p_{\mathbf{X}_0,\mathbf{X}_1}$;
for $k = 1, \dots, N$ **do**
  $p_{\mathbf{Y}_0^{(k)},\mathbf{Y}_1^{(k)}} = \textbf{Rectify}(p_{\mathbf{Y}_0^{(k-1)},\mathbf{Y}_1^{(k-1)}})$ ▷ train FM on pairs from the previous FM
end for

First, we train the Flow Matching on these independent pairs and obtain an ODE, which produces a plan between $p^{\mathcal{S}}$ and $p^{\mathcal{T}}$ with a lower transport cost. To further reduce the transport cost, we initialize a new model and train it on the (dependent!) pairs generated by the previously trained model. The procedure is repeated multiple times. It has two main issues:

1. Each gradient step amounts in generating a whole ODE trajectory, which requires dozens evaluations of the previous neural network checkpoint;
2. Accumulation of error: each next Flow Matching can not generate samples of better quality than the samples of the previous model.

Both problems make practical usage of the Rectified Flow for the unpaired translation problems somewhat limited. However, the resulting model will have very appealing theoretical properties, which will allow to use it in some problems beyond unpaired domain translation. Furthermore, in the latter lectures we will construct the stochastic modification of the Rectified Flow procedure, that achieves competitive performance in solving the unpaired translation. Connection between the algorithms will be crucial to fully understand how they work.

Fixed points of the Rectify procedure

Going back to the theoretical description of the Rectified Flow, one could ask the following question: does the obtained sequence of plans converge? Does it converge to a plan with desired properties (e.g. to the optimal transport plan)? First, the sequence of transport costs is monotonic and bounded from below, thus it converges, i.e. from some point the algorithm almost does not reduce the transport cost. We will not prove that the sequence of plans converges. However, if it converges, it converges to a fixed point $p_{\mathbf{X}_0,\mathbf{X}_1}$ of the **Rectify** procedure in the following sence: the transport cost of its output $p_{\mathbf{Y}_0,\mathbf{Y}_1} = \textbf{Rectify}(p_{\mathbf{X}_0,\mathbf{X}_1})$ is not reduced compared to its input $p_{\mathbf{X}_0,\mathbf{X}_1}$. It turns out that this fixed point has many appealing properties:

Theorem 3. *Let* $(\mathbf{X}_0, \mathbf{X}_1)$ *be a pair of random variables corresponding to the input plan* $p_{\mathbf{X}_0,\mathbf{X}_1}$ *and* $(\mathbf{Y}_0, \mathbf{Y}_1)$ *be the pair of variables corresponding to the output* $p_{\mathbf{Y}_0,\mathbf{Y}_1} = \textbf{Rectify}(p_{\mathbf{X}_0,\mathbf{X}_1})$. *Then the following 4 statements are equivalent:*

1. *The transport cost does not reduce:* $\mathbb{E}\,\|\mathbf{Y}_1 - \mathbf{Y}_0\|^2 = \mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2$;
2. *The input* $p_{\mathbf{X}_0,\mathbf{X}_1}$ *and the output* $p_{\mathbf{Y}_0,\mathbf{Y}_1}$ *plans are equal, i.e. pairs* $(\mathbf{X}_0, \mathbf{X}_1)$ *and* $(\mathbf{Y}_0, \mathbf{Y}_1)$ *have the same distribution;*
3. *The interpolation process* $(\mathbf{X}_t, t \in [0, 1])$ *and the FM ODE* $(\mathbf{Y}_t, t \in [0, 1])$ *have the same distributions;*
4. *The following equation holds:*

$$\int_0^1 \mathbb{E}\|f_t^*(\mathbf{X}_t) - (\mathbf{X}_1 - \mathbf{X}_0)\|^2 \mathrm{d}t = 0. \tag{106}$$

Let us first tell what do these statements mean ideologically. The first statement tells that the transport cost does not reduce. The second statement tells that the plan $p_{\mathbf{X}_0,\mathbf{X}_1}$ is the fixed point of the **Rectify** procedure in a traditional sence (the input and output of the procedure are equal). This is a strong observation, since having equal transport costs does not generally imply having equal joint distributions.

The third statement enhances these observations even further and tells that not only the joint distributions of the endpoints coincide, but the whole input and output processes have the same distributions! This is in contrast with Table 3, which aim was to tell that the input interpolation process and the output ODE possess very different properties. The most powerful implication of this statement is that the ODE, learned from the fixed point $p_{\mathbf{X}_0,\mathbf{X}_1}$, have straight [5] trajectories, i.e. satisfies $\mathbf{Y}_t = t\mathbf{Y}_1 + (1-t)\mathbf{Y}_0$. In practice, it means that this ODE can be solved in just one step! This is a very desirable property for all the multistep generative models. It will further allow us to apply Rectified Flow for accelerating diffusion models.

The fourth statement is the most non-intuitive from the first glance. However, it actually defines a very reasonable property. Integral of the non-negative function equals zero if and only if the function is (almost surely) equal to zero, the same applies for the expectation. It means that for all t one has $f_t^*(\mathbf{X}_t) = \mathbf{X}_1 - \mathbf{X}_0$. Remember that the optimal velocity field is equal to

$$f_t^*(\mathbf{X}_t) = \mathbb{E}[\mathbf{X}_1 - \mathbf{X}_0 | \mathbf{X}_t]. \tag{107}$$

It means that the direction in which we should move to traverse from the current distribution to the target p_1 (as was illustrated in Figure 2), is completely determined from

[5] Throughout the course, we will call the trajectory straight, if it satisfies $\boldsymbol{x}_t = t\boldsymbol{x}_1 + (1-t)\boldsymbol{x}_0$. It would be more correct to call it «straight trajectory with constant speed», however we omit this remark for brevity.

the point $\mathbf{X}_t$. This means that for each $\mathbf{X}_t$ there is only one pair $(\mathbf{X}_0, \mathbf{X}_1)$ such that their interpolation traverses through $\mathbf{X}_t$ at the time t. Now, let us return to the proof.

Proof. First, the implications $3 \to 2$ and $2 \to 1$ are obvious: equal distribution of the processes imply equal distributions of their endpoints, which implies equal transport costs. We thus only need to prove $1 \to 4$ and $4 \to 3$.

$1 \to 4$. Here, we deal with the classical case, where there is an inequality that always holds true, and we need to perform some implications from the fact that the equality holds. Let us remind the main steps of proving the inequality $\mathbb{E}\|\mathbf{Y}_1 - \mathbf{Y}_0\|^2 \leq \mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2$:

$$\mathbb{E}\|\mathbf{Y}_1 - \mathbf{Y}_0\|^2 \leq \int_0^1 \mathbb{E}\|f_t^*(\mathbf{Y}_t)\|^2 \mathrm{d}t = \int_0^1 \mathbb{E}\|f_t^*(\mathbf{X}_t)\|^2 \mathrm{d}t \leq \mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2. \tag{108}$$

Since the equality holds, the latter two terms are equal:

$$\int_0^1 \mathbb{E}\|f_t^*(\mathbf{X}_t)\|^2 \mathrm{d}t = \mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2 = \int_0^1 \mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2 \mathrm{d}t, \tag{109}$$

which seems relevant to the equation that we need to prove:

$$\int_0^1 \mathbb{E}\|f_t^*(\mathbf{X}_t) - (\mathbf{X}_1 - \mathbf{X}_0)\|^2 \mathrm{d}t \stackrel{?}{=} 0. \tag{110}$$

It means that we actually aim at proving that

$$\int_0^1 \mathbb{E}\|f_t^*(\mathbf{X}_t) - (\mathbf{X}_1 - \mathbf{X}_0)\|^2 \mathrm{d}t = \int_0^1 \mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2 \mathrm{d}t - \int_0^1 \mathbb{E}\|f_t^*(\mathbf{X}_t)\|^2 \mathrm{d}t, \tag{111}$$

or, equivalently,

$$\int_0^1 \mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2 \mathrm{d}t = \int_0^1 \mathbb{E}\|f_t^*(\mathbf{X}_t)\|^2 \mathrm{d}t + \int_0^1 \mathbb{E}\|(\mathbf{X}_1 - \mathbf{X}_0) - f_t^*(\mathbf{X}_t)\|^2 \mathrm{d}t, \tag{112}$$

which looks like the Pythagorean theorem for the random variables and the expected squared norm $\mathbb{E}\|\cdot\|^2$ instead of the regular squared norm $\|\cdot\|^2$. In fact, this is indeed the Pythagorean theorem for the conditional expectation, which is a projection of a random variable into the space of functions of the random variable on which we condition. We first take the squared difference and represent it as the sum:

$$\mathbb{E}\|f_t^*(\mathbf{X}_t) - (\mathbf{X}_1 - \mathbf{X}_0)\|^2 = \mathbb{E}\|f_t^*(\mathbf{X}_t)\|^2 + \mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2 - 2\mathbb{E}\left\langle f_t^*(\mathbf{X}_t), \mathbf{X}_1 - \mathbf{X}_0 \right\rangle. \tag{113}$$

Now let us work with the dot product. Recall the law of total expectation:

$$\mathbb{E}\left\langle f_t^*(\mathbf{X}_t), \mathbf{X}_1 - \mathbf{X}_0 \right\rangle = \mathbb{E}\,\mathbb{E}\Big[\left\langle f_t^*(\mathbf{X}_t), \mathbf{X}_1 - \mathbf{X}_0 \right\rangle \mid \mathbf{X}_t\Big]. \tag{114}$$

The conditional expectation can be rewritten as the sum

$$\mathbb{E}\Big[\sum_i f_t^*(\mathbf{X}_t)_i(\mathbf{X}_1-\mathbf{X}_0)_i \mid \mathbf{X}_t\Big] = \sum_i \mathbb{E}\Big[f_t^*(\mathbf{X}_t)_i(\mathbf{X}_1-\mathbf{X}_0)_i \mid \mathbf{X}_t\Big]. \tag{115}$$

The first multiplier is a function of the condition, thus can be moved outside of the expectation. We thus obtain

$$\sum_i f_t^*(\mathbf{X}_t)_i \mathbb{E}\left[(\mathbf{X}_1-\mathbf{X}_0)_i|\mathbf{X}_t\right] = \left\langle f_t^*(\mathbf{X}_t), \mathbb{E}[\mathbf{X}_1-\mathbf{X}_0|\mathbf{X}_t]\right\rangle = \left\langle f_t^*(\mathbf{X}_t), f_t^*(\mathbf{X}_t)\right\rangle, \tag{116}$$

since the optimal velocity field is equal to $\mathbb{E}[\mathbf{X}_1-\mathbf{X}_0|\mathbf{X}_t]$. Thus, we obtain

$$\mathbb{E}\|f_t^*(\mathbf{X}_t)-(\mathbf{X}_1-\mathbf{X}_0)\|^2 = \mathbb{E}\|f_t^*(\mathbf{X}_t)\|^2 + \mathbb{E}\|\mathbf{X}_1-\mathbf{X}_0\|^2 - 2\,\mathbb{E}\|f_t^*(\mathbf{X}_t)\|^2 = \tag{117}$$

$$= \mathbb{E}\|\mathbf{X}_1-\mathbf{X}_0\|^2 - \mathbb{E}\|f_t^*(\mathbf{X}_t)\|^2. \tag{118}$$

Taking the integral at both sides completes the proof.

4 → 3. We know that the equality above holds true and need to prove that the processes $(\mathbf{X}_t, t\in[0,1])$ and $(\mathbf{Y}_t, t\in[0,1])$ have the same joint distributions. In fact, it is enough to prove that the interpolation process $\mathbf{X}_t$ satisfies the ODE $\mathrm{d}\mathbf{X}_t = f_t^*(\mathbf{X}_t)\mathrm{d}t$. If this is true, both processes will satisfy the same initial condition $p_{\mathbf{X}_0} = p_{\mathbf{Y}_0}$ and will have the same transitions $\mathbf{X}_{t+h} \approx \mathbf{X}_t + hf_t^*(\mathbf{X}_t)$ and $\mathbf{Y}_{t+h} \approx \mathbf{Y}_t + hf_t^*(\mathbf{Y}_t)$, which completely determines the distribution on trajectories. Well, we know that the equation from the fourth statement implies

$$\mathbf{X}_1-\mathbf{X}_0 = \mathbb{E}[\mathbf{X}_1-\mathbf{X}_0|\mathbf{X}_t]. \tag{119}$$

At the same time $\mathbf{X}_t = t\mathbf{X}_1 + (1-t)\mathbf{X}_0$, so the interpolation process satisfies

$$\mathrm{d}\mathbf{X}_t = (\mathbf{X}_1-\mathbf{X}_0)\mathrm{d}t = \mathbb{E}[\mathbf{X}_1-\mathbf{X}_0|\mathbf{X}_t]\mathrm{d}t = f_t^*(\mathbf{X}_t)\mathrm{d}t, \tag{120}$$

which completes the proof. □

As a consequence, we have proved the following crucial statement:

Corollary. If training Flow Matching does not reduce the transport cost compared to the joint distribution on training, the learned ODE has straight trajectories.

As we have discussed previously, this observation is itself very powerful, since it allows to use Rectified Flow as a tool to reduce the amount of computations at inference time by straightening paths of the corresponding ODEs. In addition to the properties of the fixed point, one could also define the measure of «straightness» $l(\mathbf{Y})$, defined for a process $\mathbf{Y}$ as

$$l(\mathbf{Y}) = \int_0^1 \mathbb{E}\|\dot{\mathbf{Y}} - (\mathbf{Y}_1-\mathbf{Y}_0)\|\mathrm{d}t \tag{121}$$

and prove [21] that the process $\mathbf{Y}^{(k)}$ produced by the k-th iteration of the Rectified Flow, achieves

$$l(\mathbf{Y}^{(k)}) = \mathcal{O}\left(\frac{1}{k}\right). \tag{122}$$

Thus, Rectified Flow provably straightens the trajectories, since low values of l mean that the velocity field along the path is in average close to the straight direction between the endpoints.

Connection with the OT map

We have observed numerous properties of the Rectified Flow: reduction of the transport cost, straightening of the trajectories. However, what can we say about its connection with the solution to the Monge/Kantorovich problem? Well, at least we can say that the OT plan π^* is the fixed point of the **Rectify** procedure: since π^* achieves the least transport cost between the distributions $p^{\mathcal{S}}$ and $p^{\mathcal{T}}$, it cannot be further reduced. Since π^* is the fixed point, it produces the ODE

$$\begin{cases} \mathrm{d}\mathbf{Y}_t = f_t^{\mathrm{OT}}(\mathbf{Y}_t)\mathrm{d}t; \\ \mathbf{Y}_0 \sim p^{\mathcal{S}}, \end{cases} \tag{123}$$

by training the Flow Matching (applying the **Rectify** procedure). This ODE has straight trajectories and defines the mapping

$$G(\mathbf{Y}_0) = \mathrm{ODESolve}(\mathbf{Y}_0, 0 \to 1, f_t^{\mathrm{OT}}(\cdot)),$$

which also achieves the optimal transport cost, since it is the fixed point! This means that the optimal transport plan/map can be represented by an ODE. Thus, Monge/Kantorovich OT problems could be solved by optimizing the equivalent problem

$$\begin{cases} \mathbb{E}\|\mathbf{Y}_1 - \mathbf{Y}_0\|^2 \to \min\limits_{f}; \\ \mathrm{d}\mathbf{Y}_t = f_t(\mathbf{Y}_t)\mathrm{d}t; \\ \mathbf{Y}_0 \sim p^{\mathcal{S}}; \quad \mathbf{Y}_1 \sim p^{\mathcal{T}}, \end{cases} \tag{124}$$

which produces the OT map in the form of an **ODE with the straight trajectories**.

It turns that the ODE-based OT problem can be modified even further and produce yet another interpretation of the OT problem in general. Let us come back to the sequence of equalities that hold true if the transport cost does not reduce. In this case, inequalities in Equation 108 become equalities:

$$\mathbb{E}\|\mathbf{Y}_1 - \mathbf{Y}_0\|^2 = \int_0^1 \mathbb{E}\|f_t^{\mathrm{OT}}(\mathbf{Y}_t)\|^2\mathrm{d}t = \int_0^1 \mathbb{E}\|f_t^{\mathrm{OT}}(\mathbf{X}_t)\|^2\mathrm{d}t = \mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2, \tag{125}$$

which means that the transport cost, induced by the OT ODE, is equal to

$$\int_0^1 \mathbb{E}\|f_t^{\text{OT}}(\mathbf{Y}_t)\|^2 \mathrm{d}t = \mathbb{E}\int_0^1 \|f_t^{\text{OT}}(\mathbf{Y}_t)\|^2 \mathrm{d}t, \tag{126}$$

which is called the **kinetic energy functional**, since it calculates the expected squared norm of the velocity field along the traversed path. It is indeed the kinetic energy of the trajectory up to multiplication by $m/2$. The transport cost in the optimal point coincides with the kinetic energy, while the kinetic energy upper bounds it in general. This means that the kinetic energy is a variational upper bound for the L_2 transport cost, and the problem

$$\begin{cases} \mathbb{E}\int_0^1 \|f_t(\mathbf{Y}_t)\|^2 \mathrm{d}t \to \min\limits_f; \\ \mathrm{d}\mathbf{Y}_t = f_t(\mathbf{Y}_t)\mathrm{d}t; \\ \mathbf{Y}_0 \sim p^{\mathcal{S}}; \quad \mathbf{Y}_1 \sim p^{\mathcal{T}}, \end{cases} \tag{127}$$

is equivalent to the ODE-based OT problem defined in Equation 124, thus also solves the Monge/Kantorovich problem! This problem is called **Dynamic Optimal Transport**, and the representation of the L_2 OT cost through the kinetic energy of the optimal ODE is called the Benamou-Brenier [3] formula. Summarizing all the formulations of the OT problem, we have obtained 4 equivalent problems: **Kantorovich, Monge, OT** with **ODE** and **Dynamic OT**:

$$\begin{cases} \mathbb{E}\|\mathbf{Y}_1 - \mathbf{Y}_0\|^2 \to \min\limits_\pi; \\ (\mathbf{Y}_0, \mathbf{Y}_1) \sim \pi; \\ \mathbf{Y}_0 \sim p^{\mathcal{S}}; \quad \mathbf{Y}_1 \sim p^{\mathcal{T}}; \end{cases} \iff \begin{cases} \mathbb{E}\|\mathbf{Y}_1 - \mathbf{Y}_0\|^2 \to \min\limits_G; \\ \mathbf{Y}_1 = G(\mathbf{Y}_0); \\ \mathbf{Y}_0 \sim p^{\mathcal{S}}; \quad \mathbf{Y}_1 \sim p^{\mathcal{T}}; \end{cases} \iff \tag{128}$$

$$\iff \begin{cases} \mathbb{E}\|\mathbf{Y}_1 - \mathbf{Y}_0\|^2 \to \min\limits_f; \\ \mathrm{d}\mathbf{Y}_t = f_t(\mathbf{Y}_t)\mathrm{d}t; \\ \mathbf{Y}_0 \sim p^{\mathcal{S}}; \quad \mathbf{Y}_1 \sim p^{\mathcal{T}}; \end{cases} \iff \begin{cases} \mathbb{E}\int_0^1 \|f_t(\mathbf{Y}_t)\|^2 \mathrm{d}t \to \min\limits_f; \\ \mathrm{d}\mathbf{Y}_t = f_t(\mathbf{Y}_t)\mathrm{d}t; \\ \mathbf{Y}_0 \sim p^{\mathcal{S}}; \quad \mathbf{Y}_1 \sim p^{\mathcal{T}}. \end{cases} \tag{129}$$

The Dynamic representation makes the established properties of the OT ODE much more illustrative: minimizing the kinetic energy along the path pushes the path to be as straight as possible! This also lead to the essential (one-directional) connection between the two practical problems:

Claim. *If one wants to obtain a generative model with straight trajectories, it is possible through solving the OT problem.*

The natural question, however, is whether the two properties of the process being represented by an ODE and having straight trajectories guarantee the corresponding map

to be optimal. From the practical standpoint, is it possible to solve the OT problem by constructing an ODE with straight trajectories? From the more theoretical standpoint, is the OT map **the only** fixed point of **Rectify**?

Unfortunately, the answer is **no**: one should add one more property to the ODE to make it solve the OT problem

Theorem 4. *[20] Let the ODE* $\mathrm{d}\mathbf{Y}_t = f_t(\mathbf{Y}_t)\mathrm{d}t$ *have straight trajectories and satisfy marginal constraints* $\mathbf{Y}_0 \sim p^{\mathcal{S}}$ *and* $\mathbf{Y}_1 \sim p^{\mathcal{T}}$*. Then, it solves the Dynamic OT problem if and only if the velocity field* $f_t(\boldsymbol{x})$ *can be represented as a gradient of the scalar field* $\nabla s_t(\boldsymbol{x})$*.*

While being counter-intuitive at the first glance, the result is strongly connected to the **curl** of the velocity field and such vector calculus results as Helmholtz decomposition that states that every vector field f can be decomposed into the sum of divergence-free and curl-free fields. In addition, it turns out that the vector field is curl-free if and only if it is a gradient of the scalar field. Thus, the ODE-based mapping is optimal if and only if **curl** of the corresponding velocity field is zero. From a physical standpoint [6], it means that the corresponding velocity field produces paths with no additional «rotations» that can increase path length and the kinetic energy along it.

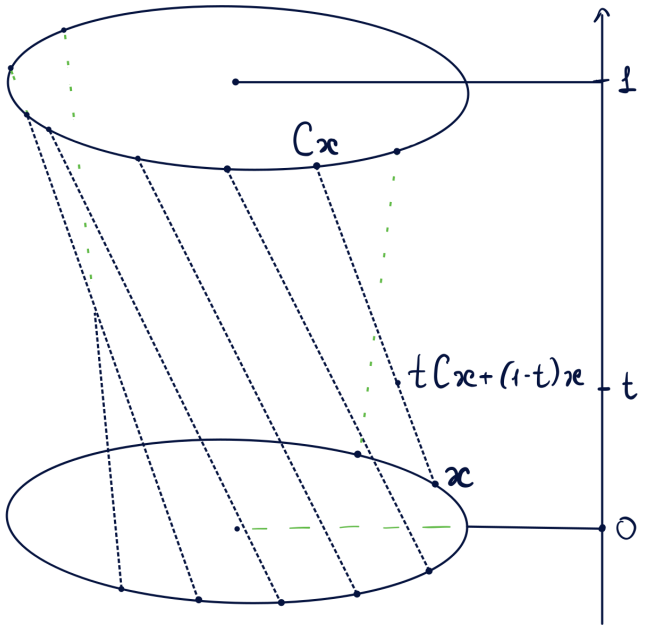

Figure 5: Example of an ODE with straight trajectories that is not optimal. Here, $\mathbf{Y}_t = tC\mathbf{Y}_0 + (1-t)\mathbf{Y}_0$ for a rotation matrix C and the $\mathcal{N}(0, I) \to \mathcal{N}(0, I)$ translation problem.

Meaning of the additional requirement is illustrated in Figure 5. Here we deal with a toy problem of transporting $\mathcal{N}(0, I)$ to itself. Obviously, the optimal transport map is

[6] The author has little experience with physics, so the corresponding reasonings may be somewhat inaccurate.

$G(\boldsymbol{x}) = \boldsymbol{x}$. However, there are numerous suboptimal transport maps of the form $G(\boldsymbol{x}) = C\boldsymbol{x}$ for an orthogonal matrix C. One could construct an interpolation process $\mathbf{X}_t = tC\mathbf{X}_0 + (1-t)\mathbf{X}_0$, which has straight trajectories. If one proves it has no intersections, it will mean that this process satisfies some ODE with straight trajectories and transports $\mathcal{N}(0, I)$ to itself. Finally, the connection with the requirement on the velocity field can be illustrated by the following exercise:

Exercise. Prove that if C is a rotation matrix, then $\mathbf{Y}_t = tC\mathbf{Y}_0 + (1-t)\mathbf{Y}_0$ has non-intersecting trajectories and satisfies some ODE $\mathrm{d}\mathbf{Y}_t = f_t(\mathbf{Y}_t)\mathrm{d}t$. Prove that $f_t(\boldsymbol{x})$ is a gradient of the scalar field for all t if and only if $f_t(\boldsymbol{x}) \equiv 0$, which implies $C = I$.

This example illustrates that the requirement on the velocity field excludes some scenarios, in which the mapping has unnecessary additional curvature. In practice, one can satisfy the requirement by parameterizing the velocity field as $f_t^\theta(\boldsymbol{x}) = \nabla s_t^\theta(\boldsymbol{x})$ [27, 28], however, this significantly reduces the architecture's capacity. Thus, in practice it is common to ignore this requirement and think of ODEs with straight trajectories as of a proxy guarantee that the corresponding map is optimal. In summary, we have obtained the following:

Claim. *If one needs to solve the OT problem, it is (almost) possible through training a generative model with straight trajectories. If one needs the exact theoretical guarantees, this generative model should be parameterized as* $\nabla s_t^\theta(\boldsymbol{x})$.

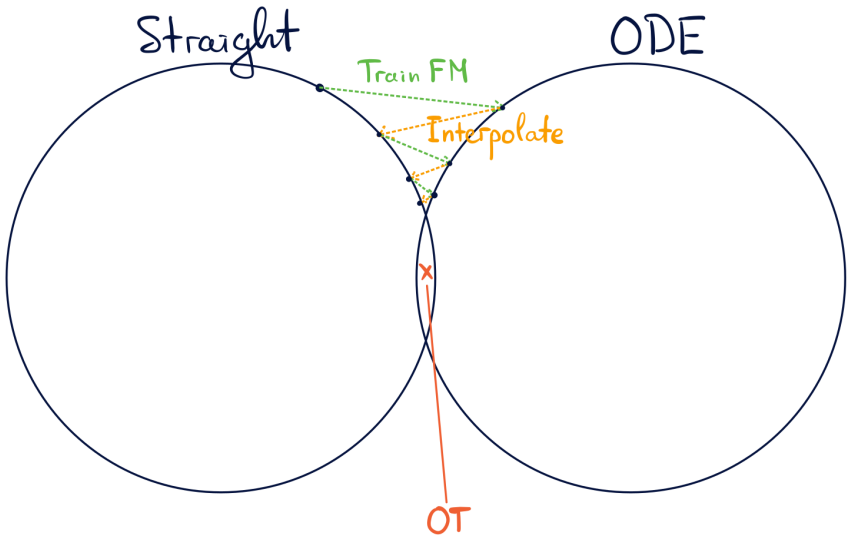

Figure 6: Interpretation of the Rectified Flow algorithm as a sequence of projections onto the set of straight paths and the set of processes that satisfy some ODE.

After making all of these theoretical observations, it is useful to go back to the Rectified Flow algorithm and interpret it in terms of this «duality» between the OT problem and ODEs with straight trajectories. Each rectification step can actually be seen as a

combination of two projections (see Figure 6). First takes the current learned process and projects it onto the set of straight processes by constructing the interpolation. Second transforms the interpolation process into the ODE by training Flow Matching on top of it. Naturally, this sequence should converge to a point, which lies in the intersection of the sets. While there is more than one point in this set, one of them is OT map and the others are (from the practical point of view) not very far from it.

Lecture 10. Bridge Matching.

The previous part of the course was mostly focused around the Flow Matching model. We first derived it as a model for **unconditional generation**, then proved that it reduces the transport cost, thus justified its usage for **paired translation** problems. Finally, we made an attempt to apply Flow Matching for **unpaired translation** problems through iterative training of new models on the couplings generated by the previous model. The obtained Rectified Flow procedure unfortunately has limited application for the unpaired translation, however it found its application in accelerating diffusion (arbitrary ODE-based) models by straightening their trajectories.

One of the only drawbacks of the Flow Matching is that its fundamentally an ODE-based model, thus it is able to generate only one output per input. This is not very important when applied to **unconditional generation** due to the connection of FM with the diffusion models (here the velocity field $f_t(\boldsymbol{x})$ is affinely connected with the score function, thus one can construct an SDE alternative). Overall, here one could just generate another Gaussian sample and obtain a different output. However, the situation is different in **translation problems**, where the input cannot be replaced or modified. Moreover, the ability to generate many outputs per each input is essential here, since e.g. in the inverse problems as super-resolution and inpainting there are many pictures that produce the degraded input. In this type of problems, the ability to generate many candidates (and then e.g. choose the best fitting one) is desirable. In addition, Rectified Flow can only be applied for accelerating ODE-based diffusion and not the SDE-based, which could also be useful. All this leads us to the idea of modifying the Flow Matching procedure in a way to produce an SDE with the desired dynamics instead of an ODE.

The model that we will build in the lecture is called **Bridge Matching** [31, 37, 39]. It is the best to illustrate its construction by constantly comparing with the original Flow Matching.

	Flow Matching	Bridge Matching
$(\mathbf{X}_0, \mathbf{X}_1) \sim p_{0,1}$	Arbitrary $p_{0,1}$	Arbitrary $p_{0,1}$
Interpolation $\mathbf{X}_t$	$\mathbf{X}_t \vert \mathbf{X}_0, \mathbf{X}_1 \sim p_{t\vert 0,1}(\cdot \vert \mathbf{X}_0, \mathbf{X}_1)$	$\mathbf{X}_t \vert \mathbf{X}_0, \mathbf{X}_1 \sim p_{t\vert 0,1}(\cdot \vert \mathbf{X}_0, \mathbf{X}_1)$
$p_{t\vert 0,1}(\boldsymbol{x} \vert \boldsymbol{x}_0, \boldsymbol{x}_1)$	ODE Velocity $f_{t\vert 0,1}(\boldsymbol{x} \vert \boldsymbol{x}_0, \boldsymbol{x}_1)$	**SDE** Velocity $f_{t\vert 0,1}(\boldsymbol{x} \vert \boldsymbol{x}_0, \boldsymbol{x}_1)$ Diff. coef. $g(t)$
$p_{\mathbf{X}_t}(\boldsymbol{x})$	ODE $f_t^*(\boldsymbol{x}) = \mathbb{E}[f_t(\mathbf{X}_t \vert \mathbf{X}_0, \mathbf{X}_1) \vert \mathbf{X}_t = \boldsymbol{x}]$	**SDE** $f_t^*(\boldsymbol{x}) =?\ \ g^*(t) =?$

Table 5: Summarised comparison between the Flow Matching and the Bridge Matching methods.

BM through conditional dynamics

As previously, we aim to construct a model that transports the original sample from the distribution p_0 to the distribution p_1. At training, we are given a data coupling $(\mathbf{X}_0, \mathbf{X}_1) \sim p_{0,1}$, where $\mathbf{X}_0 \sim p_0$ and $\mathbf{X}_1 \sim p_1$. We construct an interpolation process $\mathbf{X}_t$ by sampling the intermediate value from the distribution $p_{t|0,1}(\cdot|\mathbf{X}_0, \mathbf{X}_1)$ conditioned on the endpoints. Flow Matching then assumes that the conditional dynamics $p_{t|0,1}(\boldsymbol{x}|\boldsymbol{x}_0, \boldsymbol{x}_1)$ is generated by an ODE with the velocity field $f_t(\boldsymbol{x}|\boldsymbol{x}_0, \boldsymbol{x}_1)$. Here comes the first distinction between the algorithms: unlike Flow Matching, Bridge Matching assumes the conditional dynamics is generated by an SDE with the velocity field $f_t(\boldsymbol{x}|\boldsymbol{x}_0, \boldsymbol{x}_1)$ and the diffusion coefficient $g(t)$.

Previously we derived that the unconditional dynamics $p_{\mathbf{X}_t}(\boldsymbol{x})$ of the interpolation process $\mathbf{X}_t$ is also generated by the ODE with the velocity field $f_t^*(\boldsymbol{x}) = \mathbb{E}[f_t(\mathbf{X}_t|\mathbf{X}_0, \mathbf{X}_1)|\mathbf{X}_t = \boldsymbol{x}]$. Our next goal is to derive the analogous result for Bridge Matching.

Claim. *If the conditional dynamics $p_{t|0,1}(\boldsymbol{x}|\boldsymbol{x}_0, \boldsymbol{x}_1)$ is generated by an SDE with the velocity field $f_t(\boldsymbol{x}|\boldsymbol{x}_0, \boldsymbol{x}_1)$ and the diffusion coefficient $g(t)$, then the unconditional interpolation dynamics $p_{\mathbf{X}_t}$ is also generated by the SDE with the velocity field*

$$f_t^*(\boldsymbol{x}) = \mathbb{E}[f_t(\mathbf{X}_t|\mathbf{X}_0, \mathbf{X}_1)|\mathbf{X}_t = \boldsymbol{x}]$$

and the diffusion coefficient $g(t)$.

Proof. As previously, we use the duality between SDEs and the corresponding Fokker-Planck equations and consider the time derivative of the unconditional dynamics $p_{\mathbf{X}_t}$ and

connect it with the conditional dynamics:

$$\frac{\partial}{\partial t}p_{\mathbf{X}_t}(\boldsymbol{x}) = \frac{\partial}{\partial t}\int_{\mathbb{R}^d} p_{t|0,1}(\boldsymbol{x}|\boldsymbol{x}_0,\boldsymbol{x}_1)p_{0,1}(\boldsymbol{x}_0,\boldsymbol{x}_1)\mathrm{d}\boldsymbol{x}_0\mathrm{d}\boldsymbol{x}_1 = \tag{130}$$

$$= \int_{\mathbb{R}^d} \frac{\partial}{\partial t}p_{t|0,1}(\boldsymbol{x}|\boldsymbol{x}_0,\boldsymbol{x}_1)p_{0,1}(\boldsymbol{x}_0,\boldsymbol{x}_1)\mathrm{d}\boldsymbol{x}_0\mathrm{d}\boldsymbol{x}_1. \tag{131}$$

Since the conditional dynamics is generated by an SDE, it satisfies the corresponding F-P equation. Given this, we replace the time-derivative with the corresponding spatial derivatives and obtain:

$$-\int_{\mathbb{R}^d} \mathrm{div}\Big(f_t(\boldsymbol{x}|\boldsymbol{x}_0,\boldsymbol{x}_1)p_{t|0,1}(\boldsymbol{x}|\boldsymbol{x}_0,\boldsymbol{x}_1)\Big)p_{0,1}(\boldsymbol{x}_0,\boldsymbol{x}_1)\mathrm{d}\boldsymbol{x}_0\mathrm{d}\boldsymbol{x}_1 + \tag{132}$$

$$+\int_{\mathbb{R}^d} \frac{g^2(t)}{2}\Delta p_{t|0,1}(\boldsymbol{x}|\boldsymbol{x}_0,\boldsymbol{x}_1)p_{0,1}(\boldsymbol{x}_0,\boldsymbol{x}_1)\mathrm{d}\boldsymbol{x}_0\mathrm{d}\boldsymbol{x}_1. \tag{133}$$

Manipulations with the first summand fully replicate the analogous proof for FM: we move the divergence outside of the integral, divide and multiply by $p_t(\boldsymbol{x})$, encounter the Bayes' theorem for $p_{0,1|t}$ and obtain

$$-\mathrm{div}\left(p_t(\boldsymbol{x})\int_{\mathbb{R}^d} f_{t|0,1}(\boldsymbol{x}|\boldsymbol{x}_0,\boldsymbol{x}_1)p_{0,1|t}(\boldsymbol{x}_0,\boldsymbol{x}_1|\boldsymbol{x}_t)\mathrm{d}t\right). \tag{134}$$

As for the second summand, the situation is even simpler: the Laplacian contains only $\boldsymbol{x}$-derivatives and can be moved outside of integral:

$$\frac{g^2(t)}{2}\Delta\int_{\mathbb{R}^d} p_{t|0,1}(\boldsymbol{x}|\boldsymbol{x}_0,\boldsymbol{x}_1)p_{0,1}(\boldsymbol{x}_0,\boldsymbol{x}_1)\mathrm{d}\boldsymbol{x}_0\mathrm{d}\boldsymbol{x}_1 = \frac{g^2(t)}{2}\Delta p_t(\boldsymbol{x}). \tag{135}$$

Combining the two observations, we arrive at

$$\frac{\partial}{\partial t}p_{\mathbf{X}_t}(\boldsymbol{x}) = --\mathrm{div}\left(p_t(\boldsymbol{x})\int_{\mathbb{R}^d} f_{t|0,1}(\boldsymbol{x}|\boldsymbol{x}_0,\boldsymbol{x}_1)p_{0,1|t}(\boldsymbol{x}_0,\boldsymbol{x}_1|\boldsymbol{x}_t)\mathrm{d}t\right) + \frac{g^2(t)}{2}\Delta p_t(\boldsymbol{x}), \tag{136}$$

which means that the unconditional dynamics is generated by the SDE with the velocity field $f_t^*(\boldsymbol{x}) = \mathbb{E}[f_t(\mathbf{X}_t|\mathbf{X}_0,\mathbf{X}_1)|\mathbf{X}_t = \boldsymbol{x}]$ and the diffusion coefficient $g(t)$. □

Summarizing the obtained result, if the conditional dynamics is generated by the SDE, one can generate the unconditional interpolation dynamics by solving the SDE

$$\begin{cases}\mathrm{d}\mathbf{Y}_t = f_t^*(\mathbf{Y}_t)\mathrm{d}t + g(t)\mathrm{d}\mathbf{W}_t;\\ \mathbf{Y}_0 \sim p_0,\end{cases} \tag{137}$$

where $f_t^*(\boldsymbol{x})$ is the aforementioned conditional expectation, which can learned by solving the least squares problem

$$\int_0^1 \mathbb{E}\|f_t^\theta(\mathbf{X}_t) - f_t(\mathbf{X}_t|\mathbf{X}_0, \mathbf{X}_1)\|^2 \mathrm{d}t \to \min_\theta, \tag{138}$$

where $(\mathbf{X}_0, \mathbf{X}_1) \sim p_{0,1}$ and $\mathbf{X}_t|\mathbf{X}_0, \mathbf{X}_1 \sim p_{t|0,1}(\cdot|\mathbf{X}_0, \mathbf{X}_1)$.

We thus defined the training and sampling procedures for the Bridge Matching. However, there is still one important question that requires investigation: how to define an SDE that interpolated between two points? In case of FM, we needed to define an ODE, which was done by first defining an interpolant $I_t(\boldsymbol{x}_0, \boldsymbol{x}_1)$ and then differentiating it to obtain $f_t(\boldsymbol{x}|\boldsymbol{x}_0, \boldsymbol{x}_1) = \partial_t I_t(\boldsymbol{x}_0, \boldsymbol{x}_1)$. However, with SDEs the situation is not that evident.

Doob's h-transform

How could one define an SDE that interpolates between two points? First, one should only care about coming into the endpoint, since changing the initial value does not change the SDE. Then, how one could guarantee that the SDE comes into the prescribed endpoint? Condition it on this event!

Assume we are given an arbitrary SDE $\mathrm{d}\mathbf{X}_t = f_t(\mathbf{X}_t)\mathrm{d}t + g(t)\mathrm{d}\mathbf{W}_t$. Observe that if we manage to construct an SDE that generates the conditional dynamics $p_{\mathbf{X}_t|\mathbf{X}_1}(\boldsymbol{x}|\boldsymbol{x}_1)$, this will be the desired solution, since $p_{\mathbf{X}_1|\mathbf{X}_1}(\boldsymbol{x}|\boldsymbol{x}_1) = I\{\boldsymbol{x} = \boldsymbol{x}_1\}$ guarantees us the final condition. It does not, however, seem trivial at the first glance: $\mathbf{X}_t$ is a Markov process, which makes it simple to condition on the past, but not on the future. However, in the first part of the course we observed that the time reversal of the Markov process is also Markov (in case of Markov chains and SDEs). And if $\mathbf{X}_t^{(b)}$ is the time reversal of the process $\mathbf{X}_t$, then the conditional distribution $p_{\mathbf{X}_t|\mathbf{X}_1}$ transforms into the distribution $p_{\mathbf{X}_t^{(b)}|\mathbf{X}_0^{(b)}}$, conditioned on the initial point, which is much simpler!

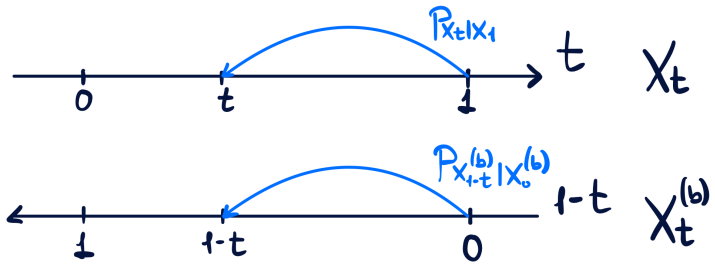

Figure 7: Illustration of the endpoint conditioning for the forward and the reverse processes.

Beauty of that observation lies in the fact that if one conditions a random process, satisfying some SDE, on its initial point, it still satisfies the same SDE. Thus, the conditional distribution $p_{\mathbf{X}_t^{(b)}|\mathbf{X}_0^{(b)}}$ of the backward process is generated by the same SDE as the backward process dynamics $p_{\mathbf{X}_t^{(b)}}$, which is obtained by constructing the time reversal of the forward dynamics $p_{\mathbf{X}_t}$, for which we know the corresponding SDE.

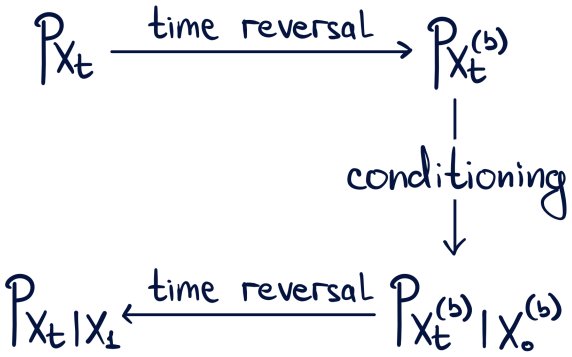

Figure 8: Illustration of derivation of the conditional dynamics $p_{\mathbf{X}_t|\mathbf{X}_0}$.

The scheme illustrated in the Figure 8 gives us the algorithm of finding the SDE that generates $p_{\mathbf{X}_t|\mathbf{X}_0}$. First, we know how to construct the backward SDE given the forward:

$$\mathrm{d}\mathbf{X}_t^{(b)} = \Big(-f_{1-t}(\mathbf{X}_t^{(b)}) + g^2(1-t)\nabla_{\boldsymbol{x}} \log p_{\mathbf{X}_t^{(b)}}(\mathbf{X}_t^{(b)})\Big)\,\mathrm{d}t + g(1-t)\mathrm{d}\mathbf{W}_t, \tag{139}$$

or, schematically, the triple

$$\Big(p_{\mathbf{X}_t}(\boldsymbol{x}), f_t(\boldsymbol{x}), g(t)\Big) \tag{140}$$

transforms into the triple

$$\Big(p_{\mathbf{X}_t^{(b)}}(\boldsymbol{x}),\ -f_{1-t}(\boldsymbol{x}) + g^2(1-t)\nabla_{\boldsymbol{x}} \log p_{\mathbf{X}_t^{(b)}}(\boldsymbol{x}),\ g(1-t)\Big). \tag{141}$$

Then, the conditioning stage does not affect the velocity and the diffusion coefficient, changing only the dynamics:

$$\Big(p_{\mathbf{X}_t^{(b)}|\mathbf{X}_0^{(b)}}(\boldsymbol{x}|\boldsymbol{x}_1),\ -f_{1-t}(\boldsymbol{x}) + g^2(1-t)\nabla_{\boldsymbol{x}} \log p_{\mathbf{X}_t^{(b)}}(\boldsymbol{x}),\ g(1-t)\Big). \tag{142}$$

Finally, time reversal of the conditional triple performs the same transformation and gives us the triple

$$\Big(p_{\mathbf{X}_t|\mathbf{X}_1}(\boldsymbol{x}|\boldsymbol{x}_1), v_t(\boldsymbol{x}|\boldsymbol{x}_1), g(t)\Big), \tag{143}$$

where

$$v_t(\boldsymbol{x}|\boldsymbol{x}_1) = f_t(\boldsymbol{x}) - g^2(t)\nabla_{\boldsymbol{x}} \log p_{\mathbf{X}_t}(\boldsymbol{x}) + g^2(t)\nabla_{\boldsymbol{x}} \log p_{\mathbf{X}_t|\mathbf{X}_1}(\boldsymbol{x}|\boldsymbol{x}_1). \tag{144}$$

Using the Bayes' theorem, we obtain

$$\Big(p_{\mathbf{X}_t|\mathbf{X}_1}(\boldsymbol{x}|\boldsymbol{x}_1),\ f_t(\boldsymbol{x}) + g^2(t)\nabla_{\boldsymbol{x}} \log p_{\mathbf{X}_1|\mathbf{X}_t}(\boldsymbol{x}_1|\boldsymbol{x}),\ g(t)\Big). \tag{145}$$

Additionally, if we condition on the initial point, both the velocity and the diffusion coefficient remain unchanged. Thus, we proved the following

Theorem 5 (Doob's h-transform)**.** *If the process $\mathbf{X}_t$ satisfies SDE $\mathrm{d}\mathbf{X}_t = f_t(\mathbf{X}_t)\mathrm{d}t + g(t)\mathrm{d}\mathbf{W}_t$, then the pinned process*

$$\begin{cases} \mathrm{d}\mathbf{X}_t^{0,1} = \Big(f_t(\mathbf{X}_t^{0,1}) + g^2(t)\nabla_{\boldsymbol{x}_t} \log p_{\mathbf{X}_1|\mathbf{X}_t}(\boldsymbol{x}_1|\mathbf{X}_t^{0,1})\Big)\,\mathrm{d}t + g(t)\mathrm{d}\mathbf{W}_t; \\ \mathbf{X}_0^{0,1} = \boldsymbol{x}_0 \end{cases} \tag{146}$$

generates the conditional dynamics $p_{\mathbf{X}_t|\mathbf{X}_0,\mathbf{X}_1}(\cdot|\boldsymbol{x}_0, \boldsymbol{x}_1)$ and, thus, interpolates between $\boldsymbol{x}_0$ and $\boldsymbol{x}_1$.

As it was previously with the construction of the time reversal, we proved the simplified version of the statement. Our result means that $p_{\mathbf{X}_t^{0,1}} = p_{\mathbf{X}_t|\mathbf{X}_0,\mathbf{X}_1}$ for each t, but what about the joint distributions? Can we say that the distribution of the whole process $\mathbf{X}_t^{0,1}$ coincides with the conditional distribution of the initial process $\mathbf{X}_t$? The answer is yes:

Theorem 6. *Distribution of the pinned process coincides with the conditional distributional of the initial process $\mathbf{X}_t$ i.e. for all $t_1, \ldots, t_n$*

$$p_{\mathbf{X}_{t_1}^{0,1},\ldots,\mathbf{X}_{t_n}^{0,1}}(\boldsymbol{x}_{t_1}, \ldots, \boldsymbol{x}_{t_n}) = p_{\mathbf{X}_{t_1},\ldots,\mathbf{X}_{t_n}|\mathbf{X}_0,\mathbf{X}_1}(\boldsymbol{x}_{t_1}, \ldots, \boldsymbol{x}_{t_n}|\boldsymbol{x}_0, \boldsymbol{x}_1) \tag{147}$$

holds true.

What does the form of the pinned process in the Equation 146 tells us? If one wants to modify an SDE-driven process in such a way that it comes into a predefined point $\boldsymbol{x}_1$, one should modify its direction by adding a term $g(t)\nabla_{\boldsymbol{x}_t} \log p_{\mathbf{X}_1|\mathbf{X}_t}(\boldsymbol{x}_1|\mathbf{X}_t)$, which corresponds to finding such points from which coming into $\boldsymbol{x}_1$ has high likelihood. The conditional endpoint likelihood $p_{\mathbf{X}_1|\mathbf{X}_t}(\boldsymbol{x}_1|\boldsymbol{x})$ is called the h-function. Why? Because this is originally not a standalone statement, but a special case of the more general Doob's h-transform technique that constructs an SDE for an arbitrary change of measure generated by the function, denoted by h [30, 17], which we do not touch upon here.

Brownian Bridge

Now, let us use the derived technique and construct the main family of interpolation processes that we will use. Let $\mathbf{X}_t$ be the process governed by the VE-SDE

$$\mathrm{d}\mathbf{X}_t = g(t)\mathrm{d}\mathbf{W}_t. \tag{148}$$

The only thing we need to calculate is the gradient $\nabla_{\boldsymbol{x}_t} \log p_{\mathbf{X}_1|\mathbf{X}_t}(\boldsymbol{x}_1|\boldsymbol{x})$. As always, let us derive the corresponding conditional distribution by the Euler discretization:

$$\mathbf{X}_1 \approx \mathbf{X}_t + \sum_i g(t_i)\sqrt{\Delta t_i}\varepsilon_{t_i}, \tag{149}$$

where $\boldsymbol{\varepsilon}_{t_i}$ are independent standard normal variables. Then, we obtain

$$p_{\mathbf{X}_1|\mathbf{X}_t}(\boldsymbol{x}_1|\boldsymbol{x}_t) \approx \mathcal{N}\left(\boldsymbol{x}_1 \mid \boldsymbol{x}_t, \sum_i g^2(t_i)\Delta t_i\right) \xrightarrow[\max_i \Delta t_i \to 0]{} \mathcal{N}\left(\boldsymbol{x}_1 \mid \boldsymbol{x}_t, \int_t^1 g^2(s)\mathrm{d}s\right). \tag{150}$$

Taking logarithm and differentiating, we obtain

$$\nabla_{\boldsymbol{x}_t} \log p_{\mathbf{X}_1|\mathbf{X}_t}(\boldsymbol{x}_1|\boldsymbol{x}_t) = \frac{\boldsymbol{x}_1 - \boldsymbol{x}_t}{\int_t^1 g^2(s)\mathrm{d}s}, \tag{151}$$

which leads to the pinned process

$$\mathrm{d}\mathbf{X}_t^{0,1} = \frac{g^2(t)(\boldsymbol{x}_1 - \mathbf{X}_t^{0,1})}{\int_t^1 g^2(s)\mathrm{d}s}\mathrm{d}t + g(t)\mathrm{d}\mathbf{W}_t, \tag{152}$$

which is called the *generalized Brownian Bridge*. An especially widely used example of the generalized Brownian Bridge is the Brownian Bridge :) which is obtained by setting a constant diffusion coefficient $g(t) = \sqrt{\gamma}$:

$$\mathrm{d}\mathbf{X}_t^{0,1} = \frac{\boldsymbol{x}_1 - \mathbf{X}_t^{0,1}}{1-t}\mathrm{d}t + \sqrt{\gamma}\mathrm{d}\mathbf{W}_t. \tag{153}$$

Bridge Matching with the Brownian interpolation

A remarkable property of the Brownian Bridge is that we know its distribution at each time t:

$$\mathbf{X}_t^{0,1} \sim \mathcal{N}(t\boldsymbol{x}_1 + (1-t)\boldsymbol{x}_0, \gamma t(1-t)I), \tag{154}$$

which gives us a possibility to sample the interpolation in one step. Notably, the conditional dynamics consists in a standard constant speed linear interpolation $t\boldsymbol{x}_1 + (1-t)\boldsymbol{x}_0$, which is mixed with the noise with parabolic variance which reaches its peak in the middle and vanishes at the endpoints.

Going back to the Bridge Matching algorithm, we will use the conditional dynamics

$$p_{t|0,1}(\boldsymbol{x}|\boldsymbol{x}_0, \boldsymbol{x}_1) = \mathcal{N}(\boldsymbol{x}|t\boldsymbol{x}_1 + (1-t)\boldsymbol{x}_0, \gamma t(1-t)I), \tag{155}$$

defined by the Brownian Bridge. Thus, the diffusion coefficient is equal to $\sqrt{\gamma}$ and the corresponding velocity field is equal to

$$f_t(\boldsymbol{x}|\boldsymbol{x}_0, \boldsymbol{x}_1) = \frac{\boldsymbol{x}_1 - \boldsymbol{x}}{1-t}, \tag{156}$$

which leads us to the following training procedure: sample the data pair $(\mathbf{X}_0, \mathbf{X}_1) \sim p_{0,1}$, interpolate between the endpoints as $\mathbf{X}_t = t\mathbf{X}_1 + (1-t)\mathbf{X}_0 + \sqrt{\gamma t(1-t)}\boldsymbol{\varepsilon}$ and optimize

$$\int_0^1 \mathbb{E}\left\| f_t^\theta(\mathbf{X}_t) - \frac{\mathbf{X}_1 - \mathbf{X}_t}{1-t} \right\|^2 \mathrm{d}t \to \min_\theta. \tag{157}$$

After training, one can translate between the distributions p_0 and p_1 by simply solving the corresponding SDE

$$\begin{cases} \mathrm{d}\mathbf{X}_t = f_t^\theta(\mathbf{X}_t)\mathrm{d}t + \sqrt{\gamma}\mathrm{d}\mathbf{W}_t; \\ \mathbf{X}_0 \sim p_0. \end{cases} \tag{158}$$

The resulting algorithm is very simple and similar to the original ODE-based Flow Matching procedure with the linear constant speed interpolation $\mathbf{X}_t = t\mathbf{X}_1 + (1-t)\mathbf{X}_0$. Its training objective consists in optimizing

$$\int_0^1 \mathbb{E}\| f_t^\theta(\mathbf{X}_t) - (\mathbf{X}_1 - \mathbf{X}_0)\|^2 \mathrm{d}t, \tag{159}$$

where $\mathbf{X}_1 - \mathbf{X}_0$ happens to be equal to $(\mathbf{X}_1 - \mathbf{X}_t)/(1-t)$, exactly as in Bridge Matching. Let us summarize the difference between the final versions of the algorithms:

	Flow Matching	Bridge Matching
$(\mathbf{X}_0, \mathbf{X}_1)$	Arbitrary $p_{0,1}$	Arbitrary $p_{0,1}$
Interp. $\mathbf{X}_t$	$\mathbf{X}_t = t\mathbf{X}_1 + (1-t)\mathbf{X}_0$	$\mathbf{X}_t = t\mathbf{X}_1 + (1-t)\mathbf{X}_0 + \sqrt{\gamma t(1-t)}\boldsymbol{\varepsilon}$
Training	$\int_0^1 \mathbb{E}\| f_t^\theta(\mathbf{X}_t) - \frac{\mathbf{X}_1 - \mathbf{X}_t}{1-t}\|^2 \mathrm{d}t \to \min_\theta$	$\int_0^1 \mathbb{E}\| f_t^\theta(\mathbf{X}_t) - \frac{\mathbf{X}_1 - \mathbf{X}_t}{1-t}\|^2 \mathrm{d}t \to \min_\theta$
Sampling	$\mathrm{d}\mathbf{Y}_t = f_t^\theta(\mathbf{Y}_t)\mathrm{d}t$	$\mathrm{d}\mathbf{Y}_t = f_t^\theta(\mathbf{Y}_t)\mathrm{d}t + \sqrt{\gamma}\mathrm{d}\mathbf{W}_t$

Table 6: Summarised comparison between the Flow Matching and the Bridge Matching methods.

Summing up, Bridge Matching, when picked with a Brownian Bridge as an interpolation process, offers a very straightforward one-to-many extension for the Flow Matching algorithm by using stochastic interpolation and SDE-based sampling. In practice, basic

of implementation of the two algorithms differs by 2-3 lines of code. Moreover, the additional stochasticity in the BM interpolation offers a natural regularization and often leads to higher-quality samples, when applied to tasks beyond unconditional generation.

The main question that could rise after the derivations is does the Bridge Matching somehow control the input-output transport cost as Flow Matching did? The proof specifically relied on deterministic interpolation and ODE-based sampling, so the extension is not obvious. For now, however, one can argue that for sufficiently small γ the procedures become close to each other, thus the Bridge Matching algorithm should also work correctly for the paired problems. This is validated in the paper Image-to-Image Schrödinger Bridge [19], which uses Bridge Matching for inverse problems and offers near-SOTA quality of generation. In theory, the connection between the Bridge Matching and the input-output transport cost is not that straightforward. To establish it, we will need to talk about the Entropic Optimal Transport and the Schrödinger Bridge problem.

Lecture 11. Entropic OT and Schrödinger Bridge

In the previous lecture we have defined Bridge Matching, an SDE modification of the Flow Matching method, which allows to perform one-to-many translation between two arbitrary distributions. However, we have not established its properties related to the transport cost between its input and the output to validate its usage for (at least) paired translation problems. Moreover, even in the deterministic case, Flow Matching was not directly applicable for the unpaired translation problems, and we needed to run the method multiple times to reduce transport cost and (at least, empirically) converge to the solution of the Kantorovich OT problem. However, if we aim to perform one-to-many translation, we need to modify the theoretical approach.

Previously, we mentioned that solving the Kantorovich OT problem results in a one-to-one deterministic generator: the optimal conditional distribution is concentrated in one point: $\pi^*(\boldsymbol{y}|\boldsymbol{x}) = I\{\boldsymbol{y} = G^*(\boldsymbol{x})\}$, where G^* is the solution to the Monge problem. Thus, even theoretically, we need to modify the OT problem to produce a stochastic generator and perform one-to-many translation. This problem is significant both in theory and practice: one could parameterize the generator as $G_\theta(\mathbf{X}, \mathbf{Z})$, where $\mathbf{Z}$ acts an additional source of randomness, and hope for it to learn a non-deterministic mapping. However, one will see the generator totally ignoring $\mathbf{Z}$ in full accordance with the theory. This is true, for example, for methods [16, 33, 4].

A natural solution consists in adding a regularization, which will encourage higher diversity of the aforementioned conditional distribution along with minimizing the original transport cost functional. The two most popular measures of diversity are variance and

entropy. While variance is a much easier statistic to estimate, it largely depends on samples' magnitude and can be unstable in practice. [7] Entropy, at the same time, depends only on the probability density, should be more robust, but, as we will observe, is much harder to estimate.

Entropy of the distribution p is defined as the functional

$$\mathcal{H}(p) = -\int_{\mathbb{R}^d} p(\boldsymbol{x}) \log p(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}. \tag{160}$$

Remark. In the continuous setting entropy is frequently called differential entropy to distinguish from the discrete case, where it has a more straightforward interpretation. If p is concentrated on n points $\{\boldsymbol{x}_1, \dots, \boldsymbol{x}_n\}$, entropy $-\sum_{i=1}^n p(\boldsymbol{x}_i) \log p(\boldsymbol{x}_i)$ is equal (up to ± 1) to the expected number of bits used for the optimal (in some sense) encoding of the corresponding outputs, called the Huffman code.

The corresponding optimization problem

$$\int_{\mathbb{R}^d \times \mathbb{R}^d} c(\boldsymbol{x}, \boldsymbol{y}) \mathrm{d}\pi(\boldsymbol{x}, \boldsymbol{y}) - \gamma \mathcal{H}(\pi) \to \min_{\pi \in \Pi(p^{\mathcal{S}}, p^{\mathcal{T}})} \tag{161}$$

is called the Optimal Transport problem with the entropic regularization, or the Entropic Optimal Transport (EOT). Here γ is the regularization coefficient which determines the balance between optimizing transport cost and producing diverse outputs. However, does adding a negative joint entropy solve our problem of diversifying the conditional distribution $\pi(\cdot|\boldsymbol{x})$? Turns out, regularizing the joint entropy is equivalent to regularizing the conditional:

$$\mathcal{H}(\pi) = -\int \pi(\boldsymbol{x}, \boldsymbol{y}) \log \pi(\boldsymbol{x}, \boldsymbol{y}) \mathrm{d}\boldsymbol{x} \mathrm{d}\boldsymbol{y} = \tag{162}$$

$$= -\int \pi(\boldsymbol{x}, \boldsymbol{y}) \log p^{\mathcal{S}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} \mathrm{d}\boldsymbol{y} - \int p^{\mathcal{S}}(\boldsymbol{x}) \left(\int \pi(\boldsymbol{y}|\boldsymbol{x}) \log \pi(\boldsymbol{y}|\boldsymbol{x}) \mathrm{d}\boldsymbol{y} \right) \mathrm{d}\boldsymbol{x} = \tag{163}$$

$$= \mathcal{H}(p^{\mathcal{S}}) + \int p^{\mathcal{S}}(\boldsymbol{x}) \mathcal{H}\left(\pi(\cdot|\boldsymbol{x})\right) \mathrm{d}\boldsymbol{x} = \mathcal{H}(p^{\mathcal{S}}) + \mathbb{E}\, \mathcal{H}(\pi(\cdot|\mathbf{X})). \tag{164}$$

Here, we have shown the chain rule for entropy: entropy of the joint distribution is equal to the sum of the marginal entropy and the expected conditional. In case of the OT problem marginal entropy $\mathcal{H}(p^{\mathcal{S}})$ does not depend on the optimized plan π, thus can be omitted. The EOT problem is then equivalent to

$$\int_{\mathbb{R}^d \times \mathbb{R}^d} c(\boldsymbol{x}, \boldsymbol{y}) \mathrm{d}\pi(\boldsymbol{x}, \boldsymbol{y}) - \gamma \int p^{\mathcal{S}}(\boldsymbol{x}) \mathcal{H}(\pi(\cdot|\boldsymbol{x})) \mathrm{d}\boldsymbol{x} \to \min_{\pi \in \Pi(p^{\mathcal{S}}, p^{\mathcal{T}})}, \tag{165}$$

[7] We encountered this effect even in 2d experiments with the RDMD [33] method, but determining the exact source of the problem requires investigation.

which explicitly encourages higher diversity of the conditional distribution given all conditions. In practice, we parameterize the conditional distribution with the stochastic generator $G_\theta(\mathbf{X}, \mathbf{Z})$ and let $\pi(\boldsymbol{y}|\boldsymbol{x}) = p_{G_\theta(\mathbf{X},\mathbf{Z})|\mathbf{X}}(\boldsymbol{y}|\boldsymbol{x})$ be the conditional distribution of its output given its input. This leads us to the formulation of the EOT problem in terms of the random variables:

$$\begin{cases} \mathbb{E}\, c\big(\mathbf{X}, G_\theta(\mathbf{X}, \mathbf{Z})\big) + \gamma\, \mathbb{E} \log p_{G_\theta(\mathbf{X},\mathbf{Z})|\mathbf{X}}\big(G_\theta(\mathbf{X}, \mathbf{Z})|\mathbf{X}\big) \to \min\limits_{\theta}; \\ \mathbf{X} \sim p^{\mathcal{S}};\;\; G_\theta(\mathbf{X}, \mathbf{Z}) \sim p^{\mathcal{T}}. \end{cases} \tag{166}$$

What distinguishes this problem from the Monge/Kantorovich formulations? The main issue consists in the necessity to estimate the log-likelihood of the generator's output given its input. This is feasible in a more or less one practical case where $G_\theta(\mathbf{X}, \mathbf{Z})$ is parameterized as the normalizing flow [5, 15], which is a very restrictive architecture. Another possible direction consists in tackling dual EOT formulations with energy-based models [26], which also has its limitations due to their image-to-number nature.

We have observed that the EOT problem, while possessing desirable properties, is hard to tackle with the current tools that we have. One of the keys for attempting to solve the EOT problem lies in its dynamic formulation, called the Schrödinger Bridge problem.

The Schrödinger Bridge problem in its most general form consists in finding a random process (more specifically, the corresponding distribution) that transports the source distribution $p^{\mathcal{S}}$ to the target distribution $p^{\mathcal{T}}$, while resembling some predefined *reference* process. Formally, we denote the reference process by $\mathbf{Z}$, the corresponding distribution by $\mathsf{P}_{\mathbf{Z}}$, assume that its initial distribution is equal to $p^{\mathcal{S}}$ and the process is governed by the SDE

$$\mathrm{d}\mathbf{Z}_t = f_t^{\mathbf{Z}}(\mathbf{Z}_t)\mathrm{d}t + \sqrt{\gamma}\mathrm{d}\mathbf{W}_t. \tag{167}$$

The Schrödinger Bridge (SB) Problem then consists in optimizing

$$\begin{cases} \mathrm{KL}\left(\mathsf{P}_{\mathbf{X}} \,\|\, \mathsf{P}_{\mathbf{Z}}\right) \to \min\limits_{\mathsf{P}_{\mathbf{X}}}; \\ \mathbf{X}_0 \sim p^{\mathcal{S}}; \\ \mathbf{X}_1 \sim p^{\mathcal{T}}. \end{cases} \tag{168}$$

The random process $\mathbf{X}$ (and the corresponding distribution $\mathsf{P}_{\mathbf{X}}$), that solves the problem, is called the Schrödinger Bridge between the distributions $p^{\mathcal{S}}$ and $p^{\mathcal{T}}$ with the reference process $\mathbf{Z}$ (reference distribution $\mathsf{P}_{\mathbf{Z}}$). The connection between the SB problem and the OT problems is not evident: previously, we aimed at minimizing the transport cost or the kinetic energy, but now minimize the KL divergence between the optimized process and some reference. We need some further steps to establish this connection.

KL divergence and random processes

First, how do we even define the distribution $\mathsf{P}_{\mathbf{X}}$ of the random process $\mathbf{X}$? We will not go deep into the definition, but note that we will consider only random processes with continuous trajectories, for which the distribution is fully described with the finite-dimensional distributions of its projections $\{p_{\mathbf{X}_{i_1},\dots,\mathbf{X}_{i_n}} | n \in \mathbb{N}, i_1, \dots, i_n \in [0,1]\}$. Unfortunately, this does not solve the issue, since we need to work with the KL divergence, which inevitably relies on the definition of density, unavailable for the random processes [8].

To access the KL divergence between random processes through their finite projections, we will need the following observations. The first allows to decompose the KL divergence between two densities in $\mathbb{R}^d$ into the sum of KL divergences of its subvectors. Let $\mathbf{X} = (\mathbf{X}_1, \dots, \mathbf{X}_d)$ and $\mathbf{Z} = (\mathbf{Z}_1, \dots, \mathbf{Z}_d)$ be random vectors, and $I \subset \{1, \dots, d\}$, $J = \{1, \dots, d\}/I$ be the two distinct sets of indices. Then

$$\mathrm{KL}(p_{\mathbf{X}} \,\|\, p_{\mathbf{Z}}) = \int_{\mathbb{R}^d} p_{\mathbf{X}}(\boldsymbol{x}) \log \frac{p_{\mathbf{X}}(\boldsymbol{x})}{p_{\mathbf{Z}}(\boldsymbol{x})} \mathrm{d}\boldsymbol{x} = \tag{169}$$

$$= \int_{\mathbb{R}^d} p_{\mathbf{X}_I}(\boldsymbol{x}_I) p_{\mathbf{X}_J|\mathbf{X}_I}(\boldsymbol{x}_J|\boldsymbol{x}_I) \log \frac{p_{\mathbf{X}_I}(\boldsymbol{x}_I) p_{\mathbf{X}_J|\mathbf{X}_I}(\boldsymbol{x}_J|\boldsymbol{x}_I)}{p_{\mathbf{Z}_I}(\boldsymbol{x}_I) p_{\mathbf{Z}_J|\mathbf{Z}_I}(\boldsymbol{x}_J|\boldsymbol{x}_I)} \mathrm{d}\boldsymbol{x} = \tag{170}$$

$$= \int_{\mathbb{R}^{|I|}} p_{\mathbf{X}_I}(\boldsymbol{x}_I) \log \frac{p_{\mathbf{X}_I}(\boldsymbol{x}_I)}{p_{\mathbf{Z}_I}(\boldsymbol{x}_I)} \mathrm{d}\boldsymbol{x}_I + \tag{171}$$

$$+ \int_{\mathbb{R}^{|I|}} p_{\mathbf{X}_I}(\boldsymbol{x}_I) \left(\int_{\mathbb{R}^{|J|}} p_{\mathbf{X}_J|\mathbf{X}_I}(\boldsymbol{x}_J|\boldsymbol{x}_I) \log \frac{p_{\mathbf{X}_J|\mathbf{X}_I}(\boldsymbol{x}_J|\boldsymbol{x}_I)}{p_{\mathbf{Z}_J|\mathbf{Z}_I}(\boldsymbol{x}_J|\boldsymbol{x}_I)} \mathrm{d}\boldsymbol{x}_J \right) \mathrm{d}\boldsymbol{x}_I. \tag{172}$$

The latter is simply the sum of the KL divergence between the distributions on a set of indices I and the expected KL divergences between the conditional distributions on the indices J:

$$\mathrm{KL}\big(p_{\mathbf{X}} \,\|\, p_{\mathbf{Z}}\big) = \mathrm{KL}\big(p_{\mathbf{X}_I} \,\|\, p_{\mathbf{Z}_I}\big) + \int_{\mathbb{R}^{|I|}} p_{\mathbf{X}_I}(\boldsymbol{x}_I) \mathrm{KL}\left(p_{\mathbf{X}_J|\mathbf{X}_I}(\cdot|\boldsymbol{x}_I) \,\|\, p_{\mathbf{Z}_J|\mathbf{Z}_I}(\cdot|\boldsymbol{x}_I)\right) \mathrm{d}\boldsymbol{x}_I = \tag{173}$$

$$= \mathrm{KL}\big(p_{\mathbf{X}_I} \,\|\, p_{\mathbf{Z}_I}\big) + \mathbb{E}\, \mathrm{KL}\left(p_{\mathbf{X}_J|\mathbf{X}_I}(\cdot|\mathbf{X}_I) \,\|\, p_{\mathbf{Z}_J|\mathbf{Z}_I}(\cdot|\mathbf{X}_I)\right). \tag{174}$$

This result is called the chain rule for the KL divergence and is very similar to the analogous result for entropy. One of its consequences is that reducing the number of components results in reducing the KL divergence:

$$\mathrm{KL}\big(p_{\mathbf{X}} \,\|\, p_{\mathbf{Z}}\big) \geq \mathrm{KL}\big(p_{\mathbf{X}_I} \,\|\, p_{\mathbf{Z}_I}\big). \tag{175}$$

[8] However, in case of general probability spaces, one can define the analogue of the density ratio $\frac{\mathrm{dP}}{\mathrm{dQ}}$, called the Radon-Nikodym derivative.

In fact, this result is a special case of a so-called data-processing inequality:

Claim (Data-processing inequality)**.** *Let* $\mathbf{X}$ *and* $\mathbf{Z}$ *be two random elements in any probability space and* f *be an arbitrary function. Then the KL divergence between the distributions of the transformed variables is bounded by the original KL divergence:*

$$\mathrm{KL}\big(p_{\mathbf{X}} \,\|\, p_{\mathbf{Z}}\big) \geq \mathrm{KL}\big(p_{f(\mathbf{X})} \,\|\, p_{f(\mathbf{Z})}\big). \tag{176}$$

Proof. Let us consider the case of two random vectors $\mathbf{X}, \mathbf{Z} \in \mathbb{R}^d$. Then, the previously established chain rule states that

$$\mathrm{KL}\big(p_{\mathbf{X},f(\mathbf{X})} \,\|\, p_{\mathbf{Z},f(\mathbf{Z})}\big) \geq \mathrm{KL}\big(p_{f(\mathbf{X})} \,\|\, p_{f(\mathbf{Z})}\big). \tag{177}$$

On the other hand, we can perform the chain rule in the opposite direction:

$$\mathrm{KL}\big(p_{\mathbf{X},f(\mathbf{X})} \,\|\, p_{\mathbf{Z},f(\mathbf{Z})}\big) = \mathrm{KL}\big(p_{\mathbf{X}} \,\|\, p_{\mathbf{Z}}\big) + \mathbb{E}\,\mathrm{KL}\big(p_{f(\mathbf{X})|\mathbf{X}}(\cdot|\mathbf{X}) \,\|\, p_{f(\mathbf{Z})|\mathbf{Z}}(\cdot|\mathbf{X})\big). \tag{178}$$

Since both conditional distributions are concentrated in one point $f(\mathbf{X})$, the second KL divergence is zero, and we obtain

$$\mathrm{KL}\big(p_{\mathbf{X}} \,\|\, p_{\mathbf{Z}}\big) = \mathrm{KL}\big(p_{\mathbf{X},f(\mathbf{X})} \,\|\, p_{\mathbf{Z},f(\mathbf{Z})}\big) \geq \mathrm{KL}\big(p_{f(\mathbf{X})} \,\|\, p_{f(\mathbf{Z})}\big), \tag{179}$$

which is what we aimed for. □

The intuition behind the inequality is pretty straightforward: applying the same function reduces the amount of differences between the two distributions, which leads to the decrease in the KL divergence. The previously obtained result that reducing the number of components reduces the KL divergence is a special instance of the data-processing inequality, where $f(\boldsymbol{x}) = \boldsymbol{x}_I$.

Despite the fact that we have not explicitly define the KL divergence between the distributions $\mathsf{P}_{\mathbf{X}}$ and $\mathsf{P}_{\mathbf{Z}}$ of two random processes, we can now perform several observations. First, data-processing inequality states that the corresponding KL divergence is bounded from below by the KL between the finite-dimensional projections:

$$\mathrm{KL}\big(\mathsf{P}_{\mathbf{X}} \,\|\, \mathsf{P}_{\mathbf{Z}}\big) \geq \mathrm{KL}\big(p_{\mathbf{X}_{t_1},\ldots,\mathbf{X}_{t_n}} \,\|\, p_{\mathbf{Z}_{t_1},\ldots,\mathbf{Z}_{t_n}}\big) \tag{180}$$

for all $t_1, \ldots, t_n \in [0, 1]$. On the other hand, intuition suggests that if the sequence of timesteps partitions the interval $0 = t_0 < t_1 < \ldots < t_n = 1$ and the partition diameter $\max_{i=1\ldots n} \Delta t_i$ is close to zero, then any continuous function is closely approximated by its, say, piecewise-linear version. Thus, the distributions of the original processes should be close to the distributions of their approximations, completely determined by the corresponding finite projections. This leads us to the (pseudo)-definition of the KL divergence between two random processes:

Definition. KL divergence between the distributions of two random processes is defined as the limit of the corresponding projections, as the partition diameters tends to zero:

$$\mathrm{KL}\big(\mathsf{P}_{\mathbf{X}} \,\|\, \mathsf{P}_{\mathbf{Z}}\big) = \lim_{\substack{\max_i \Delta t_i \to 0 \\ 0=t_0<t_1<\ldots<t_n=1}} \mathrm{KL}\big(p_{\mathbf{X}_{t_0},\ldots,\mathbf{X}_{t_n}} \,\|\, p_{\mathbf{Z}_{t_0},\ldots,\mathbf{Z}_{t_n}}\big). \tag{181}$$

This definition, while being informal, gives us a possibility to further study the Schrödinger Bridge problem. Our goal remains the same: build the connection between the SB problem and the EOT problem. To fulfill it, we should somehow represent the optimized KL divergence as a functional of the joint distribution of the endpoints $p_{\mathbf{X}_0,\mathbf{X}_1}$. To this end, we will heavily rely on the following result:

Claim. *KL divergence between the distributions $\mathsf{P}_{\mathbf{X}}$ and $\mathsf{P}_{\mathbf{Z}}$ of the two random processes is equal to the sum of the KL divergence between their endpoints and the expected KL divergence between the conditional «bridge» distributions:*

$$\mathrm{KL}\big(\mathsf{P}_{\mathbf{X}} \,\|\, \mathsf{P}_{\mathbf{Z}}\big) = \mathrm{KL}\big(p_{\mathbf{X}_0,\mathbf{X}_1} \,\|\, p_{\mathbf{Z}_0,\mathbf{Z}_1}\big) + \mathbb{E}\,\mathrm{KL}\big(\mathsf{P}_{\mathbf{X}|\mathbf{X}_0,\mathbf{X}_1}(\cdot|\mathbf{X}_0,\mathbf{X}_1) \,\|\, \mathsf{P}_{\mathbf{Z}|\mathbf{Z}_0,\mathbf{Z}_1}(\cdot|\mathbf{X}_0,\mathbf{X}_1)\big).$$

Proof. The resuls is a simple combination of our definition and the chain rule for random vectors. We first write KL as the limit and perform the chain rule under the limit:

$$\mathrm{KL}\big(\mathsf{P}_{\mathbf{X}} \,\|\, \mathsf{P}_{\mathbf{Z}}\big) = \lim_{\substack{\max_i \Delta t_i \to 0 \\ 0=t_0<t_1<\ldots<t_n=1}} \mathrm{KL}\big(p_{\mathbf{X}_{t_0},\ldots,\mathbf{X}_{t_n}} \,\|\, p_{\mathbf{Z}_{t_0},\ldots,\mathbf{Z}_{t_n}}\big) \tag{182}$$

$$= \lim_{\substack{\max_i \Delta t_i \to 0 \\ 0=t_0<t_1<\ldots<t_n=1}} \Big(\mathrm{KL}\big(p_{\mathbf{X}_0,\mathbf{X}_1} \,\|\, p_{\mathbf{Z}_0,\mathbf{Z}_1}\big) \tag{183}$$

$$+ \mathbb{E}\,\mathrm{KL}\big(p_{\mathbf{X}_{t_0},\ldots,\mathbf{X}_{t_n}|\mathbf{X}_0,\mathbf{X}_1}(\cdot|\mathbf{X}_0,\mathbf{X}_1) \,\|\, p_{\mathbf{Z}_{t_0},\ldots,\mathbf{Z}_{t_n}|\mathbf{Z}_0,\mathbf{Z}_1}(\cdot|\mathbf{X}_0,\mathbf{X}_1)\big)\Big). \tag{184}$$

The first summand does not depend on the limit and can be moved outside. At the same time, we can (as always in our course) move the limit inside the expectation and obtain the definition of the KL divergence:

$$\lim_{\substack{\max_i \Delta t_i \to 0 \\ 0=t_0<t_1<\ldots<t_n=1}} \mathrm{KL}\big(p_{\mathbf{X}_{t_0},\ldots,\mathbf{X}_{t_n}|\mathbf{X}_0,\mathbf{X}_1}(\cdot|\mathbf{X}_0,\mathbf{X}_1) \,\|\, p_{\mathbf{Z}_{t_0},\ldots,\mathbf{Z}_{t_n}|\mathbf{Z}_0,\mathbf{Z}_1}(\cdot|\mathbf{X}_0,\mathbf{X}_1)\big) = \tag{185}$$

$$= \mathrm{KL}\big(\mathsf{P}_{\mathbf{X}|\mathbf{X}_0,\mathbf{X}_1}(\cdot|\mathbf{X}_0,\mathbf{X}_1) \| \mathsf{P}_{\mathbf{Z}|\mathbf{Z}_0,\mathbf{Z}_1}(\cdot|\mathbf{Z}_0,\mathbf{Z}_1)\big). \tag{186}$$

□

Schrödinger Bridge and Entopic OT

With this observation, we can finally move on towards investigating properties of the Schrödinger Bridge. Recall that we optimize the KL divergence $\mathrm{KL}(\mathsf{P}_{\mathbf{X}} \,\|\, \mathsf{P}_{\mathbf{Z}})$ given the

boundary conditions on $p_{\mathbf{X}_0}$ and $p_{\mathbf{X}_1}$. The chain rule above actually allows to decompose the optimized functional into the part $\mathrm{KL}(p_{\mathbf{X}_0,\mathbf{X}_1} \,\|\, p_{\mathbf{Z}_0,\mathbf{Z}_1})$, which still depends on the conditions, and the KL divergence between bridges, which can be optimized without taking them into account! Indeed, if one sets

$$\mathsf{P}_{\mathbf{X}|\mathbf{X}_0,\mathbf{X}_1}(\cdot|\boldsymbol{x}_0,\boldsymbol{x}_1) = \mathsf{P}_{\mathbf{Z}|\mathbf{Z}_0,\mathbf{Z}_1}(\cdot|\boldsymbol{x}_0,\boldsymbol{x}_1) \tag{187}$$

for all $\boldsymbol{x}_0, \boldsymbol{x}_1$, the second part of the KL divergence becomes zero reagardless of the joint distribution $p_{\mathbf{X}_0,\mathbf{X}_1}$ and the corresponding boundaries. Thus, we have established one very important property of the Schrödinger Bridge: **it has the same conditional bridge distribution as the reference process Z**. But can we describe this distribution in a more constructive way (say, generate it with some SDE)? The answer is positive and is given by the Doob's SDE:

$$\mathrm{d}\mathbf{Z}_t^{0,1} = \Big(f_t^{\mathbf{Z}}(\mathbf{Z}_t^{0,1}) + \gamma\nabla_{\boldsymbol{z}_t}\log p_{\mathbf{Z}_1|\mathbf{Z}_t}(\boldsymbol{x}_1|\mathbf{Z}_t)\Big)\,\mathrm{d}t + \sqrt{\gamma}\mathrm{d}\mathbf{W}_t, \tag{188}$$

which, as previously established, is distributed as $\mathsf{P}_{\mathbf{Z}|\mathbf{Z}_0,\mathbf{Z}_1}(\cdot|\boldsymbol{x}_0,\boldsymbol{x}_1)$.

From this point on, we will consider the (multiplied) Wiener process, or VE-SDE with constant diffusion coefficient

$$\mathrm{d}\mathbf{Z}_t = \sqrt{\gamma}\mathrm{d}\mathbf{W}_t, \tag{189}$$

as reference. The corresponding bridge distribution corresponds to the Brownian bridge and is generated by the SDE

$$\mathrm{d}\mathbf{Z}_t^{0,1} = \frac{\boldsymbol{x}_1 - \mathbf{Z}_t^{0,1}}{1-t}\mathrm{d}t + \sqrt{\gamma}\mathrm{d}\mathbf{W}_t. \tag{190}$$

The only remaining part of the original SB problem consists in optimizing the KL divergence between the joint distribution of the endpoints:

$$\begin{cases} \mathrm{KL}(p_{\mathbf{X}_0,\mathbf{X}_1} \,\|\, p_{\mathbf{Z}_0,\mathbf{Z}_1}) \to \min\limits_{p_{\mathbf{X}_0,\mathbf{X}_1}}; \\ \mathbf{X}_0 \sim p^{\mathcal{S}};\;\; \mathbf{X}_1 \sim p^{\mathcal{T}}, \end{cases} \tag{191}$$

which is called the **static Schrödinger Bridge** problem and is connected to the original by performing the Brownian interpolation between the endpoints of its solution. As it was done in the Entropic OT, one can reformulate it through the conditional KL divergence:

$$\begin{cases} \mathbb{E}\,\mathrm{KL}\left(p_{\mathbf{X}_1|\mathbf{X}_0}(\cdot|\mathbf{X}_0) \,\|\, p_{\mathbf{Z}_1|\mathbf{Z}_0}(\cdot|\mathbf{X}_0)\right) \to \min\limits_{p_{\mathbf{X}_0,\mathbf{X}_1}}; \\ \mathbf{X}_0 \sim p^{\mathcal{S}};\;\; \mathbf{X}_1 \sim p^{\mathcal{T}}, \end{cases} \tag{192}$$

since the first summand in the corresponding chain rule, $\mathrm{KL}(p_{\mathbf{X}_0} \,\|\, p_{\mathbf{Z}_0})$, is zero by assumption that the reference process starts at the source distribution.

Finally, the conditional distribution $p_{\mathbf{Z}_1|\mathbf{Z}_0}(\boldsymbol{x}_1|\boldsymbol{x}_0)$ has a tractable form, since it is the transition distribution of the Wiener process $\mathrm{d}\mathbf{Z}_t = \sqrt{\gamma}\mathrm{d}\mathbf{W}_t$, and is equal to

$$p_{\mathbf{Z}_1|\mathbf{Z}_0}(\boldsymbol{x}_1|\boldsymbol{x}_0) = \mathcal{N}(\boldsymbol{x}_1|\boldsymbol{x}_0, \gamma I). \tag{193}$$

Thus, optimizing the conditional KL divergence is equivalent to optimizing

$$\mathbb{E}\,\log p_{\mathbf{X}_1|\mathbf{X}_0}(\mathbf{X}_1|\mathbf{X}_0) - \mathbb{E}\,\log p_{\mathbf{Z}_1|\mathbf{Z}_0}(\mathbf{X}_1|\mathbf{X}_0) \tag{194}$$

$$= -\mathbb{E}\,\mathcal{H}\left(p_{\mathbf{X}_1|\mathbf{X}_0}(\cdot|\mathbf{X}_0)\right) - \mathbb{E}\,\log p_{\mathbf{Z}_1|\mathbf{Z}_0}(\mathbf{X}_1|\mathbf{X}_0) = \tag{195}$$

$$= -\mathbb{E}\,\mathcal{H}\left(p_{\mathbf{X}_1|\mathbf{X}_0}(\cdot|\mathbf{X}_0)\right) - \mathbb{E}\,\log\left(\text{const}\cdot\exp\left(-\frac{1}{2\gamma}\|\mathbf{X}_1 - \mathbf{X}_0\|^2\right)\right) = \tag{196}$$

$$= -\mathbb{E}\,\mathcal{H}\left(p_{\mathbf{X}_1|\mathbf{X}_0}(\cdot|\mathbf{X}_0)\right) + \frac{1}{2\gamma}\mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2 + \text{const}. \tag{197}$$

Thus, the static SB problem for the Wiener reference process is equivalent to

$$\begin{cases} \frac{1}{2\gamma}\mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2 - \mathbb{E}\,\mathcal{H}(p_{\mathbf{X}_1|\mathbf{X}_0}(\cdot|\mathbf{X}_0)) \to \min\limits_{p_{\mathbf{X}_0,\mathbf{X}_1}}; \\ \mathbf{X}_0 \sim p^{\mathcal{S}};\;\; \mathbf{X}_1 \sim p^{\mathcal{T}}, \end{cases} \tag{198}$$

or, multiplying by 2γ,

$$\begin{cases} \mathbb{E}\|\mathbf{X}_1 - \mathbf{X}_0\|^2 - 2\gamma\cdot\mathbb{E}\,\mathcal{H}(p_{\mathbf{X}_1|\mathbf{X}_0}(\cdot|\mathbf{X}_0)) \to \min\limits_{p_{\mathbf{X}_0,\mathbf{X}_1}}; \\ \mathbf{X}_0 \sim p^{\mathcal{S}};\;\; \mathbf{X}_1 \sim p^{\mathcal{T}}, \end{cases} \tag{199}$$

which is the Entropic OT problem with the regularization coefficient 2γ, which we will denote as EOT(2γ).

This observation finally sheds light on how is the general SB problem with KL minimization connected to minimizing the cost of transportation. The answer is: if the reference process is Wiener with coefficient $\sqrt{\gamma}$, then the «stochastic mapping», defined by the Schrödinger Bridge, solves the entropic optimal transport with regularization coefficient 2γ and the quadratic cost $c(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|^2$. Interestingly, the regularization coefficient grows with the scale of the Wiener process, which coincides with minimizing the discrepancy between the process and the reference. Ideologically, our result states that transporting $p^{\mathcal{S}}$ into $p^{\mathcal{T}}$ in the most «Wiener-like» means performing the transportation while minimizing the expected L_2 distance and accounting for the diversity of the resulting distribution. Formally, we have provided the first complete characteristic of the Schrödinger Bridge and proved the following

Theorem 7. *If the reference process* $\mathbf{Z}$ *is given by the scaled Wiener process* $\mathrm{d}\mathbf{Z}_t = \sqrt{\gamma}\mathrm{d}\mathbf{W}_t$*, then the solution to the Schrödinger Bridge problem is the random process* $\mathbf{X}^{\mathrm{SB}}$*, which has two properties:*

1. *Its joint distribution* $p_{\mathbf{X}_0^{\mathrm{SB}},\mathbf{X}_1^{\mathrm{SB}}}$ *on the endpoints solves the Entropic OT problem* $\mathrm{EOT}(2\gamma)$ *between* $p^{\mathcal{S}}$ *and* $p^{\mathcal{T}}$*;*

2. *Its bridge distribution* $\mathsf{P}_{\mathbf{X}^{\mathrm{SB}}|\mathbf{X}_0^{\mathrm{SB}},\mathbf{X}_1^{\mathrm{SB}}}(\cdot|\boldsymbol{x}_0,\boldsymbol{x}_1)$ *is equal to the Brownian bridge distribution* $\mathrm{BB}(\boldsymbol{x}_0,\boldsymbol{x}_1,\sqrt{\gamma})$.

Technical supplementary

In the supplementary part of the manuscript we review all the technical background important for and used in the lectures.

A. Probability theory.

A.1. Probability density and expected value

Definition. Given a random vector $\mathbf{X} \in \mathbb{R}^d$ and a vector $\boldsymbol{x} \in \mathbb{R}^d$ the function

$$F_{\mathbf{X}}(\boldsymbol{x}) = \mathsf{P}\left(\mathbf{X}_1 \leq \boldsymbol{x}_1, \ldots, \mathbf{X}_d \leq \boldsymbol{x}_d\right) \tag{200}$$

is called the *cumulative distribution function (cdf)* of $\mathbf{X}$'s probability distribution.

Remark. Cumulative distribution function of random variable is one of the essential objects that uniquely define its distribution (see Lebesgue/Carathéodory theorem of uniqueness).

Definition. A random vector $\mathbf{X} \in \mathbb{R}^d$ is said to be absolutely continuous with *probability density function (pdf)* $p_{\mathbf{X}}(\boldsymbol{x})$, if $p_{\mathbf{X}}(\boldsymbol{x})$ is a non-negative integrable function such that $\mathbf{X}$'s cdf can be represented as

$$F_{\mathbf{X}}(\boldsymbol{x}) = \int_{-\infty}^{\boldsymbol{x}_1} \ldots \int_{-\infty}^{\boldsymbol{x}_d} p_{\mathbf{X}}(\boldsymbol{z}_1, \ldots, \boldsymbol{z}_d) \mathrm{d}\boldsymbol{z}_1 \ldots \mathrm{d}\boldsymbol{z}_d. \tag{201}$$

Probability density $p_{\mathbf{X}}(\boldsymbol{x})$ of the random vector $\mathbf{X}$ is also called the *joint* density of the random variables $\mathbf{X}_1, \ldots, \mathbf{X}_d$. In this context, densities of random variables $\mathbf{X}_1, \ldots, \mathbf{X}_d$ are called *marginal* densities.

Lemma. *Probability density function can be expressed through the cumulative distribution function as*

$$p_{\mathbf{X}}(\boldsymbol{x}) = \frac{\partial^d}{\partial \boldsymbol{x}_1 \ldots \partial \boldsymbol{x}_d} F_{\mathbf{X}}(\boldsymbol{x}). \tag{202}$$

In particular, one can express density as the limit

$$p_{\mathbf{X}}(\boldsymbol{x}) = \lim_{h \to 0} \frac{\mathsf{P}(\mathbf{X} \in [\boldsymbol{x}_1 - h, \boldsymbol{x}_1 + h] \times \ldots \times [\boldsymbol{x}_d - h, \boldsymbol{x}_d + h])}{(2h)^d}, \tag{203}$$

which means that for small cubes density is approximately equal to the fraction of probability «mass» and the corresponding volume, hence the name. Moreover, the formula holds for a wider case of «good» set sequences: if $\operatorname{vol}(B_n) \to 0$ *then*

$$p_{\mathbf{X}}(\boldsymbol{x}) = \lim_{n \to 0} \frac{\mathsf{P}(\mathbf{X} \in B_n)}{\operatorname{vol}(B_n)}. \tag{204}$$

Lemma. *If* $\mathbf{X} \in \mathbb{R}^d$ *is an absolutely continuous vector, then for all subsets of indices* $I \subset \{1, \ldots, d\}$ *the corresponding subvectors are also absolutely continuous with densities*

$$p_{\mathbf{X}_I}(\boldsymbol{x}_I) = \int_{\mathbb{R}^{d-|I|}} p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}_J, \tag{205}$$

where $J = \{1, \ldots, d\}/I$ *is the complement set of indices.*

Definition. Random variables $\mathbf{X}_1, \ldots, \mathbf{X}_d$ are called *independent* if for all (Borel measurable) sets $B_1, \ldots, B_d$ holds

$$\mathsf{P}(\mathbf{X}_1 \in B_1, \ldots, \mathbf{X}_d \in B_d) = \prod_{i=1}^{d} \mathsf{P}(\mathbf{X}_i \in B_i). \tag{206}$$

Lemma. *Random variables* $\mathbf{X}_1, \ldots, \mathbf{X}_d$ *are* independent *if and only if their joint cdf is equal to the product of marginal cdfs*

$$F_{\mathbf{X}_1, \ldots, \mathbf{X}_d}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_d) = \prod_{i=1}^{d} F_{\mathbf{X}_i}(\boldsymbol{x}_i). \tag{207}$$

If the random vector $\mathbf{X} = (\mathbf{X}_1, \ldots, \mathbf{X}_d)$ *is absolutely continuous, then* $\mathbf{X}_1, \ldots, \mathbf{X}_d$ *are* independent *if and only if their joint pdf is equal to the product of marginal pdfs*

$$p_{\mathbf{X}_1, \ldots, \mathbf{X}_d}(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_d) = \prod_{i=1}^{d} p_{\mathbf{X}_i}(\boldsymbol{x}_i). \tag{208}$$

Definition. The expected value of an absolutely continuous random variable $\mathbf{X}$ with density $p_{\mathbf{X}}(\boldsymbol{x})$ is defined as

$$\mathbb{E}\,\mathbf{X} = \begin{cases} \int_{\mathbb{R}} \boldsymbol{x}\, p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}, & \text{if } \int_{-\infty}^{0} \boldsymbol{x} p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} > -\infty \text{ and } \int_{0}^{+\infty} \boldsymbol{x} p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} < +\infty \\ +\infty, & \text{if } \int_{-\infty}^{0} \boldsymbol{x} p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} > -\infty \text{ and } \int_{0}^{+\infty} \boldsymbol{x} p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = +\infty \\ -\infty, & \text{if } \int_{-\infty}^{0} \boldsymbol{x} p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = -\infty \text{ and } \int_{0}^{+\infty} \boldsymbol{x} p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} < +\infty \\ \text{undefined}, & \text{if } \int_{-\infty}^{0} \boldsymbol{x} p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = -\infty \text{ and } \int_{0}^{+\infty} \boldsymbol{x} p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x} = +\infty. \end{cases} \tag{209}$$

Definition. The expected value of a random vector $\mathbf{X}$ is defined as the vector of expected values of its components

$$\mathbb{E}\,\mathbf{X} = \begin{pmatrix} \mathbb{E}\,\mathbf{X}_1 \\ \vdots \\ \mathbb{E}\,\mathbf{X}_d \end{pmatrix}. \tag{210}$$

Definition. Variance of a random variable $\mathbf{X}$ is the expected squared difference between the variable and its expected value:

$$\mathrm{Var}\mathbf{X} = \mathbb{E}(\mathbf{X} - \mathbb{E}\mathbf{X})^2 = \mathbb{E}\mathbf{X}^2 - (\mathbb{E}\mathbf{X})^2. \tag{211}$$

Definition. Covariance between random variables $\mathbf{X}$ and $\mathbf{Y}$ is defined as

$$\mathrm{cov}(\mathbf{X}, \mathbf{Y}) = \mathbb{E}\left[(\mathbf{X} - \mathbb{E}\mathbf{X})(\mathbf{Y} - \mathbb{E}\mathbf{Y})\right] = \mathbb{E}\left[\mathbf{X}\mathbf{Y}\right] - \mathbb{E}\mathbf{X}\,\mathbb{E}\mathbf{Y}. \tag{212}$$

Definition. In the multivariate case, variance of a random vector $\mathbf{X}$ is the matrix that consists of the covariances between its components

$$\mathrm{Var}\mathbf{X} = (\mathrm{cov}(\mathbf{X}_i, \mathbf{X}_j))_{i,j=1}^{d}. \tag{213}$$

In the matrix form

$$\mathrm{Var}\mathbf{X} = \mathbb{E}\left[(\mathbf{X} - \mathbb{E}\mathbf{X})(\mathbf{X} - \mathbb{E}\mathbf{X})^\top\right]. \tag{214}$$

Theorem 8 (Change of variables for Lebesgue integral)**.** *If $\mathbf{X}$ is absolutely continuous with density $p_{\mathbf{X}}(\boldsymbol{x})$, then for every (Borel measureable) test function φ holds*

$$\mathbb{E}\,\varphi(\mathbf{X}) = \int_{\mathbb{R}^d} \varphi(\boldsymbol{x}) p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}. \tag{215}$$

In particular, for every (Borel measurable) set B holds

$$\mathsf{P}(\mathbf{X} \in B) = \mathbb{E}\, I\{\mathbf{X} \in B\} = \int_B p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}. \tag{216}$$

Corollary. Density $p_{\mathbf{X}}(\boldsymbol{x})$ of an absolutely continuous random variable $\mathbf{X}$ can be equivalently defined as a unique non-negative function that for all (Borel measurable) sets B holds

$$\mathsf{P}(\mathbf{X} \in B) = \int_B p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}. \tag{217}$$

This equivalent definition of probability density may seem harder to verify compared to the original one with cdf, since it considers all Borel sets instead of just semi-intervals $(-\infty, \boldsymbol{x}_1) \times \ldots \times (-\infty, \boldsymbol{x}_d)$. However, it is sometimes more intuitive, since many proofs related to finding the density do not explicitly rely on the semi-interval nature of the test sets.

Theorem 9 (Differentiable change of variables)**.** *If* $\mathbf{X}$ *is an absolutely continuous random vector and* φ *is a diffeomophism (bijective differentiable with* φ^{-1} *also differentiable), then* $\varphi(\mathbf{X})$ *is also absolutely continous with density*

$$p_{\varphi(\mathbf{X})}(\boldsymbol{y}) = p_{\mathbf{X}}(\varphi^{-1}(\boldsymbol{y}))\left|\det\frac{\partial\varphi^{-1}}{\partial\boldsymbol{y}}\right|. \tag{218}$$

A convenient way to remember the formula may be the following. Remember that density can be approximated as the probability of a small cube $[\boldsymbol{y}-\mathbf{1}h, \boldsymbol{y}+\mathbf{1}h] = [\boldsymbol{y}_1 - h, \boldsymbol{y}_1 + h] \times \ldots \times [\boldsymbol{y}_d - h, \boldsymbol{y}_d + h]$ centered in $\boldsymbol{y}$ divided by its volume $(2h)^d$. Then probability of $\varphi(\mathbf{X})$ being in the cube is equal to the probability of $\mathbf{X}$ being in its preimage $\varphi^{-1}([\boldsymbol{y} - \mathbf{1}h, \boldsymbol{y}+\mathbf{1}h])$. The only remaining modification consists in taking into account the change of volume:

$$p_{\varphi(\mathbf{X})}(\boldsymbol{y}) \approx \frac{\mathsf{P}(\varphi(\mathbf{X}) \in [\boldsymbol{y}-\mathbf{1}h, \boldsymbol{y}+\mathbf{1}h])}{(2h)^d} = \frac{\mathsf{P}(\mathbf{X} \in \varphi^{-1}([\boldsymbol{y}-\mathbf{1}h, \boldsymbol{y}+\mathbf{1}h]))}{(2h)^d} \approx \tag{219}$$

$$\approx p_{\mathbf{X}}(\varphi^{-1}(\boldsymbol{y})) \cdot \frac{\mathrm{vol}(\varphi^{-1}([\boldsymbol{y}-\mathbf{1}h, \boldsymbol{y}+\mathbf{1}h]))}{(2h)^d}. \tag{220}$$

Since φ^{-1} is a differentiable function, it is locally linear thus the change of volume can be approximated by multiplying on the corresponding determinant (of the Jacobi matrix):

$$p_{\mathbf{X}}(\varphi^{-1}(\boldsymbol{y})) \cdot \frac{\mathrm{vol}(\varphi^{-1}([\boldsymbol{y}-\mathbf{1}h, \boldsymbol{y}+\mathbf{1}h]))}{(2h)^d} \approx p_{\mathbf{X}}(\varphi^{-1}(\boldsymbol{y}))\frac{(2h)^d\left|\det\frac{\partial\varphi^{-1}}{\partial\boldsymbol{y}}\right|}{(2h)^d} \approx \tag{221}$$

$$\approx p_{\mathbf{X}}(\varphi^{-1}(\boldsymbol{y}))\left|\det\frac{\partial\varphi^{-1}}{\partial\boldsymbol{y}}\right|. \tag{222}$$

A.2. Conditional expectation

Definition. Conditional expectation $\mathbb{E}[\mathbf{Y}|\mathbf{X}]$ of the random vector $\mathbf{Y}$ given the random vector $\mathbf{X}$ is the unique random vector that is the (Borel measurable) function $g(\mathbf{X})$ of the condition that satisfies

$$\mathbb{E}\left[\mathbf{Y} I\{\mathbf{X} \in B\}\right] = \mathbb{E}\left[\mathbb{E}[\mathbf{Y}|\mathbf{X}] I\{\mathbf{X} \in B\}\right] \tag{223}$$

for all (Borel measurable) sets B.

We do not use this definition of the conditional expectation throughout the lectures, but give it for the sake of completeness. Despite being hard and unobvious from the first

glance, this set of integral equalities essentially pushes an intuitive property: $g(\mathbf{X})$ should be a good approximation of $\mathbf{Y}$. To see this, one may consider the following fact: if

$$\mathbb{E}\left[\xi I_A\right] = \mathbb{E}\left[\eta I_A\right] \tag{224}$$

holds for all measurable sets A, then $\xi = \eta$. In other words, if the integrals of two functions over all test sets coincide, then the functions coincide. In case of the conditional expectation, one restricts the test sets to have the form $A = \{\mathbf{X} \in B\}$, which means that $g(\mathbf{X}) = \mathbb{E}\left[\mathbf{Y}|\mathbf{X}\right]$ should approximate $\mathbf{Y}$ well on the sets connected to $\mathbf{X}$ (which makes sense, since we want to define a function of $\mathbf{X}$ that is close to $\mathbf{Y}$ instead of being equal to $\mathbf{Y}$, which may be impossible in general).

Another useful interpretation comes from observing that the integral equalities may be rewritten in the following way:

$$\mathbb{E}\left[\mathbf{Y}\varphi(\mathbf{X})\right] = \mathbb{E}\left[\mathbb{E}[\mathbf{Y}|\mathbf{X}]\varphi(\mathbf{X})\right] \tag{225}$$

for all bounded measurable test functions φ (it holds for indicators $\implies$ for their linear combinations $\implies$ for all bounded functions, since they can be approximated by indicators). Then the difference between left- and right-hand side expressions should be equal to zero:

$$\mathbb{E}\left[(\mathbf{Y} - \mathbb{E}[\mathbf{Y}|\mathbf{X}])\varphi(\mathbf{X})\right] = 0 \tag{226}$$

for all bounded test functions φ. Compare this with the property of the projection $\mathrm{proj}_V(\boldsymbol{y})$ of a vector $\boldsymbol{y}$ on a linear subspace V:

$$\langle \boldsymbol{y} - \mathrm{proj}_V(\boldsymbol{y}), \mathbf{v}\rangle = 0 \tag{227}$$

for all vectors $\mathbf{v} \in V$. Continuing this analogy to the space of random variables with the scalar product $\langle \xi, \eta\rangle \coloneqq \mathbb{E}\,\xi\eta$, one can interpret this set of integral identities as a property that defines $\mathbb{E}[\mathbf{Y}|\mathbf{X}]$ as a projection of the variable $\mathbf{Y}$ on the subspace of random variables which are the functions of $\mathbf{X}$. We will further formally prove that this is indeed the case when $\mathbf{Y}$ is square-integrable. For now, this is yest another evidence that $\mathbb{E}[\mathbf{Y}|\mathbf{X}]$ is such a function of $\mathbf{X}$ that approximates $\mathbf{Y}$ well in some sense.

Definition. Conditional expectation $\mathbb{E}[\mathbf{Y}|\mathbf{X} = \boldsymbol{x}]$ of the random vector $\mathbf{Y}$ given $\mathbf{X} = \boldsymbol{x}$ is the unique (Borel measurable) function $g(\boldsymbol{x})$ that satisfies

$$\mathbb{E}\left[\mathbf{Y} I\{\mathbf{X} \in B\}\right] = \mathbb{E}\left[g(\mathbf{X}) I\{\mathbf{X} \in B\}\right] \tag{228}$$

for all (Borel measurable) sets B. In other words, $\mathbb{E}[\mathbf{Y}|\mathbf{X} = \boldsymbol{x}]$ is the function $g(\boldsymbol{x})$ such that $\mathbb{E}[\mathbf{Y}|\mathbf{X}] = g(\mathbf{X})$.

Given our previous observations that $\mathbb{E}[\mathbf{Y}|\mathbf{X}]$ approximates $\mathbf{Y}$ with the function of $\mathbf{X}$, one can interpret calculation of the conditional expectation $\mathbb{E}[\mathbf{Y}|\mathbf{X}]$ as making the *prediction*

of the value $\mathbf{Y}$ while knowing the value $\mathbf{X}$. Prediction is essentially the function that maps the observed value $\mathbf{X} = \boldsymbol{x}$ into the approximation $g(\boldsymbol{x})$. This way, we can distinguish the two conditional expectations by saying that $\mathbb{E}[\mathbf{Y}|\mathbf{X} = \boldsymbol{x}]$ is the function that performs approximation, while $\mathbb{E}[\mathbf{Y}|\mathbf{X}]$ is the result of applying this function to the realization of the random variable $\mathbf{X}$.

Definition. If a random vector $(\mathbf{X}, \mathbf{Y})$, where $\mathbf{X} \in \mathbb{R}^{d_1}$ and $\mathbf{Y} \in \mathbb{R}^{d_2}$, has the joint density $p_{\mathbf{X},\mathbf{Y}}(\boldsymbol{x}, \boldsymbol{y})$, then the value

$$p_{\mathbf{Y}|\mathbf{X}}(\boldsymbol{y}|\boldsymbol{x}) = \begin{cases} \frac{p_{\mathbf{X},\mathbf{Y}}(\boldsymbol{x},\boldsymbol{y})}{p_{\mathbf{Y}}(\boldsymbol{y})}, & \text{if } p_{\mathbf{Y}}(\boldsymbol{y}) > 0; \\ 0, & \text{else} \end{cases} \tag{229}$$

is called the conditional density of $\mathbf{Y}$ given $\mathbf{X}$.

The conditional density acts exactly as the conditional probability, but in the continuous case: if one wants to determine the probability of $\mathbf{Y}$ hitting a set B given the known value $\mathbf{X} = \boldsymbol{x}$, one should calculate

$$\mathsf{P}(\mathbf{Y} \in B|\mathbf{X} = \boldsymbol{x}) = \int_B p_{\mathbf{Y}|\mathbf{X}}(\boldsymbol{y}|\boldsymbol{x})\mathrm{d}\boldsymbol{y}. \tag{230}$$

One could informally infer such a formula using the same «physical» approximation of density with the quotient of probability and volume:

$$\mathsf{P}(\mathbf{Y} \in [\boldsymbol{y} - \mathbf{1}h, \boldsymbol{y} + \mathbf{1}h] \mid \mathbf{X} \in [\boldsymbol{x} - \mathbf{1}h, \boldsymbol{x} + \mathbf{1}h]) = \tag{231}$$

$$= \frac{\mathsf{P}(\mathbf{Y} \in [\boldsymbol{y} - \mathbf{1}h, \boldsymbol{y} + \mathbf{1}h], \mathbf{X} \in [\boldsymbol{x} - \mathbf{1}h, \boldsymbol{x} + \mathbf{1}h])}{\mathsf{P}(\mathbf{X} \in [\boldsymbol{x} - \mathbf{1}h, \boldsymbol{x} + \mathbf{1}h])} \approx \tag{232}$$

$$\approx \frac{p_{\mathbf{X},\mathbf{Y}}(\boldsymbol{x}, \boldsymbol{y})(2h)^{d_1+d_2}}{p_{\mathbf{X}}(\boldsymbol{x})(2h)^{d_1}} = \frac{p_{\mathbf{X},\mathbf{Y}}(\boldsymbol{x}, \boldsymbol{y})}{p_{\mathbf{X}}(\boldsymbol{x})} \cdot \mathrm{vol}([\boldsymbol{y} - \mathbf{1}h, \boldsymbol{y} + \mathbf{1}h]), \tag{233}$$

which means that if we divide by the cube volume and take the limit, we will obtain exactly the conditional density, which is the quotient of joint and marginal densities.

Lemma. *If a random vector $(\mathbf{X}, \mathbf{Y})$, where $\mathbf{X} \in \mathbb{R}^{d_1}$ and $\mathbf{Y} \in \mathbb{R}^{d_2}$, has joint density $p_{\mathbf{X},\mathbf{Y}}(\boldsymbol{x}, \boldsymbol{y})$, then the conditional expectation $\mathbb{E}[\mathbf{Y}|\mathbf{X} = \boldsymbol{x}]$ can be calculated as*

$$\mathbb{E}[\mathbf{Y}|\mathbf{X} = \boldsymbol{x}] = \int_{\mathbb{R}^{d_2}} \boldsymbol{y}\, p_{\mathbf{Y}|\mathbf{X}}(\boldsymbol{y}|\boldsymbol{x})\mathrm{d}\boldsymbol{y}. \tag{234}$$

Moreover, if $\mathbf{X}$ and $\mathbf{Y}$ are independent, one could calculate the expected value of test functions of two arguments as

$$\mathbb{E}\left[\varphi(\mathbf{X}, \mathbf{Y}) \mid \mathbf{X} = \boldsymbol{x}\right] = \int_{\mathbb{R}^{d_2}} \varphi(\boldsymbol{x}, \boldsymbol{y})\, p_{\mathbf{Y}|\mathbf{X}}(\boldsymbol{y}|\boldsymbol{x})\mathrm{d}\boldsymbol{y}. \tag{235}$$

Lemma. *Conditional expectation* $\mathbb{E}[\mathbf{Y}|\mathbf{X}]$ *satisfies the following properties:*

1. $\mathbb{E}[a\mathbf{Z} + b\mathbf{Y}|\mathbf{X}] = a\,\mathbb{E}[\mathbf{Z}|\mathbf{X}] + b\,\mathbb{E}[\mathbf{Y}|\mathbf{X}]$ *(linearity);*
2. $\mathbb{E}[\varphi(\mathbf{X})|\mathbf{X}] = \varphi(\mathbf{X})$*;*
3. *If* $\mathbf{X}$ *and* $\mathbf{Y}$ *are independent, then* $\mathbb{E}[\mathbf{Y}|\mathbf{X}] = \mathbb{E}\mathbf{Y}$*;*
4. $\mathbb{E}\,\mathbb{E}[\mathbf{Y}|\mathbf{X}] = \mathbb{E}\mathbf{Y}$ *(law of total expectation);*
5. $\mathbb{E}[\varphi(\mathbf{X})\mathbf{Y}|\mathbf{X}] = \varphi(\mathbf{X})\mathbb{E}[\mathbf{Y}|\mathbf{X}]$*;*
6. *For all convex* φ *holds* $\mathbb{E}[\varphi(\mathbf{Y})|\mathbf{X}] \leq \varphi(\mathbb{E}[\mathbf{Y}|\mathbf{X}])$ *(Jensen's inequality);*

Most of these properties are easy to prove using the definition, or using the integral formula for the absolutely continuous case, which we leave as an exercise. Some of them, however, are important from the ideological standpoint. The second property says that if one wants to predict the function $\varphi(\mathbf{X})$ of $\mathbf{X}$ given the value of $\mathbf{X}$, he should output $\varphi(\mathbf{X})$, since the prediction is essentially known. The third property tells that if one tries to predict $\mathbf{Y}$ given the knowledge about an independent variable $\mathbf{X}$, the best he can do is ignore this variable and output the constant prediction equal to the expected value. The fourth property tells us that taking the conditional expectation essentially consists of «integrating out part of the noise». Integrating the rest gives the same result as integrating «all of the noise» simultaneously.

Theorem 10. *If* $\mathbb{E}\|\mathbf{Y}\|^2 < \infty$*, then the conditional expectation* $g^*(\boldsymbol{x}) = \mathbb{E}[\mathbf{Y}|\mathbf{X} = \boldsymbol{x}]$ *is the best* L_2 *approximation of* $\mathbf{Y}$ *among the functions of* $\mathbf{X}$*:*

$$g^* = \arg\min_{g:\mathbb{E}\|g(\mathbf{X})\|^2<\infty} \mathbb{E}\|g(\mathbf{X}) - \mathbf{Y}\|^2. \tag{236}$$

In particular, if one considers a parameteric family g_θ *and aims at learning the conditional expectation by minimizing the functional*

$$\mathbb{E}\|g_\theta(\mathbf{X}) - \mathbb{E}[\mathbf{Y}|\mathbf{X}]\|^2 \to \min_\theta, \tag{237}$$

it can be done by performing the L_2 *regression*

$$\mathbb{E}\|g_\theta(\mathbf{X}) - \mathbf{Y}\|^2 \to \min_\theta, \tag{238}$$

because the two functionals coincide up to the terms independent of θ*.*

Proof. We provide the proof of the theorem, since it is simple, illustrative, and essential for deriving most of the optimization functionals in the course. Specifically, we aim at proving the following statement:

$$\mathbb{E}\|g(\mathbf{X}) - \mathbb{E}[\mathbf{Y}|\mathbf{X}]\|^2 = \mathbb{E}\|g(\mathbf{X}) - \mathbf{Y}\|^2 + \text{const}, \tag{239}$$

where const does not depend on g. First, we simply deal with the square and represent it with the three summands

$$\mathbb{E}\|g(\mathbf{X}) - \mathbb{E}[\mathbf{Y}|\mathbf{X}]\|^2 = \mathbb{E}\|g(\mathbf{X})\|^2 - 2\,\mathbb{E}\big\langle g(\mathbf{X}), \mathbb{E}[\mathbf{Y}|\mathbf{X}]\big\rangle + \mathbb{E}\|\mathbb{E}[\mathbf{Y}|\mathbf{X}]\|^2. \tag{240}$$

The latter summand does not depend on g and can be omitted. The first appears if one opens the square $\mathbb{E}\|g(\mathbf{X}) - \mathbf{Y}\|^2$, so we keep it as it is. Thus, we only need to deal with the scalar product. We can rewrite it as the sum

$$\mathbb{E}\big\langle g(\mathbf{X}), \mathbb{E}[\mathbf{Y}|\mathbf{X}]\big\rangle = \mathbb{E}\left[\sum_{i=1}^{d} g_i(\mathbf{X}) \cdot \mathbb{E}[\mathbf{Y}_i|\mathbf{X}]\right] = \sum_{i=1}^{d} \mathbb{E}\left[g_i(\mathbf{X})\mathbb{E}[\mathbf{Y}_i|\mathbf{X}]\right] = \tag{241}$$

$$\underbrace{=}_{g_i(\mathbf{X}) \text{ is function of } \mathbf{X}} \sum_{i=1}^{d} \mathbb{E}\Big[\mathbb{E}[g_i(\mathbf{X})\mathbf{Y}_i|\mathbf{X}]\Big] \underbrace{=}_{\text{law of total expectation}} \sum_{i=1}^{d} \mathbb{E}\left[g_i(\mathbf{X})\mathbf{Y}_i\right] = \mathbb{E}\big\langle g(\mathbf{X}), \mathbf{Y}\big\rangle. \tag{242}$$

We thus obtain

$$\mathbb{E}\|g(\mathbf{X}) - \mathbb{E}[\mathbf{Y}|\mathbf{X}]\|^2 = \mathbb{E}\|g(\mathbf{X})\|^2 - 2\,\mathbb{E}\big\langle g(\mathbf{X}), \mathbf{Y}\big\rangle + \text{const}. \tag{243}$$

By adding another constant $\mathbb{E}\|\mathbf{Y}\|^2$, independent from g, and reorganizing summands into the squared difference, we obtain

$$\mathbb{E}\|g(\mathbf{X}) - \mathbb{E}[\mathbf{Y}|\mathbf{X}]\|^2 = \mathbb{E}\|g(\mathbf{X}) - \mathbf{Y}\|^2 + \text{const}. \tag{244}$$

□

First of all, this result tells the exact thing that we have announced while discussing the definition of conditional expectation: it is indeed the best (in L_2 sense) way to predict value of one random variable using the information about another random variable. Finding the conditional expectation thus, in some sense, is one of the key tasks in machine learning, where one aims to predict the target variable given the observable.

Another important observation is that the same technique is used in many classical papers when deriving the optimization functional of Score [44] and Flow [18, 43] Matching techniques. It allows one to easily infer the L_2 optimization functional instantaneously after observing that the desired function takes the form of the conditional expectation.

A.3. Multivariate Gaussian distribution

Throughout the lectures we frequently work with multivariate normal distributions. We mostly consider vectors with independent Gaussian components, however, several specific techniques for general multivariate Gaussian distributions are also useful. One of the common

Definition. Random vector $\mathbf{X} \in \mathbb{R}^d$ has multivariate normal distribution $\mathcal{N}(\mu, \Sigma)$ ($\mathbf{X}$ is a Gaussian vector) with parameters $\mu \in \mathbb{R}^d$ and $\Sigma \in \mathbb{S}^d_{++}$ if its density is equal to

$$p_{\mathbf{X}}(\boldsymbol{x}) = \frac{1}{(2\pi)^{d/2}\sqrt{\det \Sigma}} \exp\left(-\frac{1}{2}(\boldsymbol{x}-\mu)^\top \Sigma^{-1}(\boldsymbol{x}-\mu)\right). \tag{245}$$

This definition has two issues. First, it may be hard to directly work with this density, in part due to the inversion of the covariance matrix. Second, this definition does not include cases of positive semi-definite (but not positive definite) matrices $\Sigma \succeq 0$. These cases correspond to vectors, which have constant coordinates in some basis. In some cases, it may be convenient to treat constants as Gaussian variables with zero variance and therefore unite them with non-constant Gaussian variables.

A.3.1. Characteristic functions

Instead of directly defining the probability density, one could handle these issues by defining the characteristic function instead:

Definition. Characteristic function of a random vector $\mathbf{X}$ (characteristic function of $\mathbf{X}$'s distribution) is the function defined as

$$\varphi_{\mathbf{X}}(t) = \mathbb{E}\exp\left(i\langle \mathbf{X}, t\rangle\right), \tag{246}$$

where $i^2 = -1$ is the imaginary unit.

Essentially, characteristic function is the Fourier transform of the probability distribution. Indeed, if $\mathbf{X}$ has the density $p_{\mathbf{X}}(\boldsymbol{x})$, then

$$\varphi_{\mathbf{X}}(t) = \int_{\mathbb{R}^d} \exp(i\,\langle \boldsymbol{x}, t\rangle) p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}, \tag{247}$$

which coincides with the Fourier transform of the function $p_{\mathbf{X}}(\boldsymbol{x})$ up to changing t to $-t$ (and possibly dividing by $(2\pi)^d$ depending on the particular definition). This observation opens the way for simplifying work with distributions by noting that the characteristic functions uniquely determine them and using several properties such as representing convolution and differentiation with simpler multiplication operations in the Fourier space.

Theorem 11. *Characteristic function uniquely determines distribution i.e.* $\mathbf{X}$ *and* $\mathbf{Y}$ *has the same distributions if and only if* $\varphi_{\mathbf{X}}(t) = \varphi_{\mathbf{Y}}(t)$ *for all* t*.*

Theorem 12 (Inversion)**.** *If* $\int_{\mathbb{R}^d} |\varphi_{\mathbf{X}}(t)| \mathrm{d}t < \infty$ *then* $\mathbf{X}$ *has the density*

$$p_{\mathbf{X}}(\boldsymbol{x}) = \frac{1}{(2\pi)^d} \int_{\mathbb{R}^d} \exp(-i\langle \boldsymbol{x}, t\rangle)\varphi_{\mathbf{X}}(t)\mathrm{d}t. \tag{248}$$

The notable examples of manipulating with distributions include (but are not limited to) the following observations:

Lemma. *If* $\mathbf{X}$ *and* $\mathbf{Y}$ *are independent random vectors of the same dimension then* $\varphi_{\mathbf{X}+\mathbf{Y}}(t) = \varphi_{\mathbf{X}}(t)\varphi_{\mathbf{Y}}(t)$*;*

Lemma. *Random variable* $\mathbf{X}$ *has symmetric distribution if and only if* $\varphi_{\mathbf{X}}(t)$ *is real for all* t*;*

Lemma. *Random variables* $\mathbf{X}_1, \ldots, \mathbf{X}_d$ *are independent if and only if*

$$\varphi_{\mathbf{X}_1,\ldots,\mathbf{X}_d}(t_1, \ldots, t_d) = \prod_{j=1}^{d} \varphi_{\mathbf{X}_j}(t_j). \tag{249}$$

Theorem 13. *If* $\mathbf{X}$ *is a random variable and* $\mathbb{E}|\mathbf{X}|^k < \infty$*, then* $\varphi_{\mathbf{X}}$ *is* k *times differentiable with*

$$\varphi_{\mathbf{X}}^{(j)}(t) = i^j \,\mathbb{E}\left[\mathbf{X}^j e^{it\mathbf{X}}\right]. \tag{250}$$

for $j \leq k$*. In particular, one can determine moments of the distribution by evaluating derivatives of* $\varphi_{\mathbf{X}}$ *in* $t = 0$*:*

$$\varphi_{\mathbf{X}}^{(j)}(0) = i^j \,\mathbb{E}\,\mathbf{X}^j. \tag{251}$$

Using these techniques one can e.g. perform the following simplifications of calculations:

1. If the goal is to calculate density of the sum of two independent variables, one could directly calculate the convolution

   $$p_{\mathbf{X}+\mathbf{Y}}(\boldsymbol{z}) = \int_{\mathbb{R}^d} p_{\mathbf{X}}(\boldsymbol{x}) p_{\mathbf{Y}}(\boldsymbol{z} - \boldsymbol{x})\mathrm{d}\boldsymbol{x}. \tag{252}$$

   However, if the corresponding characteristic functions are known, then calculating the sum's characteristic function is trivial: one should just perform the multiplication

   $$\varphi_{\mathbf{X}+\mathbf{Y}}(t) = \varphi_{\mathbf{X}}(t)\varphi_{\mathbf{Y}}(t). \tag{253}$$

   Then, in multiple scenarios one could straightforwardly determine the resulting distribution by taking a look at the form of the characteristic function. In other cases, one may calculate the resulting density by inversion, which may be simpler in some cases;

2. Calculating moments of the distribution involves evaluating integrals of the form

$$\int_{\mathbb{R}} \boldsymbol{x}^k p_{\mathbf{X}}(\boldsymbol{x}) \mathrm{d}\boldsymbol{x}. \tag{254}$$

   In some cases, efficient schemes of calculating this type of integrals may depend on k, which results in several integral calculations. Instead, given the pre-computed characteristic function, moments are easily obtained by performing differentiation.

In case of the multivariate Gaussian distribution, the most important quality of the characteristic functions is their one-to-one correspondence with the probability distributions. However, the above examples are intended to show that they offer a fundamental and powerful tool for determining the distribution (or characteristics of the distribution) in many other scenarios.

A.3.2. Back to Gaussian vectors

The following equivalent definitions (basic in probability theory) include constants and allow for multiple tools of working with multivariate normal distribution.

Definition. Random vector $\mathbf{X} \in \mathbb{R}^d$ has multivariate normal distribution $\mathcal{N}(\mu, \Sigma)$ ($\mathbf{X}$ is a Gaussian vector) with parameters $\mu \in \mathbb{R}^d$ and $\Sigma \in \mathbb{S}_+^d$ if (the following properties are equivalent)

1. $\mathbf{X}$ can be represented as $\mathbf{X} = A\boldsymbol{\varepsilon} + b$, where $\boldsymbol{\varepsilon}$ is a vector of independent standard normal variables, A and b are some matrix and vector;
2. For each (non-random) vector $t \in \mathbb{R}^d$ random variable $\langle \mathbf{X}, t \rangle$ has normal distribution;
3. Characteristic function of $\mathbf{X}$ is equal to

$$\varphi_{\mathbf{X}}(t) = \exp\left(\langle \mu, t \rangle - \frac{1}{2} \langle \Sigma t, t \rangle \right). \tag{255}$$

As always, the parameters correspond to the mean and variance (covariance matrix) of the random vector:

$$\begin{pmatrix} \mu_1 \\ \vdots \\ \mu_d \end{pmatrix} = \begin{pmatrix} \mathbb{E}\,\mathbf{X}_1 \\ \vdots \\ \mathbb{E}\,\mathbf{X}_d \end{pmatrix} = \mathbb{E}\,\mathbf{X}; \quad \Sigma = \mathrm{Var}\mathbf{X} = (\mathrm{cov}(\mathbf{X}_i, \mathbf{X}_j))_{i,j=1}^d. \tag{256}$$

The first two definitions are usually convenient for verification that some vector is Gaussian. Representation of the characteristic function is extremely useful for limit theorems (as e.g the Central Limit Theorem) or for finding the conditional distributions of subvectors of a Gaussian vector.

The next set of claims and lemmas are the base for the convenient work with Gaussian vectors.

Lemma. *Let* $\mathbf{X} \sim \mathcal{N}(\mu, \Sigma)$ *be a Gaussian random vector. Then its affine transform* $\mathbf{Y} = A\mathbf{X} + b$ *is also Gaussian with parameters* $\mathbf{Y} \sim \mathcal{N}(A\mu + b, A\Sigma A^\top)$. *In particular, if* $\mathbf{X} \sim \mathcal{N}(0, \sigma^2 I)$ *and* $\mathbf{Y} = C\mathbf{X}$, *where* $CC^\top = I$ *is orthogonal, then* $\mathbf{Y} \sim \mathcal{N}(0, \sigma^2 I)$.

Determining parameters of the updated Gaussian distribution has nothing to do with the normal distribution. Instead, it is based on simple observations about the transformation of mean and variance under an affine transformation. The ideological part consists in verifying that affine transforms preserve normal distribution, which can be shown using any of the equivalent definitions. The last (but not the least) observation tells that a centered normal distribution with scalar covariance matrix is invariant under orthogonal transforms (e.g. rotations) which coincides with such basic traits of this distribution as e.g. circular level sets.

Lemma. *Components* $\mathbf{X}_i, \mathbf{X}_j$ *of a Gaussian vector are independent if and only if they are uncorrelated, i.e.* $\operatorname{cov}(\mathbf{X}_i, \mathbf{X}_j) = 0$.

This is an extremely useful property, which distinguishes Gaussian vectors from arbitrary vectors with Gaussian components. In general, testing independence of two random variables is a nontrivial task due to the necessity of (almost) fully characterizing the distribution e.g. finding the joint cdf/pdf or characteristic function. In the Gaussian case, one could instead calculate the covariance of the components of interest. One of the most useful application of this observation is the orthogonal decomposition of Gaussian vectors.

Lemma. *Let* $(\mathbf{X}, \mathbf{Y}) \in \mathbb{R}^2$ *be a Gaussian* $\mathcal{N}(\mu, \Sigma)$ *vector. Then,* $\mathbf{Y} - c\mathbf{X}$ *is independent with* $\mathbf{X}$ *if*

$$c = \frac{\operatorname{cov}(\mathbf{X}, \mathbf{Y})}{\operatorname{Var}\mathbf{X}}. \tag{257}$$

This lemma is a straightforward application of the independence criterion above applied to the Gaussian vector $(\mathbf{X}, \mathbf{Y} - c\mathbf{X})$: one should simply verify that the covariance of the components is zero. The result is important to examine from another angle: in Gaussian case one could perform the decomposition

$$\mathbf{Y} = \frac{\operatorname{cov}(\mathbf{X}, \mathbf{Y})}{\operatorname{Var}\mathbf{X}}\mathbf{X} + \left(\mathbf{Y} - \frac{\operatorname{cov}(\mathbf{X}, \mathbf{Y})}{\operatorname{Var}\mathbf{X}}\mathbf{X}\right), \tag{258}$$

where the first summand is the function of $\mathbf{X}$ and the second is independent from $\mathbf{X}$. Compare this with the orthogonal decomposition of the $\mathbb{R}^d$ vector $\boldsymbol{y}$ with respect to the 1-dimensional linear subspace generated by the vector $\boldsymbol{x}$:

$$\boldsymbol{y} = \frac{\langle \boldsymbol{x}, \boldsymbol{y} \rangle}{\langle \boldsymbol{x}, \boldsymbol{x} \rangle} \boldsymbol{x} + \left(\boldsymbol{y} - \frac{\langle \boldsymbol{x}, \boldsymbol{y} \rangle}{\langle \boldsymbol{x}, \boldsymbol{x} \rangle} \boldsymbol{x} \right). \tag{259}$$

The Gaussian decomposition fully replicates this formula with the standard scalar product replaced by covariance. In fact, this decomposition obviously holds for all random variables, but in the Gaussian case it is exceptionally useful since the two components are not just uncorrelated, but independent.

Another useful property (especially in case of diffusion models) of the multivariate normal distribution is that the conditional distribution of a subset of its components given the rest of the components is also Gaussian.

Lemma. *Let $(\mathbf{X}, \mathbf{Y}) \in \mathbb{R}^2$ be the Gaussian $\mathcal{N}(\mu, \Sigma)$ vector. Then, the conditional distribution $p_{\mathbf{Y}|\mathbf{X}}$ is also Gaussian:*

$$p_{\mathbf{Y}|\mathbf{X}}(\boldsymbol{y}|\boldsymbol{x}) = \mathcal{N}\left(\frac{\operatorname{cov}(\mathbf{X}, \mathbf{Y})}{\operatorname{Var}\mathbf{X}} (\boldsymbol{x} - \mathbb{E}\mathbf{X}) + \mathbb{E}\mathbf{Y}, \operatorname{Var}\mathbf{Y} - \frac{\operatorname{cov}(\mathbf{X}, \mathbf{Y})^2}{\operatorname{Var}\mathbf{X}} \right). \tag{260}$$

In particular,

$$\mathbb{E}[\mathbf{Y}|\mathbf{X}] = \frac{\operatorname{cov}(\mathbf{X}, \mathbf{Y})}{\operatorname{Var}\mathbf{X}} (\mathbf{X} - \mathbb{E}\mathbf{X}) + \mathbb{E}\mathbf{Y}. \tag{261}$$

Proof. Here, we provide the proof, since it acts as a great illustration for working with Gaussian vectors, characteristic functions and conditional expectation simultaneously. First, observe that one could calculate the conditional density straightforwardly using the definition:

$$p_{\mathbf{Y}|\mathbf{X}}(\boldsymbol{y}|\boldsymbol{x}) = \frac{p_{\mathbf{X},\mathbf{Y}}(\boldsymbol{x}, \boldsymbol{y})}{p_{\mathbf{X}}(\boldsymbol{x})} = \text{const} \cdot \frac{\exp(-\text{quadratic function}(\boldsymbol{x}, \boldsymbol{y}))}{\exp(-\text{quadratic function}(\boldsymbol{x}))}, \tag{262}$$

which gives the exponent of the quadratic function that means $p_{\mathbf{Y}|\mathbf{X}}(\boldsymbol{y}|\boldsymbol{x})$ is also Gaussian. One may then calculate the corresponding coefficients and represent the quadratic form as in the definition of Gaussian density, which will fully specify the parameters. This way has a lot of technical (even arithmetic) derivations without many ideas behind them. At the same time, one could use the powerful probabilistic tools of working with Gaussian vectors and derive the same formula much easier.

Let us instead calculate the characteristic function of the conditional distribution. If we manage to represent it as the characteristic function of a normal distribution with some parameters, then using the one-to-one correspondence bethween the c.f. and the

probability distribution, we will deduce that the conditional distribution is indeed normal. Our goal then is to calculate

$$\varphi_{\mathbf{Y}|\mathbf{X}=x}(t) = \mathbb{E}\left[\exp(it\mathbf{Y}) \mid \mathbf{X} = \boldsymbol{x}\right]. \tag{263}$$

The go-to method for calculating the conditional expectations of such form is to split the value into two components: function of the condition and some variable independent of the condition. For this, we have the orthogonal decomposition $\mathbf{Y} = c\mathbf{X} + \mathbf{Y} - c\mathbf{X}$, where $c = \mathrm{cov}(\mathbf{X}, \mathbf{Y})/\mathrm{Var}\mathbf{X}$. We then obtain

$$\mathbb{E}[\exp(it\mathbf{Y}) \mid \mathbf{X} = \boldsymbol{x}] = \mathbb{E}[\exp\left(it\left(c\mathbf{X} + \mathbf{Y} - c\mathbf{X}\right)\right) \mid \mathbf{X} = \boldsymbol{x}]. \tag{264}$$

Then we move function of the condition outside of the conditional expectation and obtain

$$\exp(it \cdot c\boldsymbol{x}) \cdot \mathbb{E}[\exp(it(\mathbf{Y} - c\mathbf{X})) \mid \mathbf{X}] = \exp(it \cdot c\boldsymbol{x}) \cdot \mathbb{E}[\exp(it(\mathbf{Y} - c\mathbf{X}))], \tag{265}$$

where we eliminate the condition due to independence of $\mathbf{Y} - c\mathbf{X}$ with $\mathbf{X}$. The first multiplier simply corresponds to the constant bias $c\boldsymbol{x}$ of the distribution (one can prove that $\varphi_{\mathbf{X}+a}(t) = e^{ita}\varphi_{\mathbf{X}}(t)$). The second multiplier is the characteristic function of the variable $\mathbf{Y} - c\mathbf{X}$, which is Gaussian as a linear combination of components $\mathbf{X}, \mathbf{Y}$ of a Gaussian vector. Its mean is equal to $\mathbb{E}\mathbf{Y} - c\mathbb{E}\mathbf{X}$ and variance can be calculated as

$$\begin{aligned}\mathrm{Var}\mathbf{Y} + c^2\mathrm{Var}\mathbf{X} - 2c \cdot \mathrm{cov}(\mathbf{X}, \mathbf{Y}) &= \mathrm{Var}\mathbf{Y} + \frac{\mathrm{cov}(\mathbf{X}, \mathbf{Y})^2}{\mathrm{Var}\mathbf{X}} - 2\frac{\mathrm{cov}(\mathbf{X}, \mathbf{Y})^2}{\mathrm{Var}\mathbf{X}} = \\ &= \mathrm{Var}\mathbf{Y} - \frac{\mathrm{cov}(\mathbf{X}, \mathbf{Y})^2}{\mathrm{Var}\mathbf{X}}.\end{aligned}$$

Thus, the resulting value is equal to

$$\exp(it \cdot c\boldsymbol{x}) \cdot \mathbb{E}[\exp(it(\mathbf{Y} - c\mathbf{X})) = \exp(it \cdot c\boldsymbol{x})\varphi_{\mathbf{Y}-c\mathbf{X}}(t) = \varphi_{\mathbf{Y}-c\mathbf{X}+c\boldsymbol{x}}(t), \tag{266}$$

which means that

$$\varphi_{\mathbf{Y}|\mathbf{X}=\boldsymbol{x}}(t) = \varphi_{\mathbf{Y}-c\mathbf{X}+c\boldsymbol{x}}(t), \tag{267}$$

and the conditional distribution $p_{\mathbf{Y}|\mathbf{X}}(\boldsymbol{y}|\boldsymbol{x})$ coincides with the distribution $p_{\mathbf{Y}-c\mathbf{X}+c\boldsymbol{x}}(\boldsymbol{y})$. Thus, the conditional distribution is Gaussian with the following parameters:

$$\mathcal{N}\left(c\boldsymbol{x} + \mathbb{E}\mathbf{Y} - c\mathbb{E}\mathbf{X}, \mathrm{Var}\mathbf{Y} - \frac{\mathrm{cov}(\mathbf{X}, \mathbf{Y})^2}{\mathrm{Var}\mathbf{X}}\right), \tag{268}$$

which is equal to

$$\mathcal{N}\left(\frac{\mathrm{cov}(\mathbf{X}, \mathbf{Y})}{\mathrm{Var}\mathbf{X}}\left(\boldsymbol{x} - \mathbb{E}\mathbf{X}\right) + \mathbb{E}\mathbf{Y}, \mathrm{Var}\mathbf{Y} - \frac{\mathrm{cov}(\mathbf{X}, \mathbf{Y})^2}{\mathrm{Var}\mathbf{X}}\right). \tag{269}$$

□

This proof illustrates the very common method of calculating conditional expectations via orthogonalization of the Gaussian vector and application of the characteristic function as the tool for defining the distribution. Finally, the formula tells us that the expected value of the conditional distribution (which coincides with the conditional expectation) is equal to

$$\mathbb{E}[\mathbf{Y}|\mathbf{X}] = \frac{\mathrm{cov}(\mathbf{X}, \mathbf{Y})}{\mathrm{Var}\mathbf{X}} (\mathbf{X} - \mathbb{E}\mathbf{X}) + \mathbb{E}\mathbf{Y}, \tag{270}$$

which in the case of centered variables $\mathbb{E}\mathbf{X} = \mathbb{E}\mathbf{Y} = 0$ is equal to

$$\mathbb{E}[\mathbf{Y}|\mathbf{X}] = \frac{\mathbb{E}[\mathbf{X} \cdot \mathbf{Y}]}{\mathbb{E}[\mathbf{X} \cdot \mathbf{X}]}\mathbf{X}. \tag{271}$$

This means that one can calculate the L_2 projection (the conditional expectation) using the same formula as for the $\mathbb{R}^d$ projection

$$\mathrm{proj}_{\boldsymbol{x}}(\boldsymbol{y}) = \frac{\langle \boldsymbol{x}, \boldsymbol{y} \rangle}{\langle \boldsymbol{x}, \boldsymbol{x} \rangle}\boldsymbol{x} \tag{272}$$

with the only exception that instead of the Euclidian scalar product one should calculate the L_2 scalar product $\langle \mathbf{X}, \mathbf{Y} \rangle_{L_2} = \mathbb{E}[\mathbf{X} \cdot \mathbf{Y}]$.

In the context of diffusion models, this formula is a very useful tool for calculating conditional distributions corresponding to the forward noising process e.g. if

$$\mathbf{X}_t = \sqrt{1 - \beta_t}\mathbf{X}_{t-1} + \sqrt{\beta_t}\boldsymbol{\varepsilon}_t, \tag{273}$$

one could use it to find the conditional distribution $p_{\mathbf{X}_{t-1}|\mathbf{X}_t, \mathbf{X}_0}(\boldsymbol{x}_{t-1}|\boldsymbol{x}_t, \boldsymbol{x}_0)$, essential for defining the sampling process in discrete-time diffusion models.

References

[1] Michael S Albergo, Mark Goldstein, Nicholas M Boffi, Rajesh Ranganath, and Eric Vanden-Eijnden. Stochastic interpolants with data-dependent couplings. *arXiv preprint arXiv:2310.03725*, 2023.

[2] Michael S Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. *arXiv preprint arXiv:2209.15571*, 2022.

[3] Jean-David Benamou and Yann Brenier. A computational fluid mechanics solution to the monge-kantorovich mass transfer problem. *Numerische Mathematik*, 84(3):375–393, 2000.

[4] Jaemoo Choi, Yongxin Chen, and Jaewoong Choi. Improving neural optimal transport via displacement interpolation. *arXiv preprint arXiv:2410.03783*, 2024.

[5] Laurent Dinh, Jascha Sohl-Dickstein, and Samy Bengio. Density estimation using real nvp. *arXiv preprint arXiv:1605.08803*, 2016.

[6] Vage Egiazarian, Denis Kuznedelev, Anton Voronov, Ruslan Svirschevski, Michael Goin, Daniil Pavlov, Dan Alistarh, and Dmitry Baranchuk. Accurate compression of text-to-image diffu-sion models via vector quantization. *arXiv preprint arXiv:2409.00492*, 2024.

[7] Vage Egiazarian, Andrei Panferov, Denis Kuznedelev, Elias Frantar, Artem Babenko, and Dan Alistarh. Extreme compression of large language models via additive quantization. *arXiv preprint arXiv:2401.06118*, 2024.

[8] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. *arXiv preprint arXiv:2403.03206*, 2024.

[9] Elias Frantar, Saleh Ashkboos, Torsten Hoefler, and Dan Alistarh. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*, 2022.

[10] Geoffrey Hinton. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

[11] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[12] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A Efros. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1125–1134, 2017.

[13] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *Advances in Neural Information Processing Systems*, 35:26565–26577, 2022.

[14] Dongjun Kim, Chieh-Hsin Lai, Wei-Hsiang Liao, Naoki Murata, Yuhta Takida, Toshimitsu Uesaka, Yutong He, Yuki Mitsufuji, and Stefano Ermon. Consistency trajectory models: Learning probability flow ode trajectory of diffusion. *arXiv preprint arXiv:2310.02279*, 2023.

[15] Durk P Kingma and Prafulla Dhariwal. Glow: Generative flow with invertible 1x1 convolutions. *Advances in neural information processing systems*, 31, 2018.

[16] Alexander Korotin, Daniil Selikhanovych, and Evgeny Burnaev. Neural optimal transport. *arXiv preprint arXiv:2201.12220*, 2022.

[17] Christian Léonard. Feynman-kac formula under a finite entropy condition. *Probability Theory and Related Fields*, 184(3):1029–1091, 2022.

[18] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[19] Guan-Horng Liu, Arash Vahdat, De-An Huang, Evangelos A Theodorou, Weili Nie, and Anima Anandkumar. I^{2}sb: Image-to-image schr\" odinger bridge. *arXiv preprint arXiv:2302.05872*, 2023.

[20] Qiang Liu. Rectified flow: A marginal preserving approach to optimal transport. *arXiv preprint arXiv:2209.14577*, 2022.

[21] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.

[22] Xingchao Liu, Xiwen Zhang, Jianzhu Ma, Jian Peng, et al. Instaflow: One step is enough for high-quality diffusion-based text-to-image generation. In *The Twelfth International Conference on Learning Representations*, 2023.

[23] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. Dpm-solver: A fast ode solver for diffusion probabilistic model sampling in around 10 steps. *Advances in Neural Information Processing Systems*, 35:5775–5787, 2022.

[24] Weijian Luo, Tianyang Hu, Shifeng Zhang, Jiacheng Sun, Zhenguo Li, and Zhihua Zhang. Diff-instruct: A universal approach for transferring knowledge from pre-trained diffusion models. *Advances in Neural Information Processing Systems*, 36, 2024.

[25] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Deepcache: Accelerating diffusion models for free. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 15762–15772, 2024.

[26] Petr Mokrov, Alexander Korotin, Alexander Kolesov, Nikita Gushchin, and Evgeny Burnaev. Energy-guided entropic neural optimal transport. *arXiv preprint arXiv:2304.06094*, 2023.

[27] Kirill Neklyudov, Rob Brekelmans, Daniel Severo, and Alireza Makhzani. Action matching: Learning stochastic dynamics from samples. In *International conference on machine learning*, pages 25858–25889. PMLR, 2023.

[28] Kirill Neklyudov, Rob Brekelmans, Alexander Tong, Lazar Atanackovic, Qiang Liu, and Alireza Makhzani. A computational framework for solving wasserstein lagrangian flows. *arXiv preprint arXiv:2310.10649*, 2023.

[29] Thuan Hoang Nguyen and Anh Tran. Swiftbrush: One-step text-to-image diffusion model with variational score distillation. *arXiv preprint arXiv:2312.05239*, 2023.

[30] Zbigniew Palmowski and Tomasz Rolski. A technique for exponential change of measure for markov processes. 2002.

[31] Stefano Peluchetti. Non-denoising forward-time diffusions. *arXiv preprint arXiv:2312.14589*, 2023.

[32] Ben Poole, Ajay Jain, Jonathan T Barron, and Ben Mildenhall. Dreamfusion: Text-to-3d using 2d diffusion. *arXiv preprint arXiv:2209.14988*, 2022.

[33] Denis Rakitin, Ivan Shchekotov, and Dmitry Vetrov. Regularized distribution matching distillation for one-step unpaired image-to-image translation. *arXiv preprint arXiv:2406.14762*, 2024.

[34] Amirmojtaba Sabour, Sanja Fidler, and Karsten Kreis. Align your steps: Optimizing sampling schedules in diffusion models. *arXiv preprint arXiv:2404.14507*, 2024.

[35] Tim Salimans and Jonathan Ho. Progressive distillation for fast sampling of diffusion models. *arXiv preprint arXiv:2202.00512*, 2022.

[36] Tim Salimans, Thomas Mensink, Jonathan Heek, and Emiel Hoogeboom. Multistep distillation of diffusion models via moment matching. *arXiv preprint arXiv:2406.04103*, 2024.

[37] Yuyang Shi, Valentin De Bortoli, Andrew Campbell, and Arnaud Doucet. Diffusion schrödinger bridge matching. *Advances in Neural Information Processing Systems*, 36, 2024.

[38] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. pmlr, 2015.

[39] Vignesh Ram Somnath, Matteo Pariset, Ya-Ping Hsieh, Maria Rodriguez Martinez, Andreas Krause, and Charlotte Bunne. Aligned diffusion schrödinger bridges. In *Uncertainty in Artificial Intelligence*, pages 1985–1995. PMLR, 2023.

[40] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. *arXiv preprint arXiv:2303.01469*, 2023.

[41] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019.

[42] Joshua B Tenenbaum, Vin de Silva, and John C Langford. A global geometric framework for nonlinear dimensionality reduction. *science*, 290(5500):2319–2323, 2000.

[43] Alexander Tong, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-Brooks, Kilian Fatras, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.

[44] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23:1661–1674, 2011.

[45] Zhengyi Wang, Cheng Lu, Yikai Wang, Fan Bao, Chongxuan Li, Hang Su, and Jun Zhu. Prolificdreamer: High-fidelity and diverse text-to-3d generation with variational score distillation. *Advances in Neural Information Processing Systems*, 36, 2024.

[46] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.

[47] Felix Wimbauer, Bichen Wu, Edgar Schoenfeld, Xiaoliang Dai, Ji Hou, Zijian He, Artsiom Sanakoyeu, Peizhao Zhang, Sam Tsai, Jonas Kohler, et al. Cache me if you can: Accelerating diffusion models through block caching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6211–6220, 2024.

[48] Shuchen Xue, Zhaoqiang Liu, Fei Chen, Shifeng Zhang, Tianyang Hu, Enze Xie, and Zhenguo Li. Accelerating diffusion sampling with optimized time steps. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 8292–8301, 2024.

[49] Tianwei Yin, Michaël Gharbi, Taesung Park, Richard Zhang, Eli Shechtman, Fredo Durand, and William T Freeman. Improved distribution matching distillation for fast image synthesis. *arXiv preprint arXiv:2405.14867*, 2024.

[50] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Fredo Durand, William T Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. *arXiv preprint arXiv:2311.18828*, 2023.

[51] Qinsheng Zhang and Yongxin Chen. Fast sampling of diffusion models with exponential integrator. *arXiv preprint arXiv:2204.13902*, 2022.

[52] Wenliang Zhao, Lujia Bai, Yongming Rao, Jie Zhou, and Jiwen Lu. Unipc: A unified predictor-corrector framework for fast sampling of diffusion models. *Advances in Neural Information Processing Systems*, 36, 2024.

[53] Mingyuan Zhou, Huangjie Zheng, Zhendong Wang, Mingzhang Yin, and Hai Huang. Score identity distillation: Exponentially fast distillation of pretrained diffusion models for one-step generation. In *Forty-first International Conference on Machine Learning*, 2024.